

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



**BÁO CÁO BÀI TẬP LỚN MÔN HỌC CÁC HỆ
THỐNG PHÂN TÁN**

Giảng viên : TS.Kim Ngọc Bách

Đề tài : Hệ thống quản lý nhiệm vụ và dự án (TaskFlow)

Nhóm học phần : 02

Nhóm đề tài: 06

Họ và tên - MSV : Bùi Công Hậu - B22DCCN283

Tạ Duy Hiếu - B22DCCN319

Lê Văn Tường - B22DCCN775

Hà Nội – 2026

MỤC LỤC

MỤC LỤC.....	1
LỜI CẢM ƠN.....	4
DANH MỤC BẢNG BIỂU.....	5
DANH MỤC HÌNH ẢNH.....	6
THUẬT NGỮ VIẾT TẮT.....	7
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	9
1.1. Đặt vấn đề.....	9
1.2. Giới thiệu bài toán.....	10
1.3. Mục tiêu đề tài.....	12
1.4. Phạm vi đề tài.....	13
1.4.1. Phạm vi thực hiện.....	14
1.4.2. Phạm vi không thực hiện.....	15
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	16
2.1. Kiến trúc Microservice.....	16
2.1.1. Khái niệm Microservice.....	16
2.1.2. Đặc điểm.....	17
2.1.3. Ưu nhược điểm.....	18
2.2. RESTful API.....	19
2.2.1. REST Architecture.....	19
2.2.2. HTTP Methods.....	20
2.2.3. JSON Data Exchange.....	20
2.2.4. HTTP Status Code.....	21
2.3. RabbitMQ và Event-Driven Architecture.....	21
2.3.1. Khái niệm Message Queue.....	21
2.3.2. RabbitMQ.....	22
2.3.3. Exchange – Queue – Routing Key.....	23
2.3.4. Publish/Subscribe.....	24
2.3.5. Eventual Consistency.....	24
2.4. Spring Cloud.....	25
2.4.1. Eureka Service Discovery.....	26
2.4.2. Spring Cloud Gateway.....	26
2.4.3. Spring Cloud Config.....	27
2.5. JWT Authentication.....	27
2.6. WebSocket Realtime.....	29
2.7. Redis Cache.....	29
2.8. Resilience4j.....	30
2.9. Saga Pattern.....	31
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	32
3.1. Phân tích yêu cầu hệ thống.....	32

3.1.1. Yêu cầu chức năng.....	32
3.1.2. Yêu cầu phi chức năng.....	35
3.2. Kiến trúc tổng thể hệ thống.....	36
3.3. Thiết kế Microservices.....	37
3.3.1. User Service.....	37
3.3.2. Project Service.....	40
3.3.3. Task Service.....	48
3.3.4. Collaboration Service.....	54
3.3.5. Notification Service.....	57
3.4. Thiết kế cơ sở dữ liệu.....	58
3.4.1. Nguyên tắc thiết kế Database-per-Service.....	58
3.4.2. Lớp AuditEntity dùng chung.....	59
3.4.3. Cơ sở dữ liệu User Service.....	60
3.4.4. Cơ sở dữ liệu Project Service.....	61
3.4.5. Cơ sở dữ liệu Task Service.....	63
3.4.6. Cơ sở dữ liệu Collaboration Service.....	65
3.4.7. Cơ sở dữ liệu Notification Service.....	66
3.4.8. Quan hệ logic xuyên service.....	67
3.5. Thiết kế giao tiếp REST.....	69
3.5.1. Hai dạng giao tiếp trong hệ thống TaskFlow.....	69
3.5.2. Phân loại API trong hệ thống.....	70
3.5.3. Các luồng giao tiếp REST nội bộ.....	70
3.5.4. Cơ chế gọi qua Eureka và bảo vệ bằng Resilience4j.....	72
3.5.5. Chiến lược cache để giảm tải REST.....	73
3.6. Thiết kế Message Queue.....	74
3.6.1. Kiến trúc RabbitMQ trong hệ thống.....	74
3.6.2. Thiết kế Exchange — Lý do chọn Topic Exchange.....	75
3.6.3. Quy ước đặt khóa định tuyến.....	76
3.6.4. Cấu trúc Envelope chung của sự kiện.....	77
3.6.5. Quy trình một sự kiện đầu cuối — Ví dụ Tạo công việc.....	78
3.6.6. Cơ chế đảm bảo độ tin cậy.....	80
3.6.7. Quản lý phiên bản schema sự kiện.....	81
3.7. Thiết kế WebSocket Realtime.....	81
3.7.1. Đặt vấn đề — Tại sao cần truyền thông realtime.....	81
3.7.2. Lựa chọn công nghệ — WebSocket kết hợp STOMP.....	81
3.7.3. Cấu hình endpoint và message broker.....	81
3.7.4. Cơ chế xác thực qua handshake.....	82
3.7.5. Hai loại destination chính.....	82
3.8. Thiết kế phân quyền.....	83
3.8.1. Mô hình phân quyền RBAC theo dự án.....	83

3.8.2. Hệ thống năm vai trò trong dự án và hai vai trò toàn cục.....	83
3.8.3. Cơ chế kiểm tra hai cấp.....	84
3.8.4. Ma trận quyền chi tiết theo Service.....	84
3.8.5. Triển khai kiểm tra phân quyền trong code.....	88
3.8.6. Đồng bộ vai trò qua sự kiện.....	88
3.8.7. Các tình huống biên trong phân quyền.....	89
3.8.8. Tổng kết.....	90
CHƯƠNG 4. XÂY DỰNG HỆ THỐNG.....	90
4.1. Môi trường phát triển.....	90
4.2. Xây dựng API Gateway.....	92
4.3. Xây dựng Eureka Service Discovery.....	93
4.4. Xây dựng User Service.....	93
4.5. Xây dựng RabbitMQ Integration.....	94
4.6. Xây dựng WebSocket Realtime.....	95
4.7. Docker Compose Deployment.....	96
CHƯƠNG 5. DEMO VÀ ĐÁNH GIÁ.....	99
5.1 Giao diện hệ thống.....	99
5.1.1 Giao diện đăng ký đăng nhập.....	99
5.1.2 Giao diện dashboard.....	100
5.1.3 Giao diện quản lý dự án.....	100
5.1.4 Giao diện quản lý task.....	101
5.2 Đánh giá hệ thống.....	101
5.2.1 Ưu điểm.....	101
5.2.2 Nhược điểm.....	102
KẾT LUẬN.....	103
HƯỚNG PHÁT TRIỂN.....	105
TÀI LIỆU THAM KHẢO.....	107

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án môn học “Phát triển ứng dụng Microservice”, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ, hướng dẫn và động viên từ phía giảng viên, nhà trường và các thành viên trong nhóm.

Trước hết, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc tới cô – giảng viên hướng dẫn môn học. Thầy đã tận tình hướng dẫn, định hướng chuyên môn và hỗ trợ nhóm trong suốt quá trình nghiên cứu và thực hiện đề tài. Những góp ý của cô không chỉ giúp nhóm hoàn thiện nội dung báo cáo mà còn giúp chúng em hiểu rõ hơn về kiến trúc Microservice, hệ thống phân tán và quy trình xây dựng một ứng dụng hiện đại theo hướng service-oriented.

Nhóm cũng xin chân thành cảm ơn Học viện Công nghệ Bưu chính Viễn thông cùng các thầy cô trong Khoa đã tạo điều kiện thuận lợi về môi trường học tập, tài liệu và kiến thức nền tảng để nhóm có thể tiếp cận và nghiên cứu các công nghệ liên quan đến Microservice Architecture, RESTful API, Message Queue và các hệ thống realtime hiện đại.

Trong quá trình thực hiện đề tài, nhóm đã có cơ hội vận dụng các kiến thức về thiết kế hệ thống phân tán, giao tiếp giữa các service, xử lý bất đồng bộ với Message Queue, xác thực JWT, WebSocket realtime và triển khai hệ thống bằng Docker. Đây là một trải nghiệm có giá trị thực tiễn lớn, giúp các thành viên nâng cao kỹ năng nghiên cứu, tư duy hệ thống và khả năng làm việc nhóm.

Mặc dù nhóm đã cố gắng hoàn thiện đề tài trong phạm vi kiến thức và thời gian cho phép, báo cáo chắc chắn vẫn còn những hạn chế và thiếu sót nhất định. Nhóm rất mong nhận được những ý kiến đóng góp và nhận xét từ cô để có thể tiếp tục hoàn thiện hơn trong các nghiên cứu và dự án tiếp theo.

Nhóm chúng em xin chân thành cảm ơn.

Nhóm sinh viên thực hiện

Bùi Công Hậu

Lê Văn Tường

Tạ Duy Hiếu

DANH MỤC BẢNG BIỂU

Bảng 3.1.1 Danh mục API của User Service.....	35
Bảng 3.3.2.1: Chi tiết các API quản lý Dự án.....	39
Bảng 3.3.2.2: Chi tiết các API quản lý Thành viên.....	40
Bảng 3.3.2.3: Chi tiết các API quản lý Boards và Lists.....	41
Bảng 3.3.2.4: Chi tiết các API Sprints và Kết xuất Báo cáo.....	43
Bảng 3.3.2.5: Chi tiết các API giao tiếp nội bộ hệ thống.....	45
Bảng 3.3.3 Danh mục API của Task Service.....	50
Bảng 3.3.4 Danh mục API của Collaboration Service.....	53
Bảng 3.3.5 Danh mục API của Notification Service.....	55
Bảng 3.4.1. Danh sách các cơ sở dữ liệu của hệ thống TaskFlow.....	56
Bảng 3.4.2. Các cột kỹ thuật chuẩn của lớp AuditEntity.....	57
Bảng 3.4.8. Cơ chế đảm bảo nhất quán cho tham chiếu xuyên service.....	67
Bảng 3.5.1. So sánh hai dạng giao tiếp trong TaskFlow.....	68
Bảng 3.5.2. Các loại API trong hệ thống TaskFlow.....	69
Bảng 3.5.3. Các luồng REST nội bộ giữa các service.....	70
Bảng 3.5.4. Cấu hình Resilience4j cho REST gọi nội bộ.....	71
Bảng 3.5.5. Các khóa cache Redis được dùng để giảm tải REST nội bộ.....	72
Bảng 3.6.1. Các hàng đợi chính và service tiêu thụ tương ứng.....	73
Bảng 3.6.2. So sánh các loại Exchange của RabbitMQ.....	74
Bảng 3.6.3. Danh mục đầy đủ các sự kiện trong hệ thống.....	75
Bảng 3.6.4. Cấu trúc envelope chung của sự kiện.....	77
Bảng 3.6.5. Các cơ chế bảo đảm độ tin cậy của truyền tin.....	79
Bảng 3.7.5. Các destination WebSocket trong hệ thống.....	81
Bảng 3.8.2. Năm vai trò trong phạm vi dự án.....	82
Bảng 3.8.4.1. Bảng ma trận quyền cho các thao tác trên Project và Member.....	83
Bảng 3.8.4.2 Bảng ma trận quyền cho các thao tác trên Board và List.....	85
Bảng 3.8.4.3 Bảng Ma trận quyền cho các thao tác trên Task.....	85
Bảng 3.8.5. Ma trận quyền cho các thao tác trên Comment và Attachment.....	86
Bảng 3.8.7. Các tình huống biên trong phân quyền và cách xử lý.....	88
Bảng 4.1 Danh mục Đặc tả Công nghệ hệ thống TaskFlow.....	89

DANH MỤC HÌNH ẢNH

Hình 3.2 Kiến trúc hệ thống.....	37
Hình 3.4.3. Sơ đồ thực thể-quan hệ — User Service (taskflow_user).....	62
Hình 3.4.4. Sơ đồ thực thể-quan hệ — Project Service (taskflow_project).....	63
Hình 3.4.5. Sơ đồ thực thể-quan hệ — Task Service (taskflow_task).....	65
Hình 3.4.6. Sơ đồ thực thể-quan hệ — Collaboration Service (taskflow_collab).....	67
Hình 3.4.8. Sơ đồ tham chiếu logic xuyên service của hệ thống TaskFlow.....	68
Hình 4.5: Sơ đồ cấu trúc mạng (Topology) của RabbitMQ.....	95
Hình 4.7.2 Sơ đồ triển khai trên Docker.....	98
Hình 5.1.1.1 Giao diện đăng nhập.....	99
Hình 5.1.1.2 Giao diện đăng ký.....	100
Hình 5.1.2 Giao diện dashboard.....	100
Hình 5.1.3 Quản lý dự án.....	101
Hình 5.1.4 Giao diện quản lý task.....	101

THUẬT NGỮ VIẾT TẮT

STT	Từ viết tắt	Tên đầy đủ	Ý nghĩa
1	API	Application Programming Interface	Giao diện lập trình ứng dụng
2	REST	Representational State Transfer	Kiến trúc giao tiếp dịch vụ web
3	JWT	JSON Web Token	Chuẩn xác thực bằng token
4	HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
5	JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu
6	MQ	Message Queue	Hàng đợi thông điệp
7	DB	Database	Cơ sở dữ liệu
8	RDBMS	Relational Database Management System	Hệ quản trị cơ sở dữ liệu quan hệ
9	UI	User Interface	Giao diện người dùng
10	CRUD	Create, Read, Update, Delete	Các thao tác dữ liệu cơ bản
11	TTL	Time To Live	Thời gian tồn tại của cache
12	RPC	Remote Procedure Call	Gọi hàm từ xa
13	DFS	Depth-First Search	Thuật toán duyệt theo chiều sâu
14	CQRS	Command Query Responsibility Segregation	Mô hình tách biệt đọc và ghi
15	ACID	Atomicity, Consistency, Isolation, Durability	Tính chất giao dịch trong CSDL
16	TCP	Transmission Control Protocol	Giao thức truyền tải TCP

17	STOMP	Simple Text Oriented Messaging Protocol	Giao thức nhắn tin cho WebSocket
18	ELK	Elasticsearch, Logstash, Kibana	Bộ công cụ centralized logging
19	CI/CD	Continuous Integration / Continuous Deployment	Tích hợp và triển khai liên tục
20	DDD	Domain-Driven Design	Thiết kế hướng miền nghiệp vụ

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Đặt vấn đề

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và xu hướng chuyển đổi số, các hệ thống quản lý công việc và cộng tác trực tuyến ngày càng được sử dụng rộng rãi trong doanh nghiệp, tổ chức và nhóm phát triển phần mềm. Những nền tảng như Trello hay Jira đã cho thấy nhu cầu rất lớn về các hệ thống hỗ trợ quản lý dự án, theo dõi tiến độ công việc, cộng tác nhóm và cập nhật dữ liệu theo thời gian thực.

Trước đây, phần lớn các ứng dụng được xây dựng theo kiến trúc Monolithic (nguyên khối), trong đó toàn bộ chức năng của hệ thống được triển khai trong một ứng dụng duy nhất. Kiến trúc này có ưu điểm là đơn giản trong giai đoạn phát triển ban đầu, dễ triển khai và quản lý khi hệ thống còn nhỏ. Tuy nhiên, khi quy mô hệ thống ngày càng mở rộng, mô hình Monolithic bắt đầu bộc lộ nhiều hạn chế.

Một trong những hạn chế lớn nhất của kiến trúc Monolithic là khó mở rộng hệ thống. Khi số lượng người dùng tăng cao, toàn bộ ứng dụng phải được scale đồng thời, kể cả những module không cần thiết, dẫn đến lãng phí tài nguyên và chi phí vận hành lớn. Bên cạnh đó, việc triển khai (deploy) một thay đổi nhỏ cũng yêu cầu build và triển khai lại toàn bộ hệ thống, làm tăng nguy cơ phát sinh lỗi và ảnh hưởng đến các chức năng khác đang hoạt động.

Ngoài ra, trong mô hình Monolithic, các module thường phụ thuộc chặt chẽ vào nhau, khiến cho việc bảo trì và phát triển trở nên khó khăn. Khi hệ thống ngày càng phức tạp, việc chỉnh sửa hoặc bổ sung tính năng mới có thể ảnh hưởng đến nhiều thành phần khác nhau. Điều này làm giảm tính linh hoạt và gây khó khăn cho việc phát triển song song giữa các nhóm lập trình viên.

Đối với các hệ thống quản lý công việc hiện đại, yêu cầu về khả năng mở rộng, tính sẵn sàng cao và cập nhật dữ liệu theo thời gian thực ngày càng trở nên quan trọng. Người dùng cần có khả năng nhận thông báo ngay lập tức khi nhiệm vụ được giao, khi có bình luận mới hoặc khi trạng thái công việc thay đổi. Đồng thời, hệ thống cần hỗ trợ nhiều người dùng truy cập đồng thời mà vẫn đảm bảo hiệu năng và tính ổn định.

Để giải quyết những vấn đề trên, kiến trúc Microservice đang trở thành xu hướng phổ biến trong phát triển phần mềm hiện đại. Thay vì xây dựng toàn bộ hệ thống dưới dạng một khối duy nhất, Microservice chia hệ thống thành nhiều dịch vụ nhỏ, độc lập, mỗi

service phụ trách một chức năng riêng biệt và có thể triển khai độc lập. Kiến trúc này giúp tăng khả năng mở rộng, dễ bảo trì, hỗ trợ phát triển song song và nâng cao khả năng chịu lỗi của hệ thống.

Trong kiến trúc Microservice, các service thường giao tiếp với nhau thông qua RESTful API để xử lý các yêu cầu đồng bộ (synchronous communication). RESTful API cho phép trao đổi dữ liệu một cách linh hoạt, rõ ràng và dễ tích hợp giữa các thành phần trong hệ thống.

Bên cạnh giao tiếp đồng bộ, các hệ thống hiện đại còn áp dụng mô hình Event-Driven Architecture nhằm hỗ trợ giao tiếp bất đồng bộ (asynchronous communication). Trong mô hình này, các service trao đổi dữ liệu thông qua Message Queue như RabbitMQ hoặc Apache Kafka. Khi một sự kiện xảy ra, service phát sinh sự kiện sẽ gửi message vào hàng đợi, các service khác có thể lắng nghe và xử lý mà không cần phụ thuộc trực tiếp vào nhau. Cách tiếp cận này giúp hệ thống giảm coupling, tăng khả năng mở rộng và cải thiện hiệu năng xử lý.

Xuất phát từ những yêu cầu thực tiễn đó, nhóm lựa chọn xây dựng hệ thống “TaskFlow – Hệ thống Quản lý Nhiệm vụ và Dự án” theo kiến trúc Microservice, sử dụng RESTful API kết hợp Message Queue nhằm xây dựng một hệ thống có khả năng mở rộng tốt, hỗ trợ realtime và đáp ứng các yêu cầu của một hệ thống quản lý công việc hiện đại.

1.2. Giới thiệu bài toán

Trong môi trường làm việc hiện đại, đặc biệt là trong các doanh nghiệp công nghệ và nhóm phát triển phần mềm, nhu cầu quản lý công việc, theo dõi tiến độ dự án và cộng tác nhóm ngày càng trở nên quan trọng. Việc quản lý thủ công bằng bảng tính hoặc các công cụ rời rạc thường gây khó khăn trong việc theo dõi tiến độ, phân chia nhiệm vụ và trao đổi thông tin giữa các thành viên. Điều này dẫn đến giảm hiệu quả làm việc, khó kiểm soát tiến độ dự án và dễ phát sinh sai sót trong quá trình phối hợp.

Để giải quyết các vấn đề trên, nhóm xây dựng hệ thống TaskFlow – Hệ thống Quản lý Nhiệm vụ và Dự án theo kiến trúc Microservice. Hệ thống được phát triển nhằm hỗ trợ cá nhân và nhóm làm việc quản lý dự án, phân chia nhiệm vụ, theo dõi tiến độ và cộng tác trực tuyến theo thời gian thực.

TaskFlow là nền tảng quản lý công việc được xây dựng theo mô hình kết hợp giữa bảng Kanban và quản lý sprint, lấy cảm hứng từ các hệ thống quản lý dự án phổ biến như Trello và Jira. Hệ thống hỗ trợ tổ chức công việc trực quan, dễ theo dõi và phù hợp với quy trình làm việc Agile/Scrum.

Hệ thống cho phép người dùng tạo và quản lý nhiều dự án khác nhau. Trong mỗi dự án, người dùng có thể tạo các Board Kanban để tổ chức công việc theo từng cột trạng thái như “To Do”, “In Progress” và “Done”. Các nhiệm vụ (Task) có thể được tạo, chỉnh sửa, gán cho thành viên cụ thể và di chuyển giữa các cột bằng thao tác kéo thả nhằm phản ánh tiến độ thực tế của công việc.

Ngoài các chức năng quản lý nhiệm vụ cơ bản, hệ thống còn hỗ trợ quản lý Sprint và Backlog nhằm phục vụ mô hình phát triển Agile. Người dùng có thể tạo Sprint, phân chia nhiệm vụ theo từng giai đoạn phát triển và theo dõi tiến độ hoàn thành của dự án.

Mỗi Task trong hệ thống bao gồm nhiều thông tin chi tiết như tiêu đề, mô tả, mức độ ưu tiên, thời hạn hoàn thành, nhãn màu và người thực hiện. Đồng thời, hệ thống hỗ trợ Checklist giúp chia nhỏ công việc thành các bước cụ thể để dễ dàng theo dõi tiến độ thực hiện.

Bên cạnh đó, TaskFlow còn tích hợp chức năng Comment và Attachment nhằm tăng khả năng cộng tác giữa các thành viên trong nhóm. Người dùng có thể trao đổi trực tiếp trên từng Task, đính kèm tài liệu hoặc hình ảnh liên quan đến công việc đang thực hiện.

Một trong những chức năng quan trọng của hệ thống là cơ chế Notification Realtime. Khi có thay đổi như giao nhiệm vụ mới, cập nhật trạng thái công việc hoặc thêm bình luận, hệ thống sẽ gửi thông báo theo thời gian thực thông qua WebSocket để người dùng có thể theo dõi ngay lập tức mà không cần tải lại trang.

Ngoài ra, hệ thống còn hỗ trợ cơ chế phân quyền theo nhóm nhằm đảm bảo tính bảo mật và kiểm soát truy cập. Mỗi thành viên trong dự án sẽ được gán vai trò khác nhau như Owner, Admin, Editor, Commenter hoặc Viewer, từ đó xác định quyền thao tác tương ứng trên hệ thống.

Để đáp ứng yêu cầu về khả năng mở rộng, tính độc lập giữa các chức năng và hỗ trợ giao tiếp realtime, hệ thống được thiết kế theo kiến trúc Microservice với nhiều service độc lập. Các service giao tiếp đồng bộ thông qua RESTful API và giao tiếp bất

đồng bộ thông qua Message Queue sử dụng RabbitMQ. Kiến trúc này giúp hệ thống dễ dàng mở rộng, tăng khả năng chịu lỗi và phù hợp với các hệ thống phân tán hiện đại.

1.3. Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng hệ thống TaskFlow – Hệ thống Quản lý Nhiệm vụ và Dự án theo kiến trúc Microservice nhằm đáp ứng các yêu cầu về khả năng mở rộng, tính linh hoạt, khả năng chịu lỗi và hỗ trợ giao tiếp thời gian thực trong các hệ thống quản lý công việc hiện đại.

Hệ thống được thiết kế theo mô hình phân tán, trong đó các chức năng được tách thành nhiều service độc lập để dễ dàng phát triển, triển khai và bảo trì. Thông qua đề tài này, nhóm hướng tới việc vận dụng các công nghệ và mô hình kiến trúc hiện đại trong quá trình xây dựng hệ thống phần mềm.

Các mục tiêu cụ thể của đề tài bao gồm:

Xây dựng hệ thống theo kiến trúc Microservice, trong đó mỗi service đảm nhận một miền chức năng riêng biệt và có thể hoạt động độc lập.

- Thiết kế và triển khai tối thiểu 5 microservice bao gồm:
 - + User Service
 - + Project Service
 - + Task Service
 - + Collaboration Service
 - + Notification Service
- Sử dụng RESTful API làm cơ chế giao tiếp đồng bộ (Synchronous Communication) giữa các service và giữa client với hệ thống, đảm bảo việc trao đổi dữ liệu rõ ràng, linh hoạt và dễ mở rộng.
- Áp dụng cơ chế giao tiếp bất đồng bộ (Asynchronous Communication) thông qua Message Queue sử dụng RabbitMQ nhằm xây dựng mô hình Event-Driven Architecture, giúp giảm sự phụ thuộc trực tiếp giữa các service và tăng khả năng chịu tải của hệ thống.
- Triển khai cơ chế Service Discovery bằng Netflix Eureka để các service có thể tự động đăng ký và tìm kiếm lẫn nhau trong môi trường phân tán.

- Xây dựng API Gateway nhằm đóng vai trò cổng giao tiếp trung tâm, thực hiện các chức năng như định tuyến request, xác thực người dùng, giới hạn truy cập và quản lý giao tiếp giữa client với các service bên trong hệ thống.
- Áp dụng cơ chế xác thực và phân quyền bằng JWT (JSON Web Token) nhằm đảm bảo tính bảo mật cho hệ thống và kiểm soát quyền truy cập của người dùng.
- Hỗ trợ cập nhật dữ liệu theo thời gian thực thông qua công nghệ WebSocket, cho phép người dùng nhận thông báo realtime khi có thay đổi liên quan đến nhiệm vụ, dự án hoặc hoạt động cộng tác.
- Tích hợp cơ chế Circuit Breaker, Retry và Timeout nhằm nâng cao khả năng chịu lỗi, tránh tình trạng lỗi lan truyền giữa các service khi xảy ra sự cố trong quá trình giao tiếp liên service.
- Triển khai và quản lý toàn bộ hệ thống bằng Docker Compose để hỗ trợ chạy đồng thời nhiều service, đảm bảo tính nhất quán môi trường và thuận tiện trong quá trình triển khai.
- Xây dựng hệ thống có khả năng mở rộng, dễ bảo trì và phù hợp với mô hình phát triển phần mềm hiện đại theo hướng cloud-native và distributed system.

Thông qua đề tài này, nhóm mong muốn hiểu rõ hơn về cách thiết kế và triển khai hệ thống Microservice trong thực tế, đồng thời nâng cao kỹ năng xây dựng hệ thống phân tán, giao tiếp liên service và xử lý dữ liệu realtime trong các ứng dụng hiện đại.

1.4. Phạm vi đề tài

Đề tài tập trung xây dựng hệ thống TaskFlow – Hệ thống Quản lý Nhiệm vụ và Dự án theo kiến trúc Microservice nhằm hỗ trợ quản lý công việc, cộng tác nhóm và theo dõi tiến độ dự án trong môi trường làm việc hiện đại. Hệ thống được phát triển dưới dạng ứng dụng web với các chức năng cốt lõi phục vụ quản lý dự án và giao tiếp realtime giữa các thành viên.

1.4.1. Phạm vi thực hiện

Trong phạm vi của đề tài, nhóm tập trung triển khai các chức năng chính sau:

Quản lý người dùng (User Management)

Hệ thống hỗ trợ các chức năng liên quan đến người dùng bao gồm:

- Đăng ký tài khoản

- Đăng nhập hệ thống
- Xác thực bằng JWT
- Quản lý thông tin cá nhân
- Phân quyền người dùng theo vai trò trong dự án

Người dùng sau khi đăng nhập có thể tham gia và thao tác trên các dự án tùy theo quyền được cấp.

Quản lý dự án (Project Management)

- Hệ thống cho phép người dùng:
- Tạo và quản lý dự án
- Quản lý thành viên dự án
- Quản lý Board Kanban
- Quản lý Sprint và Backlog
- Theo dõi tiến độ dự án

Mỗi dự án có thể bao gồm nhiều board và nhiều thành viên với các vai trò khác nhau như Owner, Admin, Editor, Commenter và Viewer.

Quản lý nhiệm vụ (Task Management)

Đây là chức năng cốt lõi của hệ thống, bao gồm:

- Tạo, cập nhật và xóa Task
- Gán người thực hiện
- Thiết lập mức độ ưu tiên
- Thiết lập thời hạn hoàn thành
- Quản lý Checklist
- Di chuyển Task giữa các List theo mô hình Kanban
- Tìm kiếm và lọc Task theo nhiều điều kiện

Hệ thống hỗ trợ tổ chức công việc trực quan và giúp theo dõi tiến độ thực hiện nhiệm vụ dễ dàng hơn.

Cộng tác nhóm (Collaboration)

Hệ thống hỗ trợ các chức năng cộng tác giữa các thành viên trong nhóm như:

- Bình luận trên Task
- Đính kèm file và hình ảnh
- Theo dõi lịch sử hoạt động (Activity Log)
- Theo dõi thay đổi trong dự án

Các chức năng này giúp tăng khả năng trao đổi thông tin và phối hợp làm việc giữa các thành viên.

Thông báo realtime (Realtime Notification)

Hệ thống triển khai cơ chế thông báo theo thời gian thực thông qua WebSocket nhằm:

- Thông báo khi có Task mới được giao
- Thông báo khi có bình luận mới
- Thông báo khi trạng thái Task thay đổi
- Đồng bộ dữ liệu realtime giữa các thành viên trong cùng Board

Ngoài ra, hệ thống còn hỗ trợ gửi thông báo thông qua email đối với một số sự kiện quan trọng.

1.4.2. Phạm vi không thực hiện

Do giới hạn về thời gian và phạm vi của đề án, nhóm chưa triển khai các chức năng và công nghệ nâng cao sau:

- Không tích hợp hệ thống gợi ý thông minh hoặc AI Recommendation cho việc phân công công việc hay phân tích tiến độ dự án.
- Không triển khai hệ thống trên môi trường Kubernetes production hoặc các nền tảng cloud thực tế.
- Không phát triển ứng dụng Mobile Native cho Android hoặc iOS; hệ thống hiện chỉ tập trung trên nền tảng web.
- Không triển khai các tính năng phân tích dữ liệu nâng cao hoặc Machine Learning.
- Không xây dựng hệ thống CI/CD hoàn chỉnh ở mức production.

Mặc dù còn một số giới hạn, hệ thống vẫn đáp ứng đầy đủ các yêu cầu chính của một ứng dụng Microservice hiện đại, bao gồm giao tiếp RESTful API, Message Queue, realtime communication và khả năng triển khai độc lập giữa các service.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Kiến trúc Microservice

2.1.1. Khái niệm Microservice

Microservice là một phong cách kiến trúc phần mềm trong đó hệ thống được chia thành nhiều dịch vụ nhỏ, độc lập và tập trung vào một chức năng nghiệp vụ cụ thể. Mỗi service hoạt động như một ứng dụng riêng biệt, có thể được phát triển, triển khai và mở rộng độc lập với các service khác.

Khác với kiến trúc Monolithic truyền thống – nơi toàn bộ chức năng được xây dựng trong một ứng dụng duy nhất – Microservice phân tách hệ thống thành nhiều thành phần nhỏ nhằm giảm sự phụ thuộc giữa các module. Các service giao tiếp với nhau thông qua các giao thức mạng như RESTful API, Message Queue hoặc gRPC.

Trong kiến trúc Microservice, mỗi service thường:

- Có logic nghiệp vụ riêng
- Có cơ sở dữ liệu riêng
- Chạy trên tiến trình riêng
- Được triển khai độc lập
- Giao tiếp thông qua mạng

Mục tiêu chính của Microservice là tăng khả năng mở rộng, dễ bảo trì, hỗ trợ phát triển song song và nâng cao tính linh hoạt của hệ thống. Hiện nay, kiến trúc Microservice được sử dụng phổ biến trong các hệ thống lớn và hiện đại nhờ khả năng đáp ứng tốt các yêu cầu về hiệu năng, realtime và distributed system.

2.1.2. Đặc điểm

Loose Coupling (liên kết lỏng) là đặc điểm quan trọng của kiến trúc Microservice. Các service được thiết kế với mức phụ thuộc thấp, nghĩa là một service có thể thay đổi hoặc cập nhật mà ít ảnh hưởng đến các service khác.

Các service không truy cập trực tiếp vào logic hoặc dữ liệu nội bộ của nhau mà giao tiếp thông qua API hoặc Message Queue. Điều này giúp:

- Hệ thống linh hoạt hơn
- Dễ bảo trì
- Giảm ảnh hưởng dây chuyền khi xảy ra lỗi
- Hỗ trợ phát triển độc lập giữa các nhóm

Nhờ loose coupling, việc mở rộng hoặc thay đổi một chức năng trong hệ thống trở nên đơn giản hơn so với kiến trúc nguyên khối.

Independently Deployable

Một trong những đặc điểm nổi bật của Microservice là mỗi service có thể được triển khai độc lập. Trong kiến trúc Monolithic, khi thay đổi một chức năng nhỏ, toàn bộ hệ thống thường phải build và deploy lại. Ngược lại, với Microservice, mỗi service có thể:

- Build riêng
- Deploy riêng
- Scale riêng
- Rollback riêng

Đặc điểm này giúp:

- Giảm downtime hệ thống
- Tăng tốc độ phát triển phần mềm
- Cho phép nhiều nhóm làm việc song song
- Dễ dàng cập nhật chức năng mới

Hiện nay, các microservice thường được triển khai dưới dạng container bằng Docker nhằm hỗ trợ quản lý và triển khai hiệu quả hơn.

Database per Service

Trong kiến trúc Microservice, mỗi service thường sở hữu cơ sở dữ liệu riêng thay vì dùng chung một database cho toàn hệ thống.

Mô hình “Database per Service” giúp:

- Tăng tính độc lập giữa các service
- Giảm coupling dữ liệu
- Dễ mở rộng database theo từng service
- Tránh ảnh hưởng lẫn nhau khi thay đổi cấu trúc dữ liệu

Mỗi service sẽ tự quản lý dữ liệu của mình và không truy cập trực tiếp vào database của service khác. Việc trao đổi dữ liệu được thực hiện thông qua API hoặc các cơ chế giao tiếp bất đồng bộ như Message Queue. Mặc dù mô hình này giúp hệ thống linh hoạt hơn, nhưng cũng làm tăng độ phức tạp trong việc đảm bảo tính nhất quán dữ liệu giữa các service.

2.1.3. Ưu nhược điểm

Ưu điểm

- Khả năng mở rộng tốt: Mỗi service có thể được scale độc lập tùy theo nhu cầu sử dụng. Điều này giúp tối ưu tài nguyên và nâng cao hiệu năng hệ thống.
- Dễ bảo trì và phát triển: Do hệ thống được chia thành nhiều service nhỏ nên mã nguồn của từng service sẽ rõ ràng và dễ quản lý hơn. Các nhóm phát triển có thể làm việc song song mà ít xung đột.
- Triển khai độc lập: Mỗi service có thể deploy riêng mà không cần triển khai lại toàn bộ hệ thống, giúp giảm rủi ro và tăng tốc độ phát triển.
- Tăng khả năng chịu lỗi: Khi một service gặp sự cố, các service khác vẫn có thể tiếp tục hoạt động. Điều này giúp hệ thống ổn định hơn và hạn chế lỗi lan truyền.
- Linh hoạt công nghệ: Mỗi service có thể sử dụng công nghệ hoặc database khác nhau tùy theo yêu cầu, miễn đảm bảo cơ chế giao tiếp thống nhất.

Nhược điểm

- Hệ thống phức tạp hơn: So với Monolithic Architecture, Microservice yêu cầu nhiều thành phần hạ tầng hơn như: API Gateway, Service Discovery, Message Queue, Monitoring, Distributed Tracing. Điều này làm tăng độ phức tạp trong thiết kế và vận hành.
- Khó quản lý dữ liệu phân tán: Do mỗi service có database riêng nên việc đảm bảo tính nhất quán dữ liệu trở nên khó khăn hơn.
- Khó debug và kiểm thử: Các service giao tiếp thông qua mạng nên việc theo dõi lỗi và kiểm thử hệ thống sẽ phức tạp hơn, đặc biệt trong các luồng bất đồng bộ.
- Tăng chi phí triển khai: Hệ thống Microservice thường tiêu tốn nhiều tài nguyên hơn do phải chạy nhiều service, database và thành phần hạ tầng độc lập.

Mặc dù còn tồn tại một số hạn chế, kiến trúc Microservice vẫn là xu hướng phổ biến trong phát triển phần mềm hiện đại nhờ khả năng mở rộng, tính linh hoạt và hỗ trợ tốt cho các hệ thống phân tán quy mô lớn.

2.2. RESTful API

2.2.1. REST Architecture

REST (Representational State Transfer) là phong cách kiến trúc được sử dụng phổ biến trong thiết kế các hệ thống web service hiện đại. REST được đề xuất bởi Roy

Fielding trong luận án tiến sĩ của ông vào năm 2000 và hiện nay được sử dụng rộng rãi trong các hệ thống phân tán và ứng dụng Microservice.

REST Architecture tổ chức hệ thống dựa trên các tài nguyên (Resource). Mỗi tài nguyên được định danh thông qua URI (Uniform Resource Identifier) và được thao tác thông qua các phương thức HTTP như GET, POST, PUT và DELETE.

Trong kiến trúc REST, client và server giao tiếp với nhau thông qua giao thức HTTP. Client gửi request đến server và server trả về response chứa dữ liệu hoặc trạng thái xử lý.

REST có một số đặc điểm quan trọng như:

- Stateless: Server không lưu trạng thái của client giữa các request. Mỗi request phải chứa đầy đủ thông tin cần thiết để xử lý.
- Client-Server: Tách biệt giao diện người dùng và xử lý dữ liệu giúp hệ thống dễ mở rộng và bảo trì.
- Uniform Interface: Các API được thiết kế theo quy tắc thống nhất, giúp dễ sử dụng và dễ tích hợp.
- Resource-Based: Mọi dữ liệu đều được biểu diễn dưới dạng tài nguyên và truy cập thông qua URI.

Nhờ tính đơn giản, linh hoạt và dễ tích hợp, RESTful API hiện là lựa chọn phổ biến trong các hệ thống Microservice.

2.2.2. HTTP Methods

Trong RESTful API, các thao tác với tài nguyên được thực hiện thông qua các phương thức HTTP. Mỗi phương thức mang một ý nghĩa riêng và được sử dụng tương ứng với từng chức năng xử lý dữ liệu.

- GET: phương thức GET được sử dụng để lấy dữ liệu từ server mà không làm thay đổi dữ liệu trên hệ thống. Đặc điểm: chỉ dùng để đọc dữ liệu, không thay đổi trạng thái hệ thống, có thể cache để tăng hiệu năng.
- POST: phương thức POST được sử dụng để tạo mới dữ liệu trên server. Đặc điểm: dùng để tạo tài nguyên mới, dữ liệu thường được gửi trong request body, có thể làm thay đổi trạng thái hệ thống.
- PUT: phương thức PUT được sử dụng để cập nhật dữ liệu đã tồn tại. Đặc điểm: dùng để cập nhật dữ liệu, có thể thay thế toàn bộ tài nguyên, có tính idempotent, nghĩa là gọi nhiều lần vẫn cho cùng kết quả.

- DELETE: phương thức DELETE được sử dụng để xóa dữ liệu trên server. Đặc điểm: dùng để xóa tài nguyên, làm thay đổi dữ liệu hệ thống, có thể triển khai theo kiểu hard delete hoặc soft delete.

2.2.3. JSON Data Exchange

Trong RESTful API, dữ liệu thường được trao đổi giữa client và server dưới định dạng JSON (JavaScript Object Notation).

JSON là định dạng dữ liệu dạng text có cấu trúc nhẹ, dễ đọc và dễ xử lý. JSON hiện là chuẩn phổ biến trong giao tiếp giữa các hệ thống web service nhờ tính đơn giản và khả năng tương thích cao với nhiều ngôn ngữ lập trình.

Ưu điểm của JSON:

- Cú pháp đơn giản, dễ đọc
- Dễ parse và xử lý
- Kích thước nhỏ hơn XML
- Tương thích với nhiều nền tảng và ngôn ngữ

Trong HTTP request và response, JSON thường được khai báo thông qua Content-Type: application/json. Việc sử dụng JSON giúp quá trình trao đổi dữ liệu giữa các service trong hệ thống Microservice trở nên linh hoạt và hiệu quả hơn.

2.2.4. HTTP Status Code

HTTP Status Code là mã trạng thái được server trả về nhằm thông báo kết quả xử lý request cho client.

Các mã trạng thái HTTP được chia thành nhiều nhóm khác nhau.

- Nhóm 2xx – Thành công
 - + 200 OK: Request thành công
 - + 201 Created: Tạo dữ liệu thành công
 - + 204 No Content: Thành công nhưng không trả dữ liệu
- Nhóm 4xx – Lỗi phía Client
 - + 400 Bad Request: Request không hợp lệ
 - + 401 Unauthorized: Chưa xác thực
 - + 403 Forbidden: Không có quyền truy cập
 - + 404 Not Found: Không tìm thấy tài nguyên
- Nhóm 5xx – Lỗi phía Server

- + 500 Internal Server Error: Lỗi server
- + 502 Bad Gateway: Gateway lỗi
- + 503 Service Unavailable: Service tạm thời không khả dụng

Việc sử dụng đúng HTTP Status Code giúp: Client dễ xử lý response, API rõ ràng và chuẩn hóa hơn, hỗ trợ debug và kiểm thử hiệu quả. Trong các hệ thống Microservice, việc chuẩn hóa status code đóng vai trò quan trọng nhằm đảm bảo tính nhất quán trong giao tiếp giữa các service và giữa client với hệ thống.

2.3. RabbitMQ và Event-Driven Architecture

2.3.1. Khái niệm Message Queue

Message Queue là cơ chế giao tiếp bất đồng bộ (Asynchronous Communication) giữa các thành phần trong hệ thống thông qua hàng đợi thông điệp (queue). Thay vì các service giao tiếp trực tiếp với nhau, dữ liệu sẽ được đóng gói thành message và gửi vào queue để các service khác nhận và xử lý.

Trong mô hình này, service gửi dữ liệu được gọi là Producer, còn service nhận và xử lý dữ liệu được gọi là Consumer. Message Queue đóng vai trò trung gian giúp các service giao tiếp mà không cần phụ thuộc trực tiếp vào nhau.

Quy trình hoạt động cơ bản của Message Queue:

- Producer gửi message vào queue.
- Queue lưu trữ message tạm thời.
- Consumer lấy message từ queue để xử lý.

Message Queue mang lại nhiều lợi ích cho hệ thống phân tán:

- Giảm coupling giữa các service
- Hỗ trợ giao tiếp bất đồng bộ
- Tăng khả năng mở rộng
- Tăng khả năng chịu lỗi
- Hỗ trợ xử lý realtime
- Giảm tải cho hệ thống

Trong các hệ thống Microservice hiện đại, Message Queue thường được sử dụng để:

- Gửi notification
- Xử lý background job
- Đồng bộ dữ liệu giữa các service
- Event-driven workflow

- Logging và monitoring

Một số công nghệ Message Queue phổ biến hiện nay gồm:

- RabbitMQ
- Apache Kafka
- Redis

2.3.2. RabbitMQ

RabbitMQ là một Message Broker mã nguồn mở được sử dụng phổ biến trong các hệ thống phân tán và Microservice. RabbitMQ hỗ trợ giao tiếp bất đồng bộ giữa các service thông qua cơ chế gửi và nhận message. RabbitMQ được xây dựng dựa trên giao thức AMQP (Advanced Message Queuing Protocol), cho phép các hệ thống trao đổi message một cách ổn định và đáng tin cậy.

Trong RabbitMQ:

- Producer gửi message vào Exchange.
- Exchange định tuyến message đến Queue.
- Consumer nhận message từ Queue để xử lý.

RabbitMQ hỗ trợ nhiều tính năng quan trọng như:

- Message persistence.
- Retry message.
- Acknowledgement.
- Routing message.
- Publish/Subscribe.
- Dead Letter Queue.

Ưu điểm của RabbitMQ:

- Dễ triển khai và sử dụng.
- Hỗ trợ nhiều ngôn ngữ lập trình.
- Hiệu năng tốt.
- Hỗ trợ routing linh hoạt.
- Hỗ trợ giao tiếp realtime.
- Phù hợp với kiến trúc Microservice.

RabbitMQ hiện được sử dụng rộng rãi trong các hệ thống yêu cầu xử lý bất đồng bộ và event-driven architecture.

2.3.3. Exchange – Queue – Routing Key

Trong RabbitMQ, quá trình truyền message được thực hiện thông qua ba thành phần chính gồm Exchange, Queue và Routing Key.

Exchange

Exchange là thành phần tiếp nhận message từ Producer và quyết định message sẽ được gửi đến Queue nào. Exchange không lưu trữ message mà chỉ thực hiện nhiệm vụ định tuyến message.

RabbitMQ hỗ trợ nhiều loại Exchange như:

- Direct Exchange
- Topic Exchange
- Fanout Exchange
- Headers Exchange

Mỗi loại Exchange có cơ chế routing khác nhau tùy theo nhu cầu sử dụng.

Queue

Queue là nơi lưu trữ message trước khi Consumer nhận và xử lý. Message sẽ tồn tại trong Queue cho đến khi được Consumer xử lý thành công hoặc hết thời gian tồn tại (TTL). Queue giúp đảm bảo message không bị mất, service có thể xử lý bất đồng bộ, hệ thống ổn định khi lưu lượng tăng cao. Một Queue có thể có nhiều Producer, nhiều Consumer.

Routing Key

Routing Key là khóa định tuyến được sử dụng để xác định message sẽ được gửi đến Queue nào. Khi Producer gửi message đến Exchange, message sẽ đi kèm Routing Key. Exchange sẽ dựa vào Routing Key để định tuyến message phù hợp. Routing Key giúp phân loại message, định tuyến linh hoạt, hỗ trợ event-driven architecture.

2.3.4. Publish/Subscribe

Publish/Subscribe (Pub/Sub) là mô hình giao tiếp trong đó một service phát hành sự kiện (Publisher) và nhiều service khác có thể đăng ký nhận sự kiện đó (Subscriber). Trong mô hình này Publisher không cần biết Subscriber là ai, Subscriber chỉ cần lắng nghe các event cần thiết

Quy trình hoạt động:

- Publisher gửi event vào Exchange
- Exchange định tuyến message đến nhiều Queue

- Các Subscriber nhận message từ Queue tương ứng

Tất cả các service trên đều có thể nhận cùng một event mà không phụ thuộc trực tiếp vào nhau.

Ưu điểm của Publish/Subscribe:

- Loose Coupling
- Dễ mở rộng hệ thống
- Hỗ trợ realtime
- Tăng khả năng mở rộng
- Dễ thêm service mới

Mô hình Publish/Subscribe là nền tảng quan trọng của Event-Driven Architecture trong các hệ thống Microservice hiện đại.

2.3.5. Eventual Consistency

Trong hệ thống Microservice, mỗi service thường sở hữu database riêng. Điều này làm cho việc duy trì tính nhất quán dữ liệu theo mô hình ACID truyền thống trở nên khó khăn. Để giải quyết vấn đề này, nhiều hệ thống áp dụng mô hình Eventual Consistency.

Eventual Consistency là mô hình đảm bảo rằng dữ liệu giữa các service sẽ đạt trạng thái nhất quán sau một khoảng thời gian nhất định thay vì cập nhật đồng thời ngay lập tức.

Quy trình hoạt động:

1. Một service cập nhật dữ liệu.
2. Service phát sinh event.
3. Event được gửi qua Message Queue.
4. Các service khác nhận event và cập nhật dữ liệu tương ứng.

Dữ liệu giữa các service có thể chưa đồng bộ ngay lập tức, nhưng cuối cùng sẽ đạt trạng thái nhất quán.

Ưu điểm của Eventual Consistency:

- Tăng hiệu năng hệ thống
- Giảm coupling giữa các service
- Hỗ trợ scalability
- Phù hợp với distributed system

Tuy nhiên, mô hình Eventual Consistency cũng làm tăng độ phức tạp trong hệ thống phân tán. Do dữ liệu được đồng bộ thông qua event nên có thể xảy ra các vấn đề như duplicate event, retry nhiều lần, lỗi distributed transaction hoặc rollback khi một service xử lý thất bại. Điều này khiến việc đảm bảo tính nhất quán dữ liệu trở nên khó khăn hơn so với kiến trúc Monolithic.

Để giải quyết các vấn đề trên, các hệ thống Microservice thường kết hợp thêm các cơ chế như Saga Pattern, Retry Policy, Dead Letter Queue và Idempotency. Những cơ chế này giúp tăng khả năng chịu lỗi, hạn chế mất dữ liệu và đảm bảo hệ thống hoạt động ổn định hơn trong môi trường phân tán.

2.4. Spring Cloud

Spring Cloud là tập hợp các công cụ hỗ trợ xây dựng hệ thống phân tán và Microservice dựa trên Spring Boot. Spring Cloud cung cấp nhiều thành phần giúp giải quyết các vấn đề thường gặp trong kiến trúc Microservice như service discovery, API gateway, centralized configuration, load balancing và fault tolerance.

Nhờ Spring Cloud, việc phát triển và quản lý các hệ thống Microservice trở nên dễ dàng và hiệu quả hơn. Một số thành phần quan trọng của Spring Cloud gồm Eureka Service Discovery, Spring Cloud Gateway và Spring Cloud Config.

2.4.1. Eureka Service Discovery

Netflix Eureka là công cụ Service Discovery được sử dụng phổ biến trong hệ thống Microservice. Eureka cho phép các service tự động đăng ký và tìm kiếm lẫn nhau trong hệ thống phân tán.

Trong môi trường Microservice, địa chỉ IP hoặc port của service có thể thay đổi thường xuyên do scaling hoặc redeploy. Eureka giúp giải quyết vấn đề này bằng cách duy trì danh sách các service đang hoạt động.

Khi một service khởi động, service đó sẽ tự động đăng ký thông tin của mình với Eureka Server, bao gồm tên service, địa chỉ IP và port đang hoạt động. Eureka Server sẽ lưu trữ và cập nhật danh sách các service trong hệ thống. Khi cần giao tiếp với nhau, các service khác có thể gửi yêu cầu đến Eureka để tìm kiếm địa chỉ của service cần gọi thay vì phải cấu hình thủ công. Nhờ đó, hệ thống Microservice trở nên linh hoạt hơn và dễ dàng mở rộng trong quá trình triển khai.

Ưu điểm của Eureka:

- Tự động quản lý service

- Hỗ trợ load balancing
- Giảm cấu hình thủ công
- Dễ mở rộng hệ thống

Eureka đóng vai trò quan trọng trong việc quản lý giao tiếp giữa các service trong kiến trúc Microservice.

2.4.2. Spring Cloud Gateway

Spring Cloud Gateway là API Gateway được xây dựng trên nền tảng Spring WebFlux, dùng để làm cổng giao tiếp trung tâm cho hệ thống Microservice. Thay vì client gọi trực tiếp đến từng service, mọi request sẽ đi qua Gateway trước khi được định tuyến đến service phù hợp.

Spring Cloud Gateway hỗ trợ:

- Routing request
- Authentication và Authorization
- Rate limiting
- Logging
- Load balancing
- Filter request/response

Ưu điểm của API Gateway:

- Tăng tính bảo mật
- Quản lý request tập trung
- Ẩn cấu trúc bên trong hệ thống
- Giảm độ phức tạp phía client

Trong hệ thống Microservice, Gateway giúp quản lý giao tiếp hiệu quả và cải thiện khả năng bảo trì hệ thống.

2.4.3. Spring Cloud Config

Spring Cloud Config là công cụ quản lý cấu hình tập trung cho các service trong hệ thống Microservice. Thay vì mỗi service tự lưu cấu hình riêng, Spring Cloud Config cho phép quản lý cấu hình tại một nơi trung tâm. Các service sẽ lấy cấu hình từ Config Server khi khởi động.

Các thông tin cấu hình được quản lý bởi Spring Cloud Config có thể bao gồm cấu hình kết nối cơ sở dữ liệu (Database Configuration), cấu hình của RabbitMQ, API key, biến môi trường (Environment Variable) và địa chỉ URL của các service trong hệ

thông. Việc quản lý cấu hình tập trung giúp các service dễ dàng cập nhật cấu hình mà không cần chỉnh sửa trực tiếp trong từng service riêng lẻ, đồng thời hỗ trợ triển khai linh hoạt trên nhiều môi trường khác nhau như development, testing và production.

Ưu điểm của Spring Cloud Config:

- Quản lý cấu hình tập trung
- Dễ thay đổi và cập nhật
- Giảm trùng lặp cấu hình
- Hỗ trợ nhiều môi trường như development, testing và production

Spring Cloud Config giúp việc quản lý và triển khai hệ thống Microservice trở nên linh hoạt và thuận tiện hơn.

2.5. JWT Authentication

Xác thực (Authentication) và phân quyền (Authorization) là hai thành phần quan trọng trong các hệ thống thông tin hiện đại nhằm đảm bảo tính bảo mật và kiểm soát truy cập tài nguyên. Trong kiến trúc microservices, việc sử dụng cơ chế xác thực dựa trên session truyền thống gặp nhiều hạn chế do yêu cầu lưu trạng thái phiên làm việc tại server, gây khó khăn cho việc mở rộng và phân tán hệ thống.

Để khắc phục vấn đề này, nhiều hệ thống hiện nay áp dụng JSON Web Token (JWT) theo chuẩn RFC 7519 nhằm triển khai cơ chế xác thực phi trạng thái (Stateless Authentication). JWT là một chuỗi dữ liệu được mã hóa và ký số, cho phép lưu trữ thông tin xác thực của người dùng dưới dạng an toàn.

Một JWT thường gồm ba thành phần:

- Header: Chứa thông tin về loại token và thuật toán ký số.
- Payload: Chứa các thông tin định danh người dùng và các claims cần thiết.
- Signature: Chữ ký số dùng để kiểm tra tính toàn vẹn của token.

JWT thường được sử dụng kết hợp với thuật toán HS256 hoặc RSA để đảm bảo dữ liệu không bị thay đổi trong quá trình truyền tải. Trong quá trình xác thực, sau khi người dùng đăng nhập thành công, hệ thống sẽ cấp một token cho client. Token này được gửi kèm trong các request tiếp theo thông qua HTTP Header theo định dạng:

Authorization: Bearer

Tại phía server hoặc API Gateway, token sẽ được kiểm tra tính hợp lệ thông qua các bước:

1. Kiểm tra chữ ký số.

2. Kiểm tra thời gian hết hạn.
3. Kiểm tra quyền truy cập của người dùng.

Ngoài Access Token, nhiều hệ thống còn sử dụng thêm:

- Refresh Token: Dùng để cấp mới Access Token khi hết hạn.
- Reset Password Token: Dùng trong chức năng khôi phục mật khẩu.

Để tăng cường bảo mật, mật khẩu người dùng thường được mã hóa bằng BCrypt trước khi lưu vào cơ sở dữ liệu. BCrypt là thuật toán băm một chiều có tích hợp Salt và Cost Factor nhằm chống lại các hình thức tấn công brute-force hoặc rainbow table.

Trong kiến trúc microservices, JWT thường được kết hợp với API Gateway để tập trung xử lý xác thực và giảm tải cho các service phía sau. Sau khi token hợp lệ, Gateway có thể truyền thông tin người dùng qua các internal headers nhằm hỗ trợ các service xử lý nghiệp vụ.

Bên cạnh xác thực, hệ thống còn áp dụng cơ chế phân quyền theo vai trò (Role-Based Access Control – RBAC). Người dùng được gán các vai trò khác nhau và mỗi vai trò sẽ có quyền truy cập tương ứng đối với từng tài nguyên hoặc chức năng của hệ thống.

2.6. WebSocket Realtime

Các ứng dụng cộng tác hiện đại yêu cầu khả năng cập nhật dữ liệu tức thời giữa nhiều người dùng để đảm bảo tính đồng bộ và trải nghiệm realtime. Những phương thức truyền thống như Polling hoặc Long Polling thường tạo ra nhiều request dư thừa và làm tăng độ trễ hệ thống. Vì vậy, WebSocket được sử dụng như một giải pháp tối ưu cho giao tiếp realtime giữa client và server.

WebSocket cho phép client và server chủ động gửi dữ liệu cho nhau thông qua một kết nối TCP duy nhất, từ đó giảm overhead mạng và cải thiện tốc độ phản hồi. Quá trình kết nối bắt đầu bằng HTTP Handshake, sau đó kết nối được nâng cấp để hỗ trợ truyền dữ liệu realtime.

Đối với các ứng dụng sử dụng Spring Framework, WebSocket thường được kết hợp với giao thức STOMP nhằm hỗ trợ mô hình Publish/Subscribe. STOMP cung cấp cấu trúc message rõ ràng, giúp quản lý việc gửi và nhận dữ liệu hiệu quả hơn.

Một số mô hình truyền dữ liệu phổ biến gồm:

- Kênh cá nhân: gửi thông báo riêng cho từng người dùng.

- Kênh nhóm: broadcast dữ liệu cho nhiều người dùng cùng lúc.

Để tăng tính bảo mật, hệ thống có thể thực hiện xác thực JWT ngay trong quá trình handshake và kiểm tra quyền truy cập trước khi cho phép subscribe vào các kênh dữ liệu.

Kết hợp với Message Broker như RabbitMQ, WebSocket giúp xây dựng mô hình Event-Driven Architecture hiệu quả hơn. Khi dữ liệu thay đổi, service nghiệp vụ sẽ phát sinh event và các service khác sẽ lắng nghe để xử lý hoặc gửi thông báo realtime đến client. Mô hình này giúp giảm phụ thuộc giữa các service, tăng khả năng mở rộng và cải thiện hiệu năng hệ thống.

2.7. Redis Cache

Hiệu năng và tốc độ phản hồi là những yếu tố quan trọng trong các hệ thống phân tán hiện đại. Khi số lượng request tăng cao, việc truy vấn liên tục vào cơ sở dữ liệu quan hệ có thể gây quá tải và làm tăng độ trễ xử lý. Để khắc phục vấn đề này, nhiều hệ thống sử dụng Redis như một lớp bộ nhớ đệm nhằm tối ưu tốc độ truy xuất dữ liệu.

Redis là hệ quản trị cơ sở dữ liệu dạng Key-Value hoạt động trên bộ nhớ RAM, cho phép truy xuất dữ liệu với tốc độ rất cao. Redis thường được sử dụng trong các hệ thống Microservice nhằm giảm tải cho database và tăng khả năng đáp ứng của hệ thống.

Một số ứng dụng phổ biến của Redis gồm:

- Authorization Cache: lưu thông tin phân quyền hoặc vai trò người dùng.
- Idempotency Key: ngăn xử lý trùng lặp request.
- Rate Limiting: giới hạn số lượng request trong một khoảng thời gian.
- JWT Blacklist: lưu danh sách token bị vô hiệu hóa.

Để đảm bảo dữ liệu luôn chính xác, hệ thống thường kết hợp cơ chế TTL (Time-To-Live) và Cache Invalidation. Khi dữ liệu gốc thay đổi, cache sẽ được cập nhật hoặc xóa để tránh sai lệch dữ liệu. Ngoài ra, Redis còn hỗ trợ nhiều chính sách giải phóng bộ nhớ như LRU nhằm tối ưu việc sử dụng RAM trong môi trường có lưu lượng truy cập lớn.

2.8. Resilience4j

Khả năng chịu lỗi là yêu cầu quan trọng trong các hệ thống Microservice do các service thường giao tiếp với nhau thông qua mạng. Những sự cố như timeout, mất kết

nối hoặc service quá tải có thể xảy ra bất kỳ lúc nào và dễ gây ảnh hưởng dây chuyền nếu không được xử lý phù hợp.

Resilience4j là thư viện hỗ trợ xây dựng các cơ chế Fault Tolerance cho ứng dụng Java. Thư viện này giúp tăng khả năng phục hồi và đảm bảo tính ổn định của hệ thống phân tán.

Một số cơ chế quan trọng của Resilience4j gồm:

- Circuit Breaker: hoạt động tương tự cầu dao điện, giúp ngăn hệ thống tiếp tục gửi request đến service đang gặp lỗi. Circuit Breaker gồm ba trạng thái:
- CLOSED: hoạt động bình thường.
- OPEN: chặn request khi tỷ lệ lỗi vượt ngưỡng.
- HALF-OPEN: thử gửi lại request để kiểm tra khả năng phục hồi.
- Retry: tự động gửi lại request khi xảy ra lỗi tạm thời như timeout hoặc mất kết nối.
- TimeLimiter: giới hạn thời gian chờ phản hồi nhằm tránh treo luồng xử lý quá lâu.

Việc kết hợp các cơ chế trên giúp hệ thống tăng độ ổn định, hạn chế lỗi dây chuyền và cải thiện trải nghiệm người dùng trong môi trường phân tán.

2.9. Saga Pattern

Quản lý transaction phân tán là một trong những thách thức lớn của kiến trúc Microservice. Do mỗi service sở hữu cơ sở dữ liệu riêng nên việc áp dụng transaction theo mô hình ACID truyền thống trở nên khó khăn khi nhiều service cùng tham gia xử lý một nghiệp vụ.

Saga Pattern được sử dụng để giải quyết vấn đề này thông qua chuỗi các giao dịch cục bộ kết hợp với cơ chế truyền sự kiện giữa các service. Thay vì sử dụng một transaction tập trung, mỗi service sẽ tự xử lý phần nghiệp vụ của mình và phát sinh event để kích hoạt service tiếp theo.

Có hai cách triển khai Saga phổ biến:

- Choreography: các service tự giao tiếp với nhau thông qua event.
- Orchestration: một service trung tâm điều phối toàn bộ luồng xử lý.

Ở mô hình Choreography, sau khi xử lý thành công, mỗi service sẽ phát sinh event để kích hoạt bước tiếp theo. Nếu xảy ra lỗi, hệ thống sẽ thực hiện các Compensating Transaction nhằm đưa dữ liệu về trạng thái nhất quán.

Saga Pattern mang lại nhiều lợi ích như:

- Giảm phụ thuộc giữa các service.
- Tăng khả năng mở rộng.
- Hạn chế khóa tài nguyên kéo dài.
- Đảm bảo Eventual Consistency.

Trên thực tế, Saga thường được triển khai kết hợp với Message Broker như RabbitMQ hoặc Apache Kafka để truyền tải sự kiện giữa các service.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

Hệ thống TaskFlow được xây dựng nhằm hỗ trợ quản lý dự án, quản lý công việc và cộng tác nhóm theo thời gian thực trên nền tảng Microservice. Hệ thống cần đáp ứng các yêu cầu về quản lý người dùng, phân chia nhiệm vụ, giao tiếp realtime và khả năng mở rộng trong môi trường phân tán.

Các yêu cầu chức năng của hệ thống được phân tích dựa trên nghiệp vụ quản lý công việc hiện đại và đặc tả của đồ án Microservice.

3.1.1. Yêu cầu chức năng

Authentication

Hệ thống cần cung cấp chức năng xác thực và phân quyền người dùng nhằm đảm bảo tính bảo mật trong quá trình sử dụng.

Các chức năng chính gồm:

- Đăng ký tài khoản
- Đăng nhập hệ thống
- Đăng xuất
- Xác thực bằng JWT
- Làm mới Access Token
- Quản lý thông tin cá nhân
- Phân quyền người dùng theo vai trò

Sau khi đăng nhập thành công, hệ thống sẽ cấp JWT Token để người dùng truy cập các API được bảo vệ.

Project Management

Hệ thống cho phép người dùng tạo và quản lý nhiều dự án khác nhau.

Các chức năng chính gồm:

- Tạo dự án mới
- Cập nhật thông tin dự án
- Xóa dự án
- Quản lý thành viên dự án
- Phân quyền thành viên
- Quản lý Sprint và Backlog
- Theo dõi tiến độ dự án

Mỗi dự án có thể bao gồm nhiều board và nhiều thành viên cùng tham gia làm việc.

Board Management

Hệ thống hỗ trợ mô hình Kanban Board để tổ chức và theo dõi công việc trực quan.

Các chức năng gồm:

- Tạo Board
- Tạo List/Column
- Cập nhật vị trí List
- Quản lý trạng thái công việc
- Kéo thả Task giữa các cột

Board giúp người dùng dễ dàng theo dõi tiến độ công việc theo từng trạng thái như To Do, In Progress và Done.

Task Management

Task là thành phần trung tâm của hệ thống quản lý công việc.

Các chức năng chính gồm:

- Tạo Task
- Cập nhật Task
- Xóa Task
- Gán người thực hiện
- Thiết lập mức độ ưu tiên
- Thiết lập deadline
- Quản lý checklist
- Gắn nhãn (Label)
- Tìm kiếm và lọc Task

Hệ thống cần hỗ trợ cập nhật trạng thái Task theo thời gian thực nhằm đảm bảo tính đồng bộ giữa các thành viên.

Notification

Hệ thống cần hỗ trợ cơ chế thông báo realtime để người dùng nhanh chóng nhận được các thay đổi liên quan đến công việc và dự án.

Các loại thông báo gồm:

- Giao Task mới
- Thay đổi trạng thái Task

- Có bình luận mới
- Mời tham gia dự án
- Deadline sắp đến

Notification được triển khai thông qua WebSocket và Message Queue nhằm đảm bảo khả năng xử lý realtime và bất đồng bộ.

Collaboration

Hệ thống cần hỗ trợ cộng tác giữa nhiều người dùng trong cùng dự án.

Các chức năng cộng tác gồm:

- Bình luận trên Task
- Mention thành viên
- Theo dõi lịch sử hoạt động
- Chia sẻ thông tin giữa các thành viên
- Đồng bộ dữ liệu realtime

Chức năng Collaboration giúp tăng khả năng trao đổi thông tin và nâng cao hiệu quả làm việc nhóm trong hệ thống quản lý dự án.

3.1.2. Yêu cầu phi chức năng

Bên cạnh các chức năng nghiệp vụ, hệ thống còn cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo khả năng vận hành ổn định, mở rộng linh hoạt và phù hợp với kiến trúc Microservice hiện đại.

Scalability

Hệ thống cần có khả năng mở rộng linh hoạt khi số lượng người dùng hoặc lưu lượng truy cập tăng cao. Mỗi service phải có khả năng scale độc lập mà không ảnh hưởng đến toàn bộ hệ thống. Kiến trúc Microservice kết hợp với Docker giúp hệ thống dễ dàng triển khai nhiều instance của service nhằm tăng khả năng xử lý và tối ưu tài nguyên.

Fault Tolerance

Hệ thống cần đảm bảo khả năng chịu lỗi khi một service gặp sự cố hoặc tạm thời không phản hồi. Lỗi từ một service không được làm gián đoạn toàn bộ hệ thống.

Các cơ chế như Circuit Breaker, Retry và Message Queue được sử dụng nhằm:

- Giám lỗi dây chuyền
- Tăng tính ổn định
- Đảm bảo hệ thống tiếp tục hoạt động khi xảy ra sự cố

Security

Hệ thống cần đảm bảo tính bảo mật trong quá trình xác thực, phân quyền và trao đổi dữ liệu giữa các service.

Các yêu cầu bảo mật gồm:

- Xác thực bằng JWT
- Phân quyền theo vai trò người dùng
- Bảo vệ RESTful API
- Mã hóa mật khẩu
- Kiểm tra quyền truy cập tài nguyên

Ngoài ra, hệ thống cần hạn chế truy cập trái phép và đảm bảo an toàn dữ liệu người dùng.

Realtime

Hệ thống cần hỗ trợ cập nhật dữ liệu theo thời gian thực nhằm đảm bảo tính đồng bộ giữa các người dùng.

Các chức năng realtime gồm:

- Notification realtime
- Đồng bộ trạng thái Task
- Cập nhật Board tức thời
- Broadcast sự kiện giữa nhiều client

Cơ chế realtime được triển khai thông qua WebSocket kết hợp với Message Broker.

Logging

Hệ thống cần có cơ chế ghi log nhằm hỗ trợ theo dõi hoạt động, debug và giám sát hệ thống.

Thông tin log có thể bao gồm:

- HTTP Request/Response
- Error log
- Message Queue event
- Authentication log
- Service communication log

Logging giúp dễ dàng phát hiện lỗi và phân tích hoạt động của hệ thống trong môi trường phân tán.

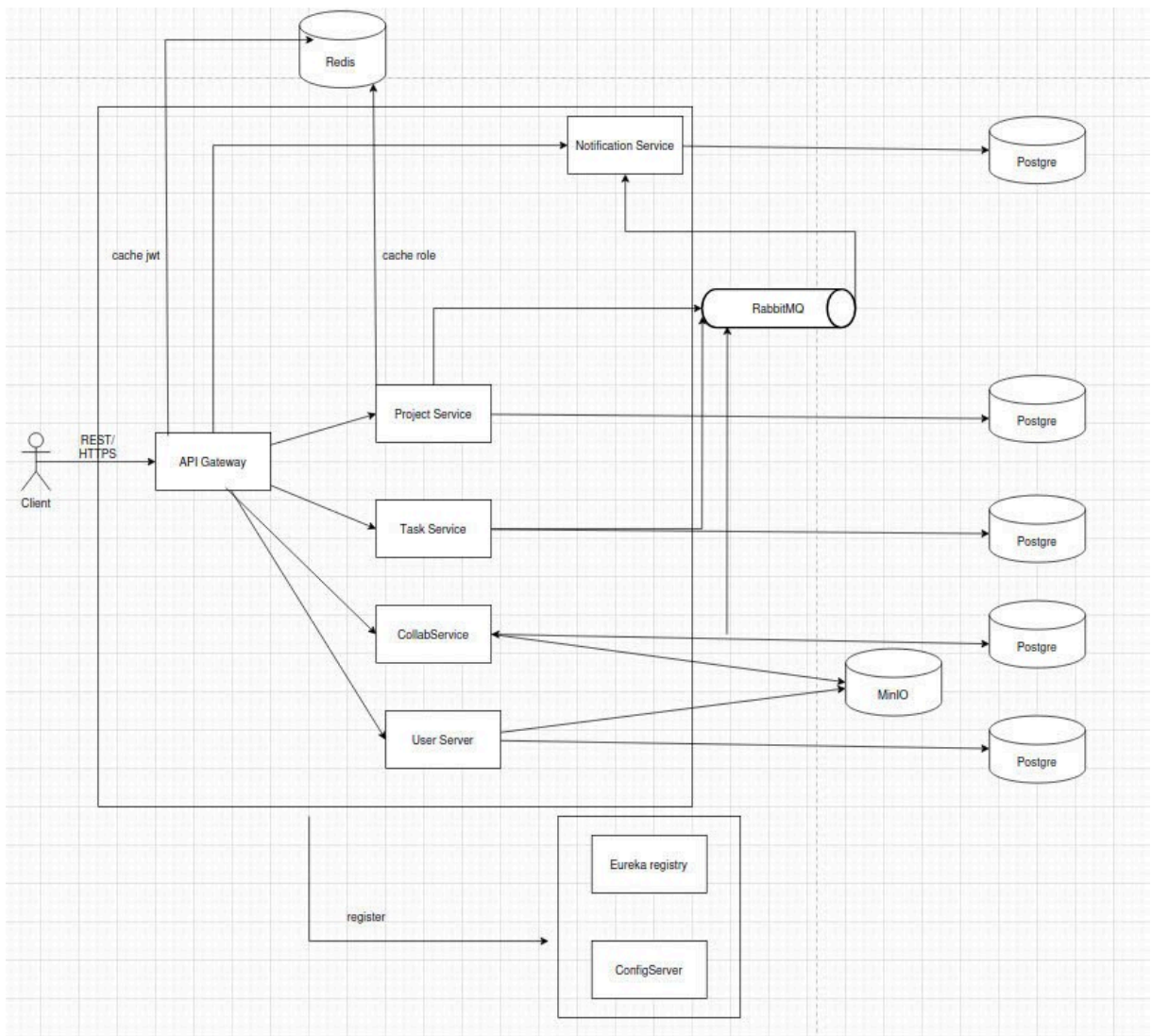
Loose Coupling

Các service trong hệ thống cần được thiết kế với mức phụ thuộc thấp nhằm đảm bảo tính độc lập và dễ mở rộng. Việc giao tiếp giữa các service được thực hiện thông qua RESTful API và Message Queue thay vì truy cập trực tiếp vào dữ liệu hoặc logic của nhau.

- Loose Coupling giúp:
- Dễ bảo trì hệ thống
- Dễ mở rộng service
- Hạn chế ảnh hưởng dây chuyền khi xảy ra lỗi
- Hỗ trợ triển khai độc lập giữa các service

Đây là một trong những đặc điểm quan trọng của kiến trúc Microservice hiện đại.

3.2. Kiến trúc tổng thể hệ thống



Hình 3.2 Kiến trúc hệ thống

3.3. Thiết kế Microservices

3.3.1. User Service

User Service chịu trách nhiệm quản lý vòng đời tài khoản người dùng, xác thực và bảo mật hệ thống. Để đảm bảo tính độc lập dữ liệu, dịch vụ này sở hữu riêng cơ sở dữ liệu taskflow_user và kết nối với cụm lưu trữ đối tượng MinIO để quản lý tệp tin.

Mô hình Dữ liệu (Database Schema)

Cơ sở dữ liệu của dịch vụ bao gồm hai bảng nghiệp vụ chính được thiết kế tối ưu:

- users: Lưu trữ thông tin tài khoản cốt lõi như tên đăng nhập, địa chỉ email duy nhất, mã băm mật khẩu, họ tên, ảnh đại diện, trạng thái tài khoản và các trường kỹ thuật để đếm số lần đăng nhập thất bại nhằm thực hiện chính sách khóa tài khoản tự động.

- password_resets: Quản lý các token khôi phục mật khẩu. Token được sinh ngẫu nhiên và lưu dưới dạng mã băm SHA-256 để bảo mật, đi kèm hạn sử dụng (30 phút) và cờ used để chống tái sử dụng.

Chức năng chính và Công nghệ cốt lõi

- Xác thực JWT và Đăng nhập: Dịch vụ kiểm tra tài khoản, mã hóa mật khẩu bằng BCrypt, và cấp phát cặp mã thông báo JWT khi đăng nhập thành công. Để hỗ trợ thu hồi token, dịch vụ duy trì phiên Token hợp lệ trong Redis Cache.
- Tích hợp lưu trữ đối tượng MinIO: Avatar của người dùng được upload trực tiếp lên cụm MinIO thông qua thư viện MinioClient, được đặt tên chuẩn hóa theo mã người dùng và lưu metadata đường dẫn vào cơ sở dữ liệu PostgreSQL.
- Chính sách an toàn tài khoản: Hệ thống tự động khóa tài khoản sau 5 lần đăng nhập sai liên tiếp và cung cấp luồng khôi phục mật khẩu an toàn qua Email.

Danh mục API của User Service

Bảng 3.1.1 Danh mục API của User Service

STT	Method	URL	Quyền hạn	Mô tả chức năng
1	POST	/api/v1/auth /register	ANONYMO US	Đăng ký tài khoản người dùng mới
2	POST	/api/v1/auth /login	ANONYMO US	Xác thực đăng nhập và cấp cấp token JWT

3	POST	/api/v1/auth /refresh	ANONYMO US	Làm mới Access Token bằng Refresh Token
4	POST	/api/v1/auth /logout	AUTH	Đăng xuất, vô hiệu hóa token trong Redis
5	POST	/api/v1/auth /change-pass sword	AUTH	Thay đổi mật khẩu tài khoản hiện tại
6	POST	/api/v1/auth /forgot-pass word	ANONYMO US	Yêu cầu gửi liên kết khôi phục mật khẩu
7	POST	/api/v1/auth /reset-pass word	ANONYMO US	Đặt lại mật khẩu mới bằng token hợp lệ

8	GET	/api/v1/users/me	AUTH	Lấy thông tin hồ sơ của tài khoản đang đăng nhập
9	PUT	/api/v1/users/me	AUTH	Cập nhật hồ sơ thông tin cá nhân
10	POST	/api/v1/users/me/avatar	AUTH	Tải lên ảnh đại diện cá nhân lưu vào MinIO
11	GET	/api/v1/users/avatars/{key}	ANONYMOUS	Endpoint tải/hiển thị ảnh đại diện hiệu năng
12	GET	/api/v1/internal/users/verify	INTERNAL	RPC nội bộ kiểm tra sự tồn tại của người dùng

3.3.2. Project Service

Đặc tả Thiết kế Project Service

Project Service đóng vai trò là xương sống trong việc quản lý cấu trúc tổ chức và thiết lập ranh giới dữ liệu cho toàn bộ ứng dụng TaskFlow. Để đảm bảo tính độc lập và giảm thiểu ảnh hưởng khi mở rộng hệ thống, dịch vụ này sở hữu riêng một cơ sở dữ liệu quan hệ độc lập mang tên `taskflow_project`, giao tiếp với các dịch vụ khác hoàn toàn qua API hoặc thông điệp sự kiện.

Mô hình Dữ liệu (Database Schema)

Cơ sở dữ liệu của Project Service được thiết kế chuẩn hóa để quản lý cấu trúc phân cấp linh hoạt từ mức vĩ mô (Dự án) đến mức vi mô (Thành phần hiển thị). Các thực thể cốt lõi bao gồm:

- `projects`: Lưu trữ thông tin tổng quan của dự án (Tên, mô tả, ngày khởi tạo, trạng thái, khóa định danh của người sở hữu).
- `project_members`: Bảng trung gian giải quyết mối quan hệ nhiều-nhiều giữa Người dùng và Dự án, đồng thời lưu trữ vai trò cụ thể của thành viên đó trong dự án.
- `boards`: Quản lý các bảng công việc Kanban thuộc về một dự án. Một dự án có thể có nhiều bảng làm việc tùy theo phân vùng nghiệp vụ.
- `lists`: Quản lý cấu trúc các cột trong một bảng Kanban (ví dụ: Cột Chờ xử lý, Cột Đang làm). Mỗi bản ghi chứa thông tin vị trí (position) để phục vụ sắp xếp.
- `sprints`: Quản lý các chu kỳ phát triển rút gọn đối với các dự án áp dụng mô hình Agile/Scrum, chứa thời gian bắt đầu, kết thúc và mục tiêu của Sprint.

Quản lý Quyền hạn (Project-level RBAC Matrix)

Dịch vụ hiện thực hóa mô hình ma trận quyền hạn chi tiết bên trong phạm vi dự án bằng cách định nghĩa 5 cấp độ vai trò với các quyền hạn giảm dần, được kiểm soát chặt chẽ tại tầng nghiệp vụ:

- **OWNER (Chủ sở hữu)**: Vai trò tối cao, có toàn quyền quản trị cấu trúc, thành viên, xóa bỏ toàn bộ dự án hoặc thực hiện bàn giao quyền sở hữu dự án cho người khác thông qua API đặc biệt `POST /api/v1/projects/{id}/transfer-ownership`.

- ADMIN (Quản trị viên): Có đầy đủ các quyền thêm/bớt thành viên dưới cấp, cấu hình Board, List và Sprint nhưng không thể xóa dự án hay can thiệp vào tài khoản Owner.
- EDITOR (Thành viên phát triển): Vai trò thao tác nghiệp vụ chính, có quyền tạo mới, chỉnh sửa hoặc di chuyển các Task, List dữ liệu trên bảng công việc.
- OMMENTER (Thành viên đóng góp ý kiến): Chỉ được phép xem thông tin dự án và thực hiện gửi các bình luận, phản hồi vào các thẻ công việc, không có quyền thay đổi cấu trúc dữ liệu.
- VIEWER (Thành viên quan sát): Quyền hạn thấp nhất, chỉ có quyền đọc dữ liệu (Read-Only) để theo dõi tiến độ, bị chặn toàn bộ các API có tính chất chỉnh sửa.

Thiết kế Core RESTful APIs

Hệ thống API công khai của Project Service được chia thành các phân nhóm chức năng riêng biệt, đảm bảo tuân thủ nghiêm ngặt chuẩn thiết kế RESTful với trạng thái phản hồi rõ ràng.

Phân nhóm 1: Quản lý Dự án (Project CRUD & Ownership)

Phân nhóm này chịu trách nhiệm quản lý vòng đời của một dự án. Khi một dự án được khởi tạo thành công qua phương thức POST, hệ thống tự động gán tài khoản thực hiện yêu cầu làm OWNER của dự án đó. Các thao tác cập nhật hoặc xóa (áp dụng kỹ thuật xóa mềm - *Soft Delete* để đảm bảo an toàn dữ liệu) được kiểm soát nghiêm ngặt theo phân cấp cấu trúc quyền.

Bảng 3.3.2.1: Chi tiết các API quản lý Dự án

STT	Method	URL	Quyền hạn	Mô tả chức năng
1	POST	/projects	AUTH	Khởi tạo dự án mới (Người tạo tự động trở thành Owner)
2	GET	/projects	AUTH	Lấy danh sách toàn bộ các dự án người dùng hiện tại đang tham gia

3	GET	/projects/{id}	MEMBER	Lấy thông tin chi tiết cấu hình của một dự án cụ thể
4	PUT	/projects/{id}	ADMIN+	Cập nhật các thông tin tổng quan của dự án
5	DELETE	/projects/{id}	OWNER	Thực hiện xóa mềm (Soft Delete) dự án khỏi hệ thống
6	GET	/projects/search?q=...	AUTH	Tìm kiếm dự án theo từ khóa trong phạm vi các dự án tham gia
7	POST	/projects/{id}/transfer-ownership	OWNER	Bàn giao và chuyển cấu trúc quyền sở hữu cao nhất cho tài khoản khác

Phân nhóm 2: Quản lý Thành viên Dự án (Project Members Management)

Phân nhóm này quản lý danh sách nhân sự tham gia vào từng dự án cụ thể, cho phép phân bổ vai trò chi tiết để phục vụ ma trận kiểm soát truy cập tài nguyên.

Bảng 3.3.2.2: Chi tiết các API quản lý Thành viên

ST T	Meth od	URL	Quyền hạn	Mô tả chức năng
1	GET	/projects/{id}/members	MEMBER	Lấy danh sách và vai trò của tất cả thành viên trong dự án

2	POST	/projects/{id}/members	ADMIN+	Thêm thành viên mới vào dự án (Yêu cầu user_id và role)
3	PATCH	/projects/{id}/members/{userId}/role	ADMIN+	Thay đổi vai trò của thành viên (Đặc quyền Owner thay đổi Owner)
4	DELETE	/projects/{id}/members/{userId}	ADMIN+	Loại bỏ thành viên khỏi dự án (Chặn hành vi xóa tài khoản Owner)

Phân nhóm 3: Quản lý Bảng công việc và Cột trạng thái (Boards & Lists)

Phân nhóm này chịu trách nhiệm quản lý không gian làm việc trực quan dưới dạng các bảng Kanban (boards) và các cột phân loại trạng thái công việc (lists). Đặc biệt, thao tác thay đổi thứ tự các cột trạng thái (reorder) được tối ưu hóa thông qua phương thức PATCH hàng loạt để tăng tốc độ phản hồi.

Bảng 3.3.2.3: Chi tiết các API quản lý Boards và Lists

STT	Method	URL	Quyền hạn	Mô tả chức năng
1	POST	/projects/{id}/boards	EDITOR+	Tạo bảng công việc mới bên trong dự án

2	GET	/projects/{id}/boards	MEMBER	Lấy danh sách toàn bộ các bảng công việc của dự án
3	GET	/boards/{id}	MEMBER	Lấy chi tiết một bảng Kanban kèm theo danh sách các cột (lists)
4	PUT	/boards/{id}	EDITOR+	Cập nhật thông tin cấu hình bảng công việc
5	DELETE	/boards/{id}	ADMIN+	Xóa bỏ bảng công việc khỏi hệ thống
6	GET	/projects/{id}/boards /search?q=...	MEMBER	Tìm kiếm nhanh bảng công việc bằng từ khóa
7	POST	/boards/{id}/lists	EDITOR+	Khởi tạo cột trạng thái mới trên bảng Kanban chỉ định
8	PUT	/lists/{id}	EDITOR+	Cập nhật tiêu đề hoặc thuộc tính của cột trạng thái
9	DELETE	/lists/{id}	EDITOR+	Xóa bỏ cột trạng thái công việc

10	PATCH	/boards/{id}/lists/reorder	EDITOR+	Sắp xếp lại thứ tự các cột (Body nhận mảng: [{id, position}])
----	-------	----------------------------	---------	---

Phân nhóm 4: Quản lý Chu kỳ và Báo cáo Thống kê (Sprints & Reports)

Phục vụ mô hình quản trị dự án theo phương pháp luận Agile, phân nhóm này cung cấp khả năng vận hành các chu kỳ phát triển (sprints) và trích xuất dữ liệu phân tích hiệu năng làm việc của đội ngũ.

Bảng 3.3.2.4: Chi tiết các API Sprints và Kết xuất Báo cáo

STT	Method	URL	Quyền hạn	Mô tả chức năng
1	POST	/projects/{id}/sprints	ADMIN+	Tạo một chu kỳ phát triển (Sprint) mới cho dự án
2	GET	/projects/{id}/sprints	MEMBER	Lấy danh sách toàn bộ các chu kỳ phát triển của dự án

3	PATCH	/sprints/{id}	ADMIN+	Cập nhật cấu hình hoặc thực hiện đóng/kết thúc chu kỳ
4	GET	/projects/{id}/reports/burndown?sprint_id=...	MEMBER	Trích xuất mảng dữ liệu phục vụ vẽ biểu đồ Burndown Chart
5	GET	/projects/{id}/reports/summary	MEMBER	Tổng hợp số liệu Task phân loại theo Status, Assignee và Priority

Hệ thống API Nội bộ (Internal APIs)

Trong kiến trúc microservices của TaskFlow, nguyên tắc đóng gói cơ sở dữ liệu (Database-per-Service) quy định các dịch vụ khác như *Task Service* hoặc *Collab Service* không được phép truy cập trực tiếp vào schema dữ liệu `taskflow_project`. Do đó, Project Service mở rộng dải API nội bộ nhằm cung cấp khả năng kiểm tra chéo, xác thực thực thể và phân quyền liên dịch vụ (**Inter-service Authorization**).

Các endpoint này được bảo vệ ở tầng API Gateway, chặn đứng hoàn toàn mọi request có tiền tố `/internal/` đến từ môi trường mạng công cộng (Internet), chỉ cho phép các

cuộc gọi RPC thông qua HTTP Client (Spring Cloud OpenFeign) giữa các container bên trong mạng ảo nội bộ thực thi.

Bảng 3.3.2.5: Chi tiết các API giao tiếp nội bộ hệ thống

STT	Method	URL	Mục đích và cơ chế vận hành nội bộ
1	GET	/internal/projects/{id}/members/{userId}/role	Phục vụ các service khác (Task, Collab) truy vấn nhanh vai trò (role) chính xác của người dùng nhằm đưa ra quyết định cho phép thực hiện thao tác nghiệp vụ (Tạo/Xóa Task, Đăng bình luận) hay không.
2	GET	/internal/projects/{id}/exists	Xác thực tính hợp lệ và tồn tại của một dự án trước khi các dịch vụ vệ tinh ghi nhận dữ liệu liên quan.
3	GET	/internal/lists/{id}/board	Trả về cấu trúc cập thông tin định danh {board_id, project_id}. Được gọi khi Task

			Service cần thẩm định tính hợp lệ của một list_id trước khi gắn thẻ công việc vào cột đó.
--	--	--	---

3.3.3. Task Service

Task Service là module nghiệp vụ cốt lõi của hệ thống TaskFlow, chịu trách nhiệm quản lý toàn bộ vòng đời của nhiệm vụ, từ khởi tạo, cập nhật nội dung, di chuyển giữa các cột Kanban cho đến xóa mềm và khôi phục dữ liệu. Ngoài chức năng quản lý task, service còn xử lý các thành phần liên quan như checklist, label, dependency và watcher nhằm hỗ trợ quản lý công việc chi tiết và trực quan hơn. Service sử dụng cơ sở dữ liệu riêng biệt mang tên taskflow_task và được triển khai độc lập trên cổng 8083 theo mô hình Microservice.

Mô hình dữ liệu (Database Schema)

Cơ sở dữ liệu của Task Service được quản lý bằng Liquibase với 5 changelog, được thiết kế xoay quanh thực thể trung tâm là tasks cùng các bảng vệ tinh phục vụ cho các chức năng bổ trợ.

Bảng Tasks là bảng trung tâm lưu trữ toàn bộ thông tin của nhiệm vụ, bao gồm tiêu đề, mô tả chi tiết, người được giao (assignee_id), người tạo (reporter_id), thời hạn hoàn thành (due_date dạng epoch millisecond), mức độ ưu tiên (priority gồm URGENT, HIGH, MEDIUM, LOW) và vị trí hiển thị trong cột Kanban (position). Mỗi task được liên kết logic với project, board và list thông qua các trường project_id, board_id và list_id, tương ứng với dữ liệu quản lý tại Project Service. Hệ thống sử dụng trường version kết hợp cơ chế @Version để triển khai Optimistic Locking, giúp hạn chế xung đột dữ liệu khi nhiều người dùng đồng thời chỉnh sửa cùng một task.

Ngoài ra, các index được xây dựng trên các cột list_id, assignee_id, project_id và due_date nhằm tối ưu hiệu năng cho các thao tác tìm kiếm và lọc dữ liệu.

labels

Bảng Labels là bảng lưu trữ các nhãn màu trong phạm vi từng dự án. Mỗi label bao gồm tên nhãn (name) và mã màu HEX (color). Hệ thống áp dụng ràng buộc

UNIQUE trên cặp (project_id, name) nhằm đảm bảo không tồn tại các label trùng tên trong cùng một dự án.

Bảng task_label_mappings là bảng trung gian quản lý quan hệ nhiều-nhiều giữa tasks và labels. Bảng sử dụng khóa chính kép (task_id, label_id), cho phép một task có thể gắn nhiều label và một label có thể được sử dụng cho nhiều task khác nhau.

checklists là bảng lưu trữ các danh sách công việc con gắn với từng task. Mỗi checklist đại diện cho một nhóm công việc nhỏ nhằm hỗ trợ phân chia nhiệm vụ chi tiết hơn trong quá trình thực hiện.

Bảng Checklist_items là bảng lưu trữ các mục công việc cụ thể bên trong checklist, bao gồm nội dung công việc (content), trạng thái hoàn thành (completed) và vị trí hiển thị (position). Chức năng này hỗ trợ theo dõi tiến độ chi tiết của từng task. Khi task có thay đổi, Notification Service sẽ gửi thông báo realtime đến toàn bộ watcher liên quan.

Soft Delete: hầu hết các bảng trong hệ thống, ngoại trừ task_label_mappings, đều áp dụng cơ chế Soft Delete thông qua cờ deleted. Dữ liệu sẽ không bị xóa vật lý khỏi cơ sở dữ liệu mà chỉ được đánh dấu đã xóa nhằm hỗ trợ chức năng Recycle Bin và khôi phục dữ liệu trong vòng 30 ngày.

Giao tiếp liên Service (Inter-Service Communication)

Giao tiếp đồng bộ (REST)

Task Service sử dụng WebClient kết hợp với cơ chế Load Balancer để thực hiện các lời gọi REST nội bộ sang Project Service thông qua Netflix Eureka. Các service được định danh thông qua tên đăng ký trên Eureka thay vì sử dụng địa chỉ IP cố định, giúp hệ thống linh hoạt hơn trong môi trường Microservice. Địa chỉ giao tiếp nội bộ được sử dụng có dạng: <http://taskflow-project/internal/...>

Thông qua các API nội bộ này, Task Service có thể kiểm tra sự tồn tại của list, lấy thông tin board hoặc project liên quan, đồng thời xác thực vai trò của người dùng trong từng dự án trước khi cho phép thực hiện các thao tác nghiệp vụ.

Để tăng khả năng chịu lỗi trong môi trường phân tán, toàn bộ các request cross-service đều được tích hợp Resilience4j với các cơ chế như Circuit Breaker, Retry và Timeout. Các cơ chế này giúp hạn chế ảnh hưởng dây chuyền khi một service gặp sự cố hoặc phản hồi chậm.

Circuit Breaker được cấu hình với cửa sổ trượt gồm 10 request và ngưỡng lỗi tối đa 50%. Khi tỷ lệ lỗi vượt quá giới hạn cho phép, Circuit Breaker sẽ chuyển sang trạng thái OPEN trong 10 giây nhằm tạm thời ngắt việc gửi request đến service lỗi. Ngoài ra, mỗi request cũng được giới hạn thời gian phản hồi tối đa là 2 giây để tránh tình trạng treo luồng xử lý quá lâu.

kích hoạt cơ chế fallback và trả về response với mã lỗi `project_service_unavailable` thay vì để client chờ vô thời hạn. Cơ chế này giúp tăng tính ổn định và cải thiện trải nghiệm người dùng khi hệ thống xảy ra sự cố tạm thời.

Giao tiếp bất đồng bộ (RabbitMQ)

Task Service tích hợp RabbitMQ theo mô hình Event-Driven Architecture nhằm hỗ trợ giao tiếp bất đồng bộ giữa các service trong hệ thống Microservice. Hệ thống sử dụng exchange kiểu topic với tên `taskflow.exchange` để định tuyến các event đến đúng queue xử lý tương ứng. Cơ chế này giúp giảm sự phụ thuộc trực tiếp giữa các service, tăng khả năng mở rộng và cải thiện hiệu năng xử lý của toàn hệ thống.

Vai trò Producer

Task Service đóng vai trò Producer khi phát sinh các sự kiện liên quan đến task. Mỗi khi trạng thái hoặc dữ liệu của task thay đổi, service sẽ publish event tương ứng thông qua `TaskEventPublisher`.

Một số event chính được phát sinh gồm:

- `task.created`
- `task.updated`
- `task.assigned`
- `task.moved`
- `task.deleted`

Mỗi event đều được đóng gói trong đối tượng `EventEnvelope` bao gồm:

- `eventId`
- `eventType`
- `actorId`
- `data payload`

Việc chuẩn hóa cấu trúc event giúp các service khác dễ dàng consume và xử lý dữ liệu một cách nhất quán. Sau khi event được publish, Notification Service và

Collaboration Service sẽ lắng nghe các message tương ứng để gửi thông báo realtime, cập nhật activity log hoặc thực hiện các nghiệp vụ liên quan khác.

Vai trò Consumer

Ngoài vai trò Producer, Task Service cũng hoạt động như một Consumer thông qua queue task.cleanup. Queue này được binding với các routing key:

- project.deleted
- board.deleted
- list.deleted

Khi Project Service phát sinh sự kiện xóa project, board hoặc list, CleanupConsumer của Task Service sẽ tiếp nhận event và thực hiện cascade soft-delete toàn bộ task liên quan. Quá trình xử lý được thực hiện thông qua các câu lệnh UPDATE hàng loạt như:

- softDeleteByProject
- softDeleteByBoard
- softDeleteByList

Cơ chế giao tiếp bất đồng bộ này giúp đảm bảo tính nhất quán dữ liệu giữa các service mà không cần thực hiện gọi REST đồng bộ. Đồng thời, hệ thống cũng hạn chế được tình trạng coupling chặt giữa các service và tăng khả năng chịu lỗi trong môi trường phân tán.

Tìm kiếm và lọc nâng cao

Task Service cung cấp endpoint filter đa tiêu chí cho phép người dùng tìm kiếm và lọc task theo nhiều điều kiện khác nhau như:

- project_id
- board_id
- list_id
- assignee_id
- priority

Cơ chế lọc linh hoạt này giúp người dùng dễ dàng truy xuất danh sách công việc theo từng dự án, board hoặc người phụ trách, từ đó nâng cao khả năng quản lý và theo dõi tiến độ công việc trong hệ thống.

Ngoài các điều kiện lọc cơ bản, hệ thống còn hỗ trợ tìm kiếm full-text theo keyword trên các trường title và description thông qua toán tử ILIKE của PostgreSQL.

Cơ chế này cho phép tìm kiếm gần đúng và không phân biệt chữ hoa chữ thường, giúp tăng trải nghiệm người dùng khi tra cứu task.

Kết quả truy vấn được hỗ trợ phân trang với mặc định 20 bản ghi trên mỗi trang nhằm tối ưu hiệu năng xử lý và giảm tải dữ liệu trả về cho client. Đồng thời, dữ liệu được sắp xếp theo position tăng dần để đảm bảo thứ tự hiển thị chính xác trên giao diện Kanban Board. Hệ thống cũng cho phép kết hợp nhiều điều kiện lọc trong cùng một request nhằm đáp ứng các nhu cầu tìm kiếm phức tạp.

Để tránh nguy cơ rò rỉ dữ liệu trên toàn hệ thống, endpoint filter yêu cầu bắt buộc phải cung cấp ít nhất một trong hai tham số:

- project_id
- assignee_id

Nếu thiếu cả hai tham số trên, hệ thống sẽ trả về mã lỗi: project_id_or_assignee_id_required

Cơ chế kiểm soát này giúp giới hạn phạm vi truy vấn dữ liệu phù hợp với quyền truy cập của người dùng, đồng thời tăng cường tính bảo mật cho hệ thống trong môi trường Microservice.

Danh mục API của Task Service

Bảng 3.3.3 Danh mục API của Task Service

STT	Method	Endpoint	Mô tả	Quyền yêu cầu
		Task		
1	POST	/api/v1/tasks	Tạo task mới trong một list	EDITOR
2	GET	/api/v1/tasks/{id}	Xem chi tiết một task	Member
3	PUT	/api/v1/tasks/{id}	Cập nhật thông tin task	EDITOR
4	DELETE	/api/v1/tasks/{id}	Xóa task (soft delete)	EDITOR

5	POST	/api/v1/tasks/{id}/restore	Khôi phục task đã xóa	EDITOR
6	POST	/api/v1/tasks/{id}/move	Di chuyển task sang list khác	EDITOR
7	GET	/api/v1/tasks?project_id=&board_id=&list_id=&assignee_id=&priority=&q=&page=&size=	Tìm kiếm & lọc task (phân trang)	Member
		Checklist		
8	POST	/api/v1/tasks/{taskId}/checklists	Tạo checklist cho task	EDITOR
9	GET	/api/v1/tasks/{taskId}/checklists	Lấy danh sách checklist của task	EDITOR
10	DELETE	/api/v1/checklists/{id}	Xóa checklist	EDITOR
11	POST	/api/v1/checklists/{id}/items	Thêm item vào checklist	EDITOR
12	PATCH	/api/v1/checklist-items/{id}	Cập nhật item (nội dung, trạng thái, vị trí)	EDITOR
13	DELETE	/api/v1/checklist-items/{id}	Xóa item khỏi checklist	EDITOR
		Label		
17	GET	/api/v1/labels?project_id=	Danh sách label theo dự án	Member
18	POST	/api/v1/labels?project_id=	Tạo label mới	ADMIN

19	PUT	/api/v1/labels/{id}	Cập nhật label	ADMIN
20	DELETE	/api/v1/labels/{id}	Xóa label	ADMIN
		Internal (Service-to-Service)		
23	GET	/internal/tasks/{id}/exists	Kiểm tra task tồn tại, trả về project_id, board_id, list_id	Không xác thực (mạng nội bộ)

3.3.4. Collaboration Service

Collaboration Service đóng vai trò trung tâm trong việc quản lý các hoạt động cộng tác của thành viên trong dự án, bao gồm thảo luận, đính kèm tệp và truy vết lịch sử hoạt động. Dịch vụ này sở hữu cơ sở dữ liệu taskflow_collab tách biệt.

Mô hình Dữ liệu (Database Schema)

Cơ sở dữ liệu của dịch vụ được thiết kế nhằm phục vụ các nghiệp vụ ghi chép dữ liệu hoạt động liên tục:

- comments: Lưu trữ nội dung bình luận dưới định dạng Markdown, tác giả bình luận, công việc đích và liên kết cha-con (parent_id) để hỗ trợ bình luận một cấp. Để tối ưu hóa phân quyền nhanh, bảng còn denormalize thêm trường project_id.
- attachments: Quản lý metadata của tệp đính kèm (tên file gốc, kiểu MIME, kích thước, khóa lưu trữ trên MinIO). Nội dung tệp thực tế được lưu trên MinIO S3 tại đường dẫn tasks/{taskId}/{uuid}.{ext} để tránh phình to database.
- activity_logs: Bảng nhật ký hoạt động có tính chất append-only (ai đã làm gì, lúc nào). Các thay đổi chi tiết được lưu linh hoạt dưới dạng jsonb để phục vụ hiển thị dòng thời gian.

Giao tiếp qua sự kiện và Đồng bộ RabbitMQ

Collaboration Service tích hợp chặt chẽ với RabbitMQ thông qua hàng đợi collab.activity.q để lắng nghe mọi sự kiện nghiệp vụ trong hệ thống. Khi sự kiện xảy ra, dịch vụ tiêu thụ thông điệp, trích xuất dữ liệu và tự động ghi nhận một bản ghi nhật ký hoạt động tương ứng vào bảng activity_logs. Cơ chế Idempotency dựa trên

event_id được áp dụng để bảo đảm không ghi trùng nhật ký khi RabbitMQ gửi lại tin nhắn nhiều lần do sự cố mạng.

Danh mục API của Collaboration Service

Bảng 3.3.4 Danh mục API của Collaboration Service

STT	Method	URL	Quyền hạn	Mô tả chức năng
1	POST	/api/v1/comments	EDITOR+	Tạo bình luận mới trong thẻ công việc
2	GET	/api/v1/comments/tasks/{taskId}	MEMBER	Lấy danh sách bình luận của công việc
3	PUT	/api/v1/comments/{id}	AUTHOR	Chỉnh sửa nội dung bình luận cá nhân
4	DELETE	/api/v1/comments/{id}	AUTHOR/ADMIN+	Xóa bình luận khỏi công việc

5	POST	/api/v1/attachments	EDITOR+	Tải tệp đính kèm lên công việc (lưu MinIO)
6	GET	/api/v1/attachments/{id}	MEMBER	Tải xuống tệp tin đính kèm
7	DELETE	/api/v1/attachments/{id}	UPLOADER/ ADMIN+	Xóa tệp đính kèm
8	GET	/api/v1/activities/projects/{projectId}	MEMBER	Xem dòng thời gian nhật ký hoạt động dự án

3.3.5. Notification Service

Notification Service đảm nhận vai trò quản lý thông báo trong ứng dụng, cung cấp kênh truyền thông realtime qua WebSocket và gửi thư điện tử bất đồng bộ đến người dùng. Dịch vụ này sở hữu cơ sở dữ liệu taskflow_notif riêng biệt.

Mô hình Dữ liệu (Database Schema)

Cơ sở dữ liệu được tinh gọn tối đa để phục vụ truy vấn tốc độ cao:

- notifications: Lưu trữ nội dung thông báo cá nhân, loại thông báo, trạng thái đọc, và liên kết điều hướng sâu để mở nhanh tài nguyên tương ứng.

- notification_preferences: Quản lý tùy chọn nhận thông báo của từng cá nhân (bật/tắt in-app, bật/tắt email, cấu hình chi tiết theo từng loại sự kiện được lưu dưới định dạng jsonb).

Đồng bộ Sự kiện RabbitMQ và Truyền thông Realtime

Tiêu thụ sự kiện từ RabbitMQ: Dịch vụ lắng nghe hàng đợi notification.q gắn với Topic Exchange taskflow.events để thu thập mọi sự kiện thay đổi dữ liệu nghiệp vụ (như khi có công việc mới được tạo, công việc di chuyển cột, được nhắc tên trong bình luận, hoặc được mời vào dự án).

Đẩy tin Realtime qua WebSocket STOMP: Khi nhận được sự kiện, dịch vụ sẽ tạo bản ghi thông báo trong cơ sở dữ liệu, đồng thời chuyển hóa dữ liệu sự kiện và chủ động đẩy tin nhắn trực tiếp xuống client thông qua kết nối WebSocket (sử dụng sub-protocol STOMP và SockJS fallback). Tin nhắn cá nhân được đẩy qua destination /user/queue/notifications, và tin nhắn broadcast bảng Kanban được đẩy qua /topic/board/{boardId}.

Xác thực kết nối: Hệ thống kiểm tra token JWT một lần duy nhất tại thời điểm HTTP Handshake nâng cấp giao thức để định danh phiên làm việc của người dùng.

Danh mục API của Notification Service

Bảng 3.3.5 Danh mục API của Notification Service

STT	Method	URL	Quyền hạn	Mô tả chức năng
1	GET	/api/v1/notifications	AUTH	Truy vấn danh sách thông báo cá nhân (phân trang)
2	PATCH	/api/v1/notifications/{id}/read	AUTH	Đánh dấu một thông báo cụ thể là đã đọc

3	PATCH	/api/v1/notifications/read-all	AUTH	Đánh dấu đã đọc toàn bộ tất cả thông báo
4	GET	/api/v1/notifications/unread-count	AUTH	Đếm số lượng thông báo chưa đọc
5	GET	/api/v1/notifications/preferences	AUTH	Lấy cấu hình tùy chọn nhận thông báo cá nhân
6	PUT	/api/v1/notifications/preferences	AUTH	Cập nhật cấu hình nhận thông báo (in-app/email)

3.4. Thiết kế cơ sở dữ liệu

3.4.1. Nguyên tắc thiết kế Database-per-Service

Hệ thống TaskFlow áp dụng triệt để nguyên tắc Database-per-Service nhằm đảm bảo tính độc lập và khả năng deploy riêng biệt cho từng microservice. Toàn bộ hệ thống bao gồm năm cơ sở dữ liệu PostgreSQL vật lý độc lập, trong đó mỗi service là chủ thể duy nhất có quyền đọc/ghi vào schema của mình. Việc giao tiếp dữ liệu xuyên ranh giới không sử dụng liên kết khóa ngoại vật lý mà dựa trên các tham chiếu logic kết hợp với cơ chế gọi REST đồng bộ để xác thực trạng thái và các sự kiện bất đồng bộ qua RabbitMQ để xử lý cascade khi dữ liệu gốc bị xóa. Cách thiết kế này loại bỏ rủi ro phụ thuộc schema và cho phép tối ưu hóa lưu trữ riêng lẻ cho từng dịch vụ.

Bảng 3.4.1. Danh sách các cơ sở dữ liệu của hệ thống TaskFlow

STT	Tên cơ sở dữ liệu	Service sở hữu	Vai trò chính
1	taskflow_user	User Service	Tài khoản người dùng, hồ sơ cá nhân, token đặt lại mật khẩu

STT	Tên cơ sở dữ liệu	Service sở hữu	Vai trò chính
2	taskflow_project	Project Service	Dự án, bảng Kanban, cột, sprint, thành viên dự án
3	taskflow_task	Task Service	Công việc, checklist, nhãn, phụ thuộc, người theo dõi
4	taskflow_collab	Collaboration Service	Bình luận, tệp đính kèm, nhật ký hoạt động
5	taskflow_notif	Notification Service	Thông báo và tùy chọn thông báo của người dùng

3.4.2. Lớp AuditEntity dùng chung

Để bảo đảm tính nhất quán về quy ước truy vết dữ liệu giữa các service, hệ thống định nghĩa một lớp cha trừu tượng tên là AuditEntity trong thư viện dùng chung taskflow-common. Mọi entity nghiệp vụ — từ User, Project, Task cho đến Comment, Notification — đều kế thừa lớp này. Nhờ đó, tất cả bảng trong toàn hệ thống đều có sáu cột kỹ thuật được tự động sinh ra, giúp tiết kiệm code lặp lại đồng thời giữ thiết kế đồng nhất.

Bảng 3.4.2. Các cột kỹ thuật chuẩn của lớp AuditEntity

Cột	Kiểu dữ liệu	Ý nghĩa
id	BIGSERIAL (khóa chính)	Khóa chính tự tăng
created_by	varchar(255)	Tên đăng nhập người tạo
created_at	bigint	Thời điểm tạo (epoch milliseconds)
last_updated_by	varchar(255)	Tên đăng nhập người sửa lần cuối

Cột	Kiểu dữ liệu	Ý nghĩa
last_updated_at	bigint	Thời điểm sửa lần cuối (epoch milliseconds)
deleted	boolean	Cờ đánh dấu đã xóa mềm, mặc định false

Để lưu trữ thời gian, hệ thống sử dụng kiểu bigint thay vì kiểu timestamp truyền thống của PostgreSQL. Quyết định này giúp loại bỏ hoàn toàn rủi ro sai lệch múi giờ khi dữ liệu đi qua các service đặt ở các múi giờ hoặc cấu hình khác nhau, tương thích trực tiếp với payload JSON trong các sự kiện RabbitMQ, đồng thời tối ưu hiệu năng so sánh, sắp xếp và lập chỉ mục.

Hệ thống áp dụng nguyên tắc xóa mềm thông qua cờ deleted kiểu boolean. Khi người dùng yêu cầu xóa một tài nguyên (công việc, bình luận, nhãn dán), bản ghi chỉ được đánh dấu deleted = true; các truy vấn nghiệp vụ đều thêm điều kiện deleted = false để bỏ qua dữ liệu đã xóa. Cơ chế này hỗ trợ chức năng "Thùng rác" cho phép khôi phục công việc trong vòng 30 ngày, đồng thời bảo toàn dữ liệu lịch sử cho Activity Log để truy vết về sau, ngoại trừ bảng activity_logs có tính chất append-only nên không cần cờ xóa.

3.4.3. Cơ sở dữ liệu User Service

Cơ sở dữ liệu taskflow_user chứa hai bảng nghiệp vụ. Bảng users là bảng trung tâm, lưu trữ thông tin tài khoản với các trường chính như tên đăng nhập, địa chỉ thư điện tử, mã băm mật khẩu (sử dụng thuật toán BCrypt với salt ngẫu nhiên), họ tên đầy đủ, ảnh đại diện, mô tả ngắn, ngày sinh, trạng thái tài khoản và một số trường phục vụ cơ chế khóa tài khoản. Trạng thái tài khoản được mô hình hóa dưới dạng enum với ba giá trị ACTIVE, LOCKED và INACTIVE. Đặc biệt, hai trường failed_attempts và last_login_at phục vụ chính sách bảo mật cơ bản: nếu một tài khoản đăng nhập thất bại quá năm lần liên tiếp, hệ thống sẽ tự động chuyển trạng thái sang LOCKED và yêu cầu reset mật khẩu trước khi đăng nhập lại.

Bảng thứ hai là password_resets, lưu các token đặt lại mật khẩu được sinh ra khi người dùng yêu cầu chức năng "Quên mật khẩu". Mỗi token sống trong vòng 30 phút và chỉ được sử dụng đúng một lần. Để bảo mật, hệ thống không lưu token gốc mà chỉ lưu giá trị băm SHA-256 của nó; khi người dùng gửi yêu cầu reset, server băm

token nhận được rồi so sánh với cột token_hash. Cờ used đảm bảo một token không thể bị tái sử dụng — sau lần đầu thành công, cờ được đặt thành true và mọi yêu cầu tiếp theo với token này đều bị từ chối.

Bảng users có hai chỉ mục duy nhất là idx_users_email và idx_users_username để hỗ trợ truy vấn đăng nhập. Bảng password_resets có chỉ mục theo token_hash để tra cứu nhanh khi xác thực yêu cầu reset.



Hình 3.4.3. Sơ đồ thực thể-quan hệ — User Service (taskflow_user).

3.4.4. Cơ sở dữ liệu Project Service

Cơ sở dữ liệu taskflow_project mô hình hóa cấu trúc tổ chức công việc theo phong cách lấy cảm hứng từ Trello và Jira. Năm bảng nghiệp vụ chính gồm projects, project_members, boards, lists và sprints có quan hệ cha-con đa cấp như sau: một dự án chứa nhiều bảng, một bảng chứa nhiều cột, một dự án có nhiều thành viên và nhiều sprint.

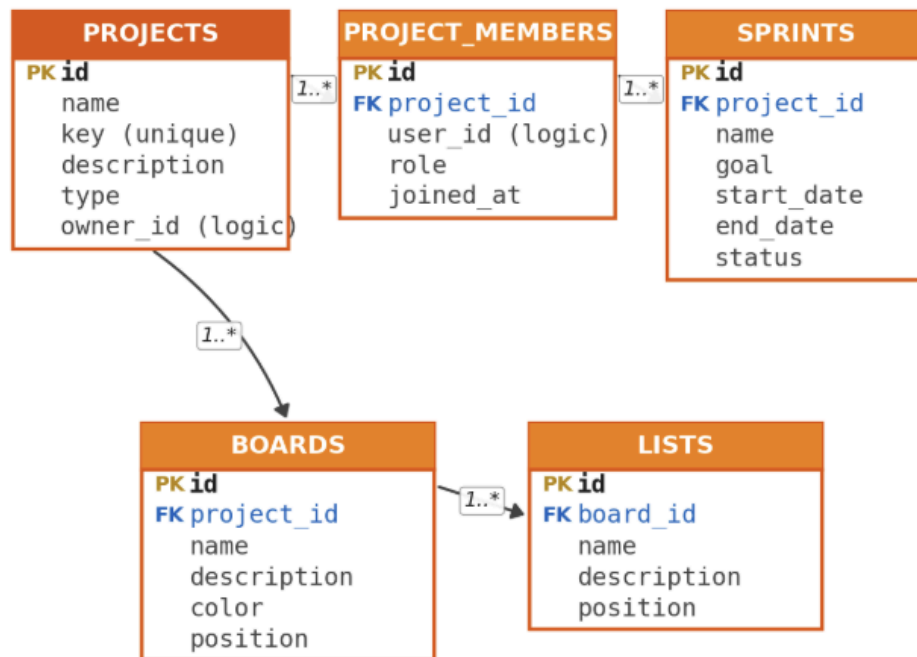
Bảng projects lưu thông tin dự án bao gồm tên hiển thị, mô tả, loại dự án, tên người sở hữu, và đặc biệt là trường key viết hoa duy nhất kiểu Jira (ví dụ TFM, MOB). Trường key này sau đó được dùng để sinh mã công việc dạng TFM-101, TFM-102 giúp người dùng dễ tham chiếu trong giao tiếp hằng ngày. Trường owner_id lưu mã người sở hữu nhưng không có khóa ngoại đến bảng users ở User Service — đây là tham chiếu logic xuyên service.

Bảng project_members đóng vai trò bảng nối nhiều-nhiều giữa người dùng và dự án, kèm thêm thông tin về vai trò của thành viên. Bảng có ràng buộc UNIQUE(project_id, user_id) để tránh một người dùng được thêm hai lần vào cùng một dự án. Trường role nhận một trong năm giá trị OWNER, ADMIN, EDITOR,

COMMENTER và VIEWER. Logic phân quyền xây trên trường này được trình bày chi tiết ở mục 3.8.

Bảng boards và lists mô hình hóa cấu trúc Kanban. Mỗi bảng thuộc đúng một dự án, có thể tùy chỉnh tên, mô tả, mã màu nền. Mỗi cột (list) thuộc đúng một bảng và mang ý nghĩa trạng thái công việc (ví dụ "Cần làm", "Đang làm", "Đã xong"). Cả hai bảng đều có trường position kiểu số nguyên để hỗ trợ chức năng sắp xếp bằng kéo-thả: khi người dùng kéo một list sang vị trí mới, server cập nhật lại trường position cho các list bị ảnh hưởng.

Bảng sprints lưu thông tin về các chu kỳ làm việc cố định theo phương pháp Agile/Scrum. Mỗi sprint có tên, mục tiêu, thời gian bắt đầu và kết thúc cùng trạng thái nhận một trong ba giá trị PLANNING (đang lên kế hoạch), ACTIVE (đang chạy) và CLOSED (đã đóng). Mặc dù sprint được khai báo ở Project Service, mối quan hệ giữa sprint và công việc lại được lưu ở Task Service thông qua cột sprint_id của bảng tasks — đây là một ví dụ điển hình về cách hệ thống tách thuộc tính nghiệp vụ về service có ranh giới ngữ nghĩa phù hợp nhất.



Hình 3.4.4. Sơ đồ thực thể-quan hệ — Project Service (taskflow_project).

3.4.5. Cơ sở dữ liệu Task Service

Cơ sở dữ liệu `taskflow_task` là cơ sở dữ liệu phức tạp nhất trong hệ thống, phản ánh vai trò trung tâm của công việc (task) trong toàn bộ nghiệp vụ. Bảng nghiệp vụ gồm `tasks`, `labels`, `task_label_mappings`, `checklists`, `checklist_items`, `task_dependencies` và `task_watchers` hoạt động cùng nhau để mô tả mọi khía cạnh của một công việc.

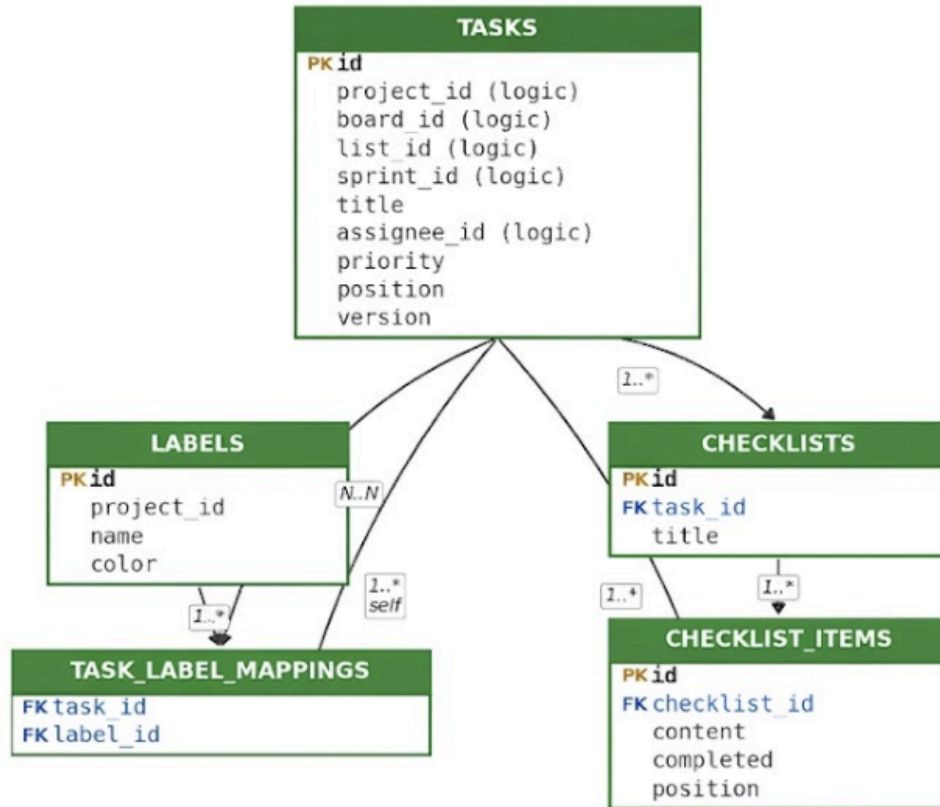
Bảng `tasks` là bảng trung tâm. Mỗi công việc có tiêu đề, mô tả (hỗ trợ định dạng Markdown), người được giao, người tạo, hạn hoàn thành, độ ưu tiên nhận giá trị URGENT, HIGH, MEDIUM hoặc LOW. Một điểm thiết kế quan trọng là bảng `tasks` lưu cùng lúc bốn mã `project_id`, `board_id`, `list_id` và `sprint_id`. Việc này thoạt nhìn có vẻ thừa (`list_id` đã ngầm xác định `board_id` và `project_id` thông qua quan hệ ở Project Service), nhưng thực chất là denormalize có chủ đích. Lý do là truy vấn lọc công việc theo dự án, theo bảng hay theo sprint là thao tác cực kỳ phổ biến trên giao diện người dùng; nếu phải gọi sang Project Service mỗi lần để tra cứu cấu trúc cha-con, hiệu năng sẽ kém. Khi `list` bị chuyển sang board khác (kịch bản hiếm), sự kiện `list.moved` sẽ kích hoạt cập nhật đồng bộ `board_id` cho các công việc liên quan.

Để hỗ trợ tình huống nhiều người dùng cùng thao tác trên một công việc, bảng `tasks` áp dụng cơ chế khóa lạc quan (optimistic locking) qua cột `version` được đánh dấu bằng chú thích `@Version` của JPA. Mỗi lần cập nhật, giá trị `version` tăng lên một đơn vị; nếu hai người cùng cố cập nhật một công việc gần như đồng thời (ví dụ kéo cùng một công việc vào hai cột khác nhau), người thứ hai sẽ nhận về mã lỗi HTTP 409 và phải nạp lại trạng thái mới nhất.

Bảng `labels` lưu các nhãn dán có thể gán cho công việc (ví dụ "frontend", "bug", "khẩn cấp"). Mỗi nhãn thuộc một dự án và có tên cùng mã màu. Bảng nối nhiều-nhiều `task_label_mappings` cho phép một công việc gán nhiều nhãn và một nhãn áp dụng cho nhiều công việc. Quyết định đặt bảng `labels` trong Task Service (thay vì Project Service) là để tránh phải gọi xuyên service mỗi lần đọc chi tiết công việc — nhãn được hiển thị cùng với công việc nên đặt trong cùng một cơ sở dữ liệu là hợp lý nhất.

Hai bảng `checklists` và `checklist_items` mô tả danh sách công việc con bên trong một công việc lớn. Mỗi `checklist` có tiêu đề và chứa nhiều `checklist_items`; mỗi item có nội dung văn bản, cờ `completed` và vị trí sắp xếp. Người dùng có thể đánh dấu hoàn thành từng item để theo dõi tiến độ chi tiết hơn ở cấp dưới của công việc.

Để hỗ trợ các truy vấn phổ biến, bảng tasks có bốn chỉ mục: idx_tasks_list (lọc theo cột Kanban), idx_tasks_assignee (lọc theo người được giao — phục vụ trang "Công việc của tôi"), idx_tasks_project (lọc theo dự án) và idx_tasks_due_date (lọc theo hạn hoàn thành — phục vụ chức năng quét công việc sắp đến hạn).



Hình 3.4.5. Sơ đồ thực thể-quan hệ — Task Service (taskflow_task).

3.4.6. Cơ sở dữ liệu Collaboration Service

Cơ sở dữ liệu taskflow_collab chứa ba bảng phục vụ ba khía cạnh khác nhau của hoạt động cộng tác: bình luận, tệp đính kèm và nhật ký hoạt động.

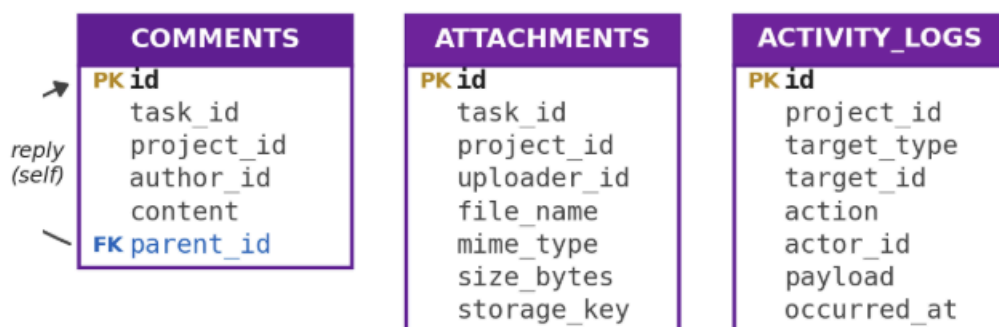
Bảng comments lưu các bình luận của người dùng trên một công việc cụ thể. Mỗi bình luận có nội dung dạng văn bản Markdown, tác giả (author_id), mã công việc (task_id) và một trường parent_id tùy chọn để hỗ trợ trả lời bình luận một cấp. Đáng chú ý, bảng còn lưu thêm trường project_id mặc dù về mặt logic có thể suy ra từ task_id. Lý do là khi kiểm tra quyền truy cập bình luận, hệ thống cần biết project_id để gọi sang Project Service tra cứu vai trò người dùng; nếu phải qua Task Service trung

gian sẽ thêm một bước mạng và tăng độ trễ. Việc lưu trực tiếp `project_id` là một dạng denormalize có chủ đích.

Bảng `attachments` lưu metadata của các tệp đính kèm, còn nội dung file thực tế lưu trên hệ thống lưu trữ đối tượng MinIO (tương thích S3). Mỗi bản ghi gồm tên file gốc, kiểu MIME, kích thước (giới hạn tối đa 25 MB) và một trường `storage_key` lưu đường dẫn trên MinIO theo định dạng `tasks/{taskId}/{uuid}.{ext}`. Quyết định tách metadata khỏi nội dung file là tiêu chuẩn ngành: cơ sở dữ liệu quan hệ tối ưu cho truy vấn có cấu trúc, trong khi hệ thống đối tượng tối ưu cho lưu trữ tệp lớn; ép cả hai vào PostgreSQL sẽ dẫn đến phình to và làm chậm sao lưu.

Bảng `activity_logs` là bảng append-only, lưu toàn bộ hoạt động xảy ra trên các đối tượng trong dự án để cung cấp dòng thời gian (timeline) cho người dùng. Mỗi bản ghi mô tả "ai đã làm gì với đối tượng nào, vào lúc nào". Các trường chính gồm `target_type` (PROJECT, BOARD, LIST, TASK, COMMENT), `target_id`, `action` (CREATED, UPDATED, DELETED, MOVED, ASSIGNED, ...), `actor_id` (người gây ra hành động) và `occurred_at`. Đặc biệt, trường `payload` kiểu jsonb lưu trạng thái chi tiết của thay đổi — ví dụ với hành động cập nhật, `payload` chứa cặp giá trị trước-sau của các trường bị thay đổi. Việc dùng kiểu jsonb cho phép linh hoạt với mọi loại thay đổi mà không cần biến bảng thành "wide table" với hàng trăm cột không hữu dụng.

Để hỗ trợ trang nhật ký hoạt động phải hiển thị nhanh các sự kiện gần đây nhất của một dự án, bảng có chỉ mục `idx_activity_project_time` (`project_id`, `occurred_at` DESC). Khi cần xem toàn bộ lịch sử của một đối tượng cụ thể, chỉ mục `idx_activity_target` (`target_type`, `target_id`) hỗ trợ tra cứu hiệu quả.

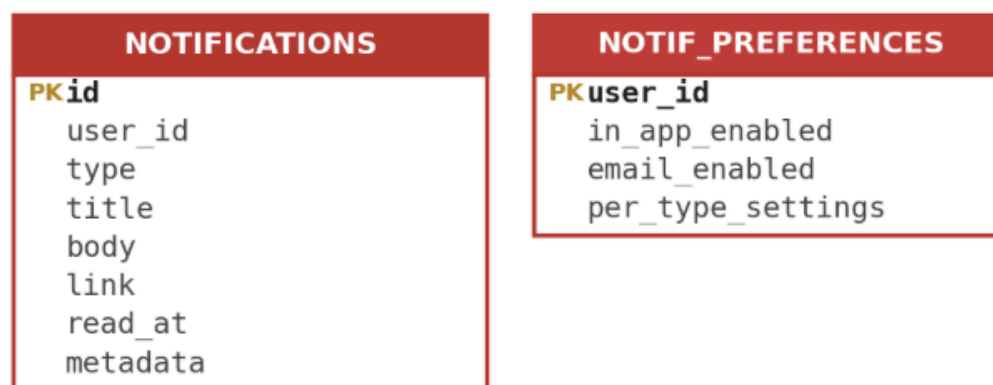


Hình 3.4.6. Sơ đồ thực thể-quan hệ — Collaboration Service (taskflow_collab).

3.4.7. Cơ sở dữ liệu Notification Service

Cơ sở dữ liệu taskflow_notif đơn giản hơn ba cơ sở dữ liệu trước, chỉ gồm hai bảng. Bảng notifications lưu các thông báo dành cho người dùng, mỗi bản ghi gồm người nhận (user_id), kiểu thông báo (nhận giá trị trong tập hợp TASK_ASSIGNED, TASK_DUE_SOON, COMMENT_ADDED, MENTION, PROJECT_INVITED, ...), tiêu đề, nội dung, liên kết sâu để mở thẳng đến đối tượng liên quan trên giao diện, và trường read_at ghi thời điểm người dùng đã đọc. Trường read_at được thiết kế dạng nullable: giá trị NULL nghĩa là chưa đọc, giá trị có thực nghĩa là đã đọc kèm thời điểm cụ thể. Cách này giúp truy vấn đếm số thông báo chưa đọc (phục vụ huy hiệu badge ở góc giao diện) chỉ cần dùng điều kiện read_at IS NULL. Chỉ mục idx_notif_user_unread(user_id, read_at) được thiết kế đặc biệt theo dạng partial index để tăng tốc truy vấn này.

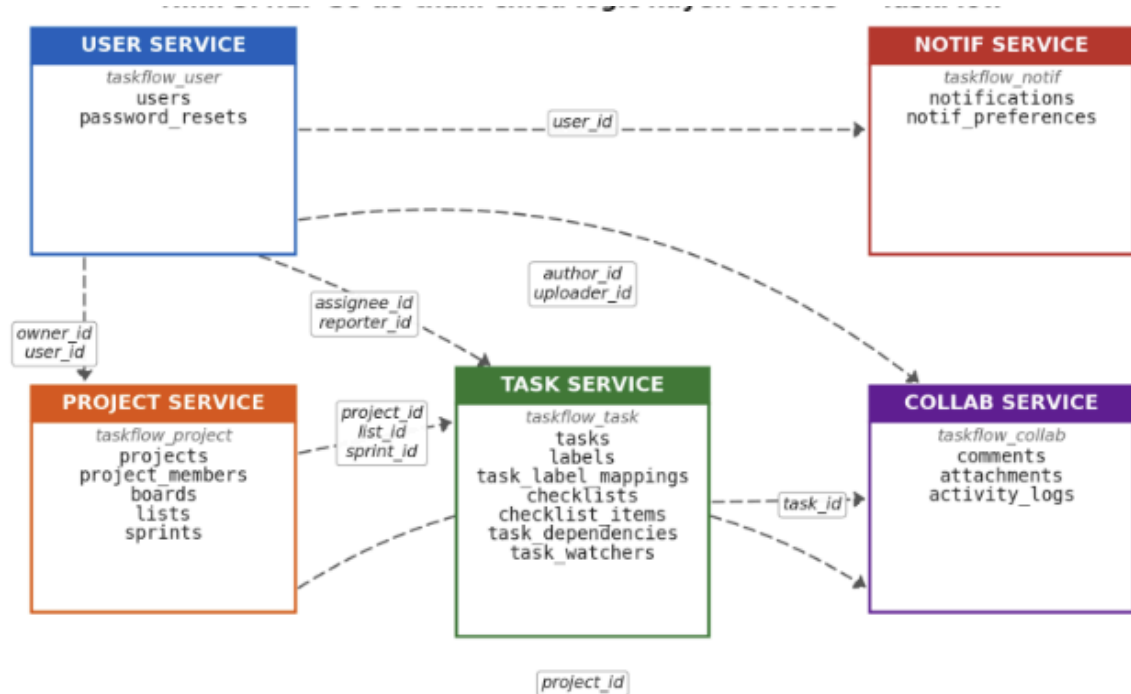
Bảng notification_preferences lưu tùy chọn nhận thông báo của từng người dùng. Bảng này dùng user_id làm khóa chính trực tiếp, không có khóa thay thế — mỗi người dùng có đúng một bản ghi tùy chọn. Các trường chính gồm in_app_enabled (bật/tắt thông báo trong ứng dụng), email_enabled (bật/tắt thông báo qua thư điện tử) và per_type_settings kiểu jsonb cho phép tùy chỉnh chi tiết theo từng kiểu thông báo (ví dụ tắt thông báo "bình luận" nhưng giữ thông báo "được giao việc").



Hình 3.4.7. Sơ đồ thực thể-quan hệ — Notification Service (taskflow_notif).

3.4.8. Quan hệ logic xuyên service

Mặc dù năm cơ sở dữ liệu hoàn toàn độc lập về mặt vật lý, về mặt logic chúng vẫn tham chiếu lẫn nhau thông qua các mã định danh. Hình 3.4.1 minh họa toàn bộ các quan hệ logic này dưới dạng sơ đồ tham chiếu.



Hình 3.4.8. Sơ đồ tham chiếu logic xuyên service của hệ thống TaskFlow.

Bảng dưới đây tổng hợp các cơ chế bảo đảm tính nhất quán cho từng loại tham chiếu logic.

Bảng 3.4.8. Cơ chế đảm bảo nhất quán cho tham chiếu xuyên service

Quan hệ logic	Cơ chế kiểm tra khi tạo	Cơ chế dọn dẹp khi xóa
task → project	Task Service gọi /internal/projects/{id}/exists	Sự kiện project.deleted → Task Service xóa cascade
task → list	Task Service gọi /internal/lists/{id}/board (vừa verify, vừa lấy board_id)	Sự kiện list.deleted → Task Service chuyển task sang list mặc định hoặc xóa

Quan hệ logic	Cơ chế kiểm tra khi tạo	Cơ chế dọn dẹp khi xóa
comment → task	Collaboration Service kiểm tra task_id qua Task Service	Sự kiện task.deleted → Collaboration xóa bình luận, tệp đính kèm
notification → user	Notification Service không verify (chấp nhận eventual)	Hiện không xóa thông báo cũ khi xóa user — chỉ ẩn

Tóm lại, mô hình Database-per-Service trong TaskFlow đánh đổi sự thuận tiện của khóa ngoại tập trung lấy lợi ích về khả năng deploy độc lập, khả năng scale theo từng service và phạm vi ảnh hưởng của sự cố được khoanh vùng. Việc duy trì tính nhất quán xuyên biên giới dữ liệu được giải quyết hiệu quả bằng hai cơ chế: kiểm tra đồng bộ qua REST và xử lý dọn dẹp bất đồng bộ qua sự kiện RabbitMQ sẽ được trình bày lần lượt ở các mục tiếp theo.

3.5. Thiết kế giao tiếp REST

3.5.1. Hai dạng giao tiếp trong hệ thống TaskFlow

Để đảm bảo sự cân bằng giữa tính nhất quán dữ liệu và hiệu năng vận hành, hệ thống TaskFlow kết hợp song song hai phương thức giao tiếp: đồng bộ (RESTful API qua HTTP) và bất đồng bộ (sự kiện qua RabbitMQ). Giao tiếp đồng bộ được áp dụng khi luồng nghiệp vụ yêu cầu phản hồi tức thời từ dịch vụ khác (ví dụ: Task Service kiểm tra vai trò thành viên tại Project Service trước khi cho phép tạo công việc). Ngược lại, giao tiếp bất đồng bộ được sử dụng cho các tác vụ phụ trợ không cản trở luồng chính (như ghi nhật ký hoạt động, gửi thông báo, dọn dẹp dữ liệu), giúp cô lập sự cố và tăng khả năng chịu tải của hệ thống. Sự khác biệt giữa hai mô hình được so sánh chi tiết trong Bảng 3.5.1.

Bảng 3.5.1. So sánh hai dạng giao tiếp trong TaskFlow

Tiêu chí	REST đồng bộ	RabbitMQ bất đồng bộ
Mô hình	Yêu cầu – Phản hồi	Phát hành – Đăng ký
Chờ kết quả	Có, blocking	Không, fire-and-forget
Phụ thuộc giữa hai bên	Temporal coupling	Loose coupling hoàn toàn

Tiêu chí	REST đồng bộ	RabbitMQ bất đồng bộ
Phù hợp khi	Cần kết quả ngay	Phát ra sự kiện để service khác phản ứng
Tác động khi service đích down	Caller cũng lỗi nếu không có Circuit Breaker	Thông điệp tích trong hàng đợi, xử lý sau
Ví dụ trong TaskFlow	Task hỏi Project về vai trò	Task phát hành sự kiện task.created

Trong mục 3.5 này, nhóm trình bày chi tiết về phân giao tiếp đồng bộ qua REST; phân giao tiếp bất đồng bộ qua RabbitMQ sẽ được trình bày trong mục 3.6.

3.5.2. Phân loại API trong hệ thống

Hệ thống TaskFlow có ba loại API REST với mục đích và phạm vi sử dụng hoàn toàn khác nhau, thể hiện trong bảng dưới đây.

Bảng 3.5.2. Các loại API trong hệ thống TaskFlow

Loại	Tiền tố URL	Đối tượng gọi	Đặc điểm
Public API	/api/v1/...	Client (FE, mobile) qua Gateway	Có wrapper ApiResponse<T>, kiểm tra JWT ở Gateway
Internal API	/internal/...	Service-to-service trong Docker network	Không có wrapper, Gateway chặn từ ngoài
WebSocket	/ws/...	Client kết nối realtime	Xác thực qua handshake

Public API là cổng giao tiếp chính dành cho client, luôn đi qua API Gateway để xác thực JWT, đo lường rate limit và được đóng gói trong cấu trúc phản hồi chuẩn ApiResponse<T> giúp đồng nhất giao diện phản hồi.

Internal API phục vụ giao tiếp trực tiếp giữa các service trong mạng Docker nội bộ. Các endpoint này không được bao bọc bởi ApiResponse<T> mà trả về trực tiếp đối

tượng DTO giúp tối giản xử lý, đồng thời được API Gateway chặn hoàn toàn đối với các yêu cầu từ bên ngoài Docker network.

3.5.3. Các luồng giao tiếp REST nội bộ

Trong hệ thống TaskFlow, có bốn service phải gọi REST nội bộ đến nhau trong quá trình xử lý nghiệp vụ. Bảng 3.5.3 liệt kê đầy đủ tám luồng REST nội bộ này.

Bảng 3.5.3. Các luồng REST nội bộ giữa các service

STT	Service gọi	Service đích	Endpoint nội bộ	Mục đích nghiệp vụ
1	Task	Project	GET /internal/projects/{id}/members/{userId}/role	Lấy vai trò của người dùng trong dự án để kiểm tra quyền
2	Task	Project	GET /internal/lists/{id}/board	Xác minh list_id hợp lệ và lấy về board_id, project_id
3	Task	Project	GET /internal/projects/{id}/exists	Kiểm tra dự án còn tồn tại
4	Collaboration	Project	GET /internal/projects/{id}/members/{userId}/role	Kiểm tra quyền bình luận và tải tệp đính kèm
5	Collaboration	Project	GET /internal/projects/{id}/exists	Xác minh dự án khi ghi nhật ký hoạt động
6	Notification	User	GET /internal/users/{id}/contact	Lấy địa chỉ thư điện tử và tên hiển thị để gửi thông báo

STT	Service gọi	Service đích	Endpoint nội bộ	Mục đích nghiệp vụ
7	Notification	Project	GET /internal/projects/{id}/members/{userId}/role	Kiểm tra quyền subscribe WebSocket theo dõi bảng Kanban
8	Notification	User	POST /internal/users/exists	Xác minh hàng loạt mã người dùng khi xử lý @mention

Các luồng REST nội bộ chủ yếu tập trung vào việc kiểm tra vai trò người dùng trong dự án tại Project Service. Do đó, Project Service là nguồn dữ liệu duy nhất về quyền thành viên, các service khác luôn gọi sang đây mỗi khi cần xác minh (có bộ đệm cache Redis hỗ trợ).

3.5.4. Cơ chế gọi qua Eureka và bảo vệ bằng Resilience4j

Trong môi trường phân tán, các instance của service có địa chỉ IP động và thay đổi liên tục. Hệ thống TaskFlow sử dụng Spring Cloud Eureka để đăng ký và phát hiện dịch vụ tự động, giúp định tuyến yêu cầu thông qua tên logic (ví dụ: taskflow-project) thay vì IP cố định. Nhằm ngăn ngừa lỗi xếp tầng khi các cuộc gọi REST xuyên service bị trễ hoặc gặp sự cố, hệ thống tích hợp Resilience4j tại lớp WebClient với bốn cơ chế bảo vệ chính như cấu hình trong Bảng 3.5.4. Trong đó, Circuit Breaker đóng vai trò chốt chặn quan trọng, tự động ngắt (OPEN) các yêu cầu tiếp theo khi tỷ lệ lỗi vượt ngưỡng 50% trong cửa sổ trượt, tránh gây nghẽn tài nguyên của dịch vụ gọi và giúp dịch vụ bị lỗi có thời gian tự phục hồi.

Bảng 3.5.4. Cấu hình Resilience4j cho REST gọi nội bộ

Cơ chế	Tham số	Ý nghĩa
Circuit Breaker	slidingWindowSize=10, failureRateThreshold=50 %, waitDurationInOpenState =10s	Khi tỉ lệ thất bại vượt 50% trong 10 yêu cầu gần nhất, ngắt mạch 10 giây — mọi gọi tiếp theo trả về fallback ngay lập tức
Timeout	3 giây	Tránh thread caller bị treo vô hạn nếu service đích không phản hồi
Retry	3 lần, backoff 0.5s / 1s / 2s	Tự động thử lại với các lỗi tạm thời (5xx, timeout); không retry với 4xx
Fallback	Trả null hoặc giá trị mặc định	Đảm bảo caller có phản hồi xác định ngay cả khi service đích hoàn toàn down

3.5.5. Chiến lược cache để giảm tải REST

Do kiểm tra vai trò người dùng xảy ra ở hầu hết các yêu cầu nghiệp vụ, hệ thống sử dụng Redis làm lớp cache đệm để giảm tải cho Project Service

Bảng 3.5.5. Các khóa cache Redis được dùng để giảm tải REST nội bộ

Khóa cache	Giá trị	Thời gian sống	Bị vô hiệu khi
role:{projectId}:{userId}	Chuỗi vai trò (OWNER, ADMIN, ..., hoặc __NULL__)	5 phút	Sự kiện project.member.ad ded / removed / role_changed
proj:exists:{projectId}	true / false	10 phút	Sự kiện project.created / project.deleted

Khóa cache	Giá trị	Thời gian sống	Bị vô hiệu khi
user:contact: {userId}	JSON {email, name}	10 phút	Sự kiện user.updated

Hệ thống thực hiện cache cả giá trị âm (người dùng không thuộc dự án) dưới dạng marker đặc biệt `__NULL__` để tránh lãng phí cuộc gọi gọi REST liên tiếp. Nhằm đảm bảo tính tức thời khi vai trò thay đổi, cơ chế vô hiệu hóa cache được kích hoạt lập tức thông qua các sự kiện bất đồng bộ từ RabbitMQ khi có thành viên bị xóa hoặc sửa vai trò ở Project Service, giúp độ trễ áp dụng vai trò mới chỉ còn cỡ vài trăm milliseconds.

3.6. Thiết kế Message Queue

3.6.1. Kiến trúc RabbitMQ trong hệ thống

Kiến trúc truyền tin bất đồng bộ của TaskFlow sử dụng RabbitMQ làm trục xương sống kết nối các service. Khi một sự kiện nghiệp vụ xảy ra (ví dụ: tạo công việc mới), service nguồn (Producer) đóng vai trò phát hành thông điệp lên kênh trao đổi trung tâm `taskflow.events`. Thông điệp sau đó được định tuyến tự động vào các hàng đợi đích (Queues) của các service tiêu thụ (Consumers) dựa trên khóa định tuyến (Routing Key) tương ứng. Mô hình này giúp tách biệt hoàn toàn ranh giới xử lý của các dịch vụ, bảo đảm thông điệp vẫn được lưu giữ an toàn trong hàng đợi ngay cả khi dịch vụ tiêu thụ tạm thời ngừng hoạt động. Cấu hình chi tiết các hàng đợi và sơ đồ topology được thể hiện trong Bảng 3.6.1 và Hình 3.6.1.

Hệ thống sử dụng duy nhất một kênh trao đổi chính `taskflow.events` và một kênh phụ `taskflow.events.dlx` phục vụ thông điệp lỗi (Dead Letter Exchange). Mọi exchange, queue và message đều được cấu hình với thuộc tính `durable = true` và `delivery_mode = 2` để bảo đảm thông tin không bị mất mát khi broker khởi động lại.

Bảng 3.6.1. Các hàng đợi chính và service tiêu thụ tương ứng

Tên hàng đợi	Service tiêu thụ	Các khóa định tuyến binding	Mục đích nghiệp vụ
notification.q	Notification Service	task.*, project.member.*, comment.*, attachment.uploaded	Gửi thông báo realtime và thư điện tử
collab.activity.q	Collaboration Service	task.*, project.*, board.*, list.*, comment.*, attachment.*	Ghi nhật ký hoạt động chi tiết
task.cleanup.q	Task Service	project.deleted, list.deleted, board.deleted	Dọn dẹp công việc khi đối tượng cha bị xóa

Hình 3.6.1. Sơ đồ topology RabbitMQ của hệ thống TaskFlow.

3.6.2. Thiết kế Exchange — Lý do chọn Topic Exchange

Hệ thống TaskFlow sử dụng duy nhất một Topic Exchange (taskflow.events) để định tuyến toàn bộ sự kiện nghiệp vụ. Lựa chọn này mang lại ba lợi thế thiết kế quan trọng so với các loại Exchange khác (Bảng 3.6.2). Thứ nhất, nó hỗ trợ cơ chế so khớp linh hoạt theo cấu trúc phân cấp <lĩnh vực>.<thực thể>.<động từ>, cho phép các service đăng ký nhận tin từ một sự kiện cụ thể (task.created) hoặc toàn bộ nhóm sự kiện liên quan (task.*). Thứ hai, nó đảm bảo tính mở rộng cao, cho phép bổ sung các sự kiện mới hoặc thêm các microservice tiêu thụ mới (ví dụ: Reporting Service) mà không cần thay đổi hay triển khai lại mã nguồn của các dịch vụ hiện tại.

Bảng 3.6.2. So sánh các loại Exchange của RabbitMQ

Loại Exchange	Cơ chế định tuyến	Phù hợp khi
Direct	Khóa định tuyến phải khớp chính xác với binding key	Mỗi sự kiện đến đúng một loại consumer cố định

Loại Exchange	Cơ chế định tuyến	Phù hợp khi
Fanout	Broadcast tới mọi hàng đợi gắn vào	Tất cả consumer đều cần nhận mọi sự kiện
Topic	Khóa định tuyến match pattern (sử dụng * và #)	Mỗi consumer chỉ muốn một subset sự kiện theo chủ đề
Headers	Match theo header thay vì khóa định tuyến	Trường hợp đặc biệt, hiếm dùng

3.6.3. Quy ước đặt khóa định tuyến

Khóa định tuyến (Routing Key) của mọi sự kiện trong TaskFlow tuân thủ nghiêm ngặt quy ước ba phần: <lĩnh vực>.<thực thể>.<động từ>, viết thường hoàn toàn và động từ ở dạng quá khứ phân từ (past participle) nhằm thể hiện sự kiện đã xảy ra (ví dụ: project.member.removed, task.moved). Quy ước này giúp ngữ nghĩa rõ ràng, đồng thời tương thích trực tiếp với cơ chế so khớp mẫu (* và #) của Topic Exchange. Danh sách các sự kiện được tổng hợp chi tiết trong Bảng 3.6.3.

Bảng 3.6.3. Danh mục đầy đủ các sự kiện trong hệ thống

Khóa định tuyến	Service phát hành	Service tiêu thụ	Mô tả ngữ nghĩa
project.created	Project	Collaboration, Notification	Dự án mới được tạo
project.deleted	Project	Task, Collaboration, Notification	Dự án bị xóa (cascade)
project.member.added	Project	Notification, Collaboration	Thành viên mới được thêm vào dự án
project.member.removed	Project	Notification, Collaboration	Thành viên bị xóa khỏi dự án
project.member.role_changed	Project	Notification, Collaboration	Vai trò của thành viên bị thay đổi
board.created	Project	Collaboration	Bảng Kanban mới

Khóa định tuyến	Service phát hành	Service tiêu thụ	Mô tả ngữ nghĩa
board.deleted	Project	Collaboration	Bảng Kanban bị xóa
list.created	Project	Collaboration	Cột Kanban mới
list.deleted	Project	Collaboration, Task	Cột Kanban bị xóa
task.created	Task	Notification, Collaboration	Công việc mới
task.updated	Task	Notification, Collaboration	Công việc bị sửa, kèm diff field
task.moved	Task	Notification, Collaboration	Công việc bị chuyển giữa các cột
task.assigned	Task	Notification, Collaboration	Người được giao công việc bị thay đổi
task.deleted	Task	Collaboration	Công việc bị xóa
task.due_soon	Task (scheduler)	Notification	Công việc sắp đến hạn (sự kiện hệ thống)
task.overdue	Task (scheduler)	Notification	Công việc đã quá hạn (sự kiện hệ thống)
task.dependency.changed	Task	Notification, Collaboration	Mối quan hệ phụ thuộc bị sửa
comment.added	Collaboration	Notification	Bình luận mới (kèm @mention)
attachment.uploaded	Collaboration	Notification	Tệp đính kèm mới được tải lên

3.6.4. Cấu trúc Envelope chung của sự kiện

Mọi sự kiện nghiệp vụ đều được đóng gói trong một cấu trúc envelope chung định nghĩa tại thư viện dùng chung taskflow-events-contract (Bảng 3.6.4). Điều này giúp đồng nhất hóa cấu trúc tin nhắn và cho phép các service parse dữ liệu an toàn. Phần payload (data) chứa thông tin chi tiết thay đổi tùy theo loại sự kiện, đặc biệt sự kiện task.updated sẽ gửi kèm cả cặp giá trị cũ-mới (changes object) giúp Collaboration Service ghi nhật ký chi tiết và Notification Service lọc gửi thông báo chính xác.

Bảng 3.6.4. Cấu trúc envelope chung của sự kiện

Trường	Kiểu	Bắt buộc	Vai trò
event_id	UUID v4	Có	Định danh duy nhất, dùng để khử trùng lặp ở consumer
event_type	Chuỗi	Có	Trùng với khóa định tuyến, dùng để rẽ nhánh xử lý
schema_version	Số nguyên	Có	Tăng khi có breaking change, hỗ trợ chạy song song nhiều phiên bản
occurred_at	Số nguyên (epoch ms)	Có	Thời điểm sự kiện xảy ra thực sự
actor_id	Số nguyên	Có (trừ sự kiện hệ thống)	Mã người dùng gây ra hành động
trace_id	Chuỗi	Có	Mã truy vết được truyền từ service nguồn (xem mục 3.5.6)
data	Đối tượng	Có	Tải trọng nghiệp vụ cụ thể, khác

Trường	Kiểu	Bắt buộc	Vai trò
			nhau giữa các loại sự kiện

3.6.5. Quy trình một sự kiện đầu cuối — Ví dụ Tạo công việc

Để hiểu rõ cách các thành phần phối hợp với nhau, mục này mô tả chi tiết một quy trình điển hình: tình huống một người dùng tạo một công việc mới có người được giao.

Bước 1: Người dùng A trên giao diện gửi yêu cầu POST đến `/api/v1/tasks` qua API Gateway. Gateway xác thực JWT, gắn các header định danh và chuyển tiếp yêu cầu đến Task Service.

Bước 2: Task Service nhận yêu cầu. Trước tiên, nó kiểm tra quyền của người dùng A đối với dự án chứa cột mục tiêu (gọi REST đến Project Service hoặc tra cache Redis). Nếu được phép, Task Service thực thi thao tác chèn bản ghi mới vào bảng tasks trong cơ sở dữ liệu `taskflow_task`.

Bước 3: Sau khi commit thành công, Task Service phát hành sự kiện `task.created` lên kênh trao đổi `taskflow.events` với khóa định tuyến là `task.created`. Việc phát hành được bọc trong khối try-catch `AmqpException`: nếu RabbitMQ tạm thời không khả dụng, sự kiện không được phát hành nhưng Task Service vẫn trả về kết quả 201 Created cho người dùng A (đây là đánh đổi giữa tính khả dụng và độ trễ của nhật ký phụ trợ).

Bước 4: Task Service trả về phản hồi HTTP 201 cho người dùng A. Toàn bộ ba bước trên hoàn tất trong khoảng 50–100 mili giây trong môi trường local.

Bước 5: Song song với việc người dùng A nhận phản hồi, RabbitMQ định tuyến sự kiện vào hai hàng đợi: `notification.q` (vì khóa khớp pattern `task.*` của hàng đợi Notification) và `collab.activity.q` (vì khóa khớp pattern `task.*` của hàng đợi Collaboration).

Bước 6: Notification Service tiêu thụ sự kiện từ `notification.q`. Nó tra cứu địa chỉ thư điện tử và tên hiển thị của người được giao (qua REST đến User Service hoặc qua cache), tạo một bản ghi mới trong bảng notifications, đẩy thông báo realtime qua WebSocket đến thiết bị của người được giao, và gửi thư điện tử tóm tắt.

Bước 7: Collaboration Service tiêu thụ sự kiện từ collab.activity.q. Nó tạo một bản ghi mới trong bảng activity_logs với target_type = TASK, action = CREATED, actor_id = <mã người tạo> và payload chứa snapshot ban đầu của công việc.

Toàn bộ phần xử lý phụ trợ (bước 5 đến bước 7) thường hoàn tất trong vòng vài trăm mili giây kể từ thời điểm người dùng A nhận phản hồi, nhưng quan trọng là nó không cản trở phản hồi đó. Nếu Notification Service đang quá tải, Collaboration Service vẫn ghi nhật ký bình thường. Nếu cả hai đều down, thông điệp vẫn được giữ trong hàng đợi RabbitMQ và sẽ được xử lý khi service hồi phục — không có thông tin nào bị mất.

3.6.6. Cơ chế đảm bảo độ tin cậy

Để đảm bảo không xảy ra mất mát dữ liệu sự kiện hoặc xử lý sai lệch, TaskFlow áp dụng đồng thời tám cơ chế kiểm soát độ tin cậy từ đầu cuối đến đầu cuối. Trong đó, cơ chế tiêu thụ trùng lặp (Idempotent Consumer) đóng vai trò then chốt: mỗi dịch vụ khi nhận thông điệp sẽ tiến hành kiểm tra event_id trong bộ đệm Redis. Nếu mã định danh này đã tồn tại, thông điệp sẽ bị bỏ qua lập tức, bảo đảm nghiệp vụ chỉ được thực thi đúng một lần ngay cả khi RabbitMQ thực hiện gửi lại tin nhắn nhiều lần do sự cố mất kết nối mạng.

Bảng 3.6.5. Các cơ chế bảo đảm độ tin cậy của truyền tin

Cơ chế	Cấu hình	Ý nghĩa
Publisher confirm	publisher-confirm-type: correlated	Broker xác nhận đã nhận thông điệp; nếu không có ack, producer biết để retry
Mandatory flag	mandatory: true	Phát hiện sớm khi không có hàng đợi nào match khóa định tuyến
Persistent message	delivery_mode = 2	Thông điệp được lưu xuống đĩa, sống sót qua broker restart
Durable queue	durable: true	Hàng đợi tồn tại sau restart

Cơ chế	Cấu hình	Ý nghĩa
Manual acknowledge	acknowledge-mode: manual	Consumer chỉ ack sau khi xử lý xong, tránh mất khi consumer crash giữa chừng
Retry tự động	3 lần với backoff 1s / 5s / 15s	Tự động phục hồi từ các lỗi tạm thời
DLQ (Dead Letter Queue)	Mỗi hàng đợi có DLQ tương ứng	Thông điệp thất bại không bị mất, giữ lại để điều tra
Idempotent consumer	Khử trùng lặp theo event_id qua Redis (TTL 24h)	Chấp nhận at-least-once delivery, đảm bảo xử lý đúng nghiệp vụ một lần

3.6.7. Quản lý phiên bản schema sự kiện

Sự tiến hóa schema sự kiện theo thời gian được quản lý qua trường `schema_version` trong `envelope`. Các thay đổi tương thích ngược (thêm trường mới) sẽ giữ nguyên phiên bản, giúp consumer cũ bỏ qua trường lạ mà không bị lỗi. Các thay đổi phá vỡ tương thích yêu cầu nâng phiên bản sự kiện và thực hiện chiến lược phát hành song song cả hai phiên bản trong giai đoạn chuyển tiếp để nâng cấp hệ thống cuốn chiếu mà không gây gián đoạn.

3.7. Thiết kế WebSocket Realtime

3.7.1. Đặt vấn đề — Tại sao cần truyền thông realtime

Trong hệ thống TaskFlow, trải nghiệm tương tác tức thời là yêu cầu bắt buộc đối với hai nghiệp vụ chính: nhận thông báo cá nhân và đồng bộ bảng Kanban. Phương pháp truyền thông như HTTP Polling bị loại bỏ do tạo tải truy vấn vô ích cực kỳ lớn lên hệ thống (lên tới 95% số request rỗng) và độ trễ cao không đáp ứng được thao tác kéo thả mượt mà. Do đó, hệ thống xây dựng kênh truyền thông tin hai chiều liên tục sử dụng giao thức WebSocket, cho phép máy chủ chủ động đẩy các sự kiện thay đổi dữ liệu xuống client với độ trễ thấp dưới mức giây.

3.7.2. Lựa chọn công nghệ — WebSocket kết hợp STOMP

Hệ thống lựa chọn sử dụng giao thức WebSocket kết hợp sub-protocol STOMP để quản lý luồng tin nhắn và phân kênh theo đường dẫn rõ ràng như `/topic/*`

(broadcast) và /queue/* (private). Phía client tích hợp @stomp/stompjs kết hợp với sockjs-client làm lớp fallback tự động sang kỹ thuật long-polling nếu môi trường mạng của người dùng chặn kết nối WebSocket thuần. Toàn bộ hạ tầng quản lý phiên và xử lý kết nối realtime được tập trung tại Notification Service nhằm đơn giản hóa việc quản lý trạng thái kết nối và tối ưu hóa tài nguyên mạng.

3.7.3. Cấu hình endpoint và message broker

Notification Service đăng ký một endpoint WebSocket duy nhất tại /ws/notifications hỗ trợ cả WebSocket thuần và SockJS fallback. Spring Boot cấu hình một in-memory broker đơn giản với tiền tố /topic/ cho các kênh broadcast công khai và /queue/ cho các kênh private cá nhân. Để scale ngang hệ thống chạy nhiều instance trong thực tế sản xuất, broker in-memory này sẽ được thay thế bằng các broker chuyên dụng bên ngoài như RabbitMQ STOMP plugin hoặc Redis Pub/Sub.

3.7.4. Cơ chế xác thực qua handshake

Việc xác thực được thực hiện một lần duy nhất vào thời điểm HTTP Handshake ban đầu nâng cấp giao thức. Client gửi token JWT qua header hoặc query parameter ?token=... Một JwtHandshakeInterceptor trên server sẽ xác minh chữ ký JWT và thời hạn sử dụng. Nếu hợp lệ, định danh người dùng được liên kết vào phiên làm việc dưới dạng Principal thông qua UserHandshakeHandler. Phiên WebSocket được cấu hình thời gian sống bằng đúng thời hạn còn lại của JWT, bảo đảm server sẽ chủ động đóng kết nối khi JWT hết hạn và buộc client phải kết nối lại bằng token mới.

3.7.5. Hai loại destination chính

Hệ thống sử dụng hai kênh destination chính được tổng hợp tại Bảng 3.7.1. Đối với thông báo cá nhân (/user/queue/notifications), Spring tự động điều hướng tin nhắn đến đúng Principal định danh người nhận trên mọi thiết bị đang mở. Đối với đồng bộ bảng Kanban (/topic/board/{boardId}), tin nhắn được broadcast đồng loạt cho mọi client đang xem bảng. Để đảm bảo an toàn thông tin, một StompChannelInterceptor sẽ chạy trước mỗi frame SUBSCRIBE để kiểm tra quyền thành viên dự án của client tại Project Service; các yêu cầu đăng ký trái phép sẽ bị từ chối ngay lập tức.

Bảng 3.7.5. Các destination WebSocket trong hệ thống

Destination	Loại	Đối tượng nhận	Mục đích
/user/queue/notifications	Private (Spring ánh xạ thành /user/{userId}/queue/notifications)	Một người dùng cụ thể	Thông báo cá nhân
/topic/board/{boardId}	Public broadcast trong nhóm	Mọi thành viên đang xem bảng	Đồng bộ thay đổi bảng Kanban

3.8. Thiết kế phân quyền

3.8.1. Mô hình phân quyền RBAC theo dự án

Hệ thống TaskFlow áp dụng mô hình phân quyền dựa trên vai trò theo ngữ cảnh dự án, thay vì gán vai trò cố định cấp hệ thống. Quyền hạn của người dùng được xác định dựa trên cặp định danh (user_id, project_id), cho phép một tài khoản nắm giữ các vai trò khác nhau (như Owner, Editor hay Viewer) trên từng dự án cụ thể. Cách tiếp cận này phản ánh chính xác cấu trúc làm việc nhóm thực tế, tăng độ an toàn bằng cách cô lập phạm vi ảnh hưởng của quyền hạn, và cho phép quản lý thành viên linh hoạt mà không cần đồng bộ trạng thái tài khoản toàn cục.

3.8.2. Hệ thống năm vai trò trong dự án và hai vai trò toàn cục

Hệ thống định nghĩa năm vai trò cấp dự án (tổng hợp trong Bảng 3.8.1) với thứ tự thừa kế quyền hạn: OWNER $\supset$ ADMIN $\supset$ EDITOR $\supset$ COMMENTER $\supset$ VIEWER. Ngoài ra, hệ thống còn áp dụng hai vai trò toàn hệ thống: GUEST (người dùng đã đăng nhập nhưng không thuộc dự án đang xét) và ANONYMOUS (người dùng chưa đăng nhập, chỉ được gọi các API public như đăng nhập/đăng ký).

Bảng 3.8.2. Năm vai trò trong phạm vi dự án

Vai trò	Tên tiếng Việt	Quyền chính
OWNER	Chủ sở hữu	Toàn quyền với dự án, bao gồm xóa dự án và chuyển quyền sở hữu cho người khác

Vai trò	Tên tiếng Việt	Quyền chính
ADMIN	Quản trị viên	Quản lý thành viên, bảng, sprint, nhãn; không xóa được dự án
EDITOR	Người chỉnh sửa	Tạo, sửa, xóa công việc; tạo bảng, cột; tải tệp đính kèm; quản lý phụ thuộc
COMMENTER	Người bình luận	Xem nội dung dự án và đăng bình luận; không sửa được công việc
VIEWER	Người xem	Chỉ đọc — không thực hiện được thao tác thay đổi nào

3.8.3. Cơ chế kiểm tra hai cấp

Để tối ưu hóa hiệu năng và bảo mật, quy trình kiểm soát quyền truy cập của hệ thống được chia làm hai cấp rõ rệt. Ở cấp một (API Gateway), hệ thống xác thực token JWT của client, bóc tách thông tin người dùng và chuyển tiếp dưới dạng các HTTP headers chuẩn hóa (X-User-Id, X-Username, X-User-Email) vào mạng nội bộ Docker. Ở cấp hai (từng Microservice nghiệp vụ), dịch vụ nhận yêu cầu đọc mã định danh từ header, truy vấn vai trò của người dùng đối với dự án chứa tài nguyên cần thao tác (sử dụng cache Redis để giảm thiểu độ trễ mạng), và so sánh trực tiếp với mức quyền yêu cầu của endpoint. Các service downstream tin tưởng hoàn toàn vào headers của Gateway vì chúng được cô lập hoàn toàn trong mạng nội bộ container và Gateway tự động làm sạch mọi header giả mạo từ client gửi lên.

3.8.4. Ma trận quyền chi tiết theo Service

Mục này tổng hợp ma trận quyền cho các endpoint trọng yếu của bốn service nghiệp vụ chính, theo cách trình bày: dòng là endpoint, cột là vai trò, ô đánh dấu "có" nghĩa là vai trò đó được phép thực hiện endpoint.

Bảng 3.8.4.1. Bảng ma trận quyền cho các thao tác trên Project và Member

Endpoint	OWNER	ADMIN	EDITOR	COMMENTER	VIEWER	GUEST
POST /projects (tạo dự án mới)	–	–	–	–	–	có (caller trở thành OWNER)
GET /projects/{id}	có	có	có	có	có	không
PUT /projects/{id}	có	có	không	không	không	không
DELETE /projects/{id}	có	không	không	không	không	không
POST /projects/{id}/transfer-ownership	có	không	không	không	không	không
POST /projects/{id}/members (thêm thành viên)	có	có	không	không	không	không
PATCH role thành viên	có	có ⁽¹⁾	không	không	không	không

Endpoint	OWNER	ADMIN	EDITOR	COMMENTER	VIEWER	GUEST
DELETE thành viên	có	có (²)	không	không	không	không

ADMIN chỉ được đổi role giữa EDITOR, COMMENTER, VIEWER. Việc nâng/hạ role lên ADMIN hoặc OWNER chỉ OWNER được phép. ADMIN không được xóa OWNER hay ADMIN khác. OWNER không thể tự xóa bản thân khỏi dự án — phải chuyển quyền sở hữu trước.

Bảng 3.8.4.2 Bảng ma trận quyền cho các thao tác trên Board và List

Endpoint	OWNER	ADMIN	EDITOR	COMMENTER	VIEWER
POST board hoặc list	có	có	có	không	không
PUT board hoặc list	có	có	có	không	không
DELETE board	có	có	không	không	không
DELETE list	có	có	có	không	không
GET board hoặc list	có	có	có	có	có

Bảng 3.8.4.3 Bảng Ma trận quyền cho các thao tác trên Task

Endpoint	OWNER	ADMIN	EDITOR	COMMENTER	VIEWER
POST /tasks (tạo công việc)	có	có	có	không	không
GET /tasks/{id}	có	có	có	có	có
PUT /tasks/{id}	có	có	có	không	không
DELETE /tasks/{id}	có	có	có ⁽³⁾	không	không
POST /tasks/{id}/move	có	có	có	không	không
POST /tasks/{id}/watch (theo dõi)	có	có	có	có	có

EDITOR chỉ được xóa công việc do chính mình tạo hoặc được giao; ADMIN và OWNER xóa được mọi công việc.

Bảng 3.8.5. Ma trận quyền cho các thao tác trên Comment và Attachment

Endpoint	OWNER	ADMIN	EDITOR	COMMENTER	VIEWER
POST comment	có	có	có	có	không
GET comment	có	có	có	có	có
PUT comment	author hoặc ADMIN+	có	author	author	không

Endpoint	OWNER	ADMIN	EDITOR	COMMENTER	VIEWER
DELETE comment	author hoặc ADMIN+	có	author	author	không
POST attachment (tải lên tệp)	có	có	có	không	không
GET attachment	có	có	có	có	có
DELETE attachment	uploader hoặc ADMIN+	có	uploader	không	không

Có thể thấy điểm đặc biệt của Comment và Attachment là sự xuất hiện của hai khái niệm "author" (tác giả bình luận) và "uploader" (người tải lên tệp): người tạo ra tài nguyên luôn được quyền sửa hoặc xóa tài nguyên đó, không phụ thuộc vai trò chung trong dự án. Cơ chế này phản ánh đúng kỳ vọng về quyền tự chủ với nội dung mình tạo ra.

3.8.5. Triển khai kiểm tra phân quyền trong code

Để tránh việc kiểm tra phân quyền lặp lại nhiều lần ở mỗi endpoint và dễ sai sót, mỗi service đóng gói logic kiểm tra trong một thành phần dùng chung tên là `AuthorizationService`. Thành phần này cung cấp hai phương thức cốt lõi.

Phương thức thứ nhất, `requireRole(projectId, userId, minRole)`, là phương thức kiểm tra cơ bản. Nó tra cứu vai trò của người dùng trong dự án thông qua REST đến Project Service (với hỗ trợ cache), sau đó so sánh với vai trò tối thiểu yêu cầu. Nếu người dùng không phải thành viên, phương thức ném ngoại lệ `ForbiddenException("not_a_member")`. Nếu vai trò thấp hơn yêu cầu, ngoại lệ `ForbiddenException("insufficient_role")` được ném ra. Ngoại lệ này được bộ xử lý ngoại lệ chung của hệ thống chuyển thành phản hồi HTTP 403.

Phương thức thứ hai, `requireSelfOrAdmin(projectId, userId, resourceOwnerId)`, được dùng cho các trường hợp mà người tạo ra tài nguyên có quyền thao tác trên chính tài nguyên đó. Logic là: nếu người dùng hiện tại trùng với người sở hữu tài nguyên thì

cho phép ngay; ngược lại, yêu cầu vai trò ADMIN trở lên. Ví dụ điển hình là sửa và xóa bình luận: tác giả luôn được phép, còn người khác phải là ADMIN hoặc OWNER.

Quy ước trong code là mọi phương thức controller hoặc facade thực hiện thao tác có thay đổi dữ liệu đều phải gọi một trong hai phương thức trên ngay ở dòng đầu tiên, trước khi đung vào nghiệp vụ. Quy ước này được áp dụng nhất quán giúp giảm rủi ro quên kiểm tra phân quyền — một loại lỗi nghiêm trọng nhưng khó phát hiện bằng review code thủ công.

3.8.6. Đồng bộ vai trò qua sự kiện

Như đã trình bày ở mục 3.5.5, vai trò được cache trong Redis với thời gian sống 5 phút để giảm tải các cuộc gọi REST đến Project Service. Tuy nhiên, riêng trong bối cảnh phân quyền, cache TTL 5 phút là quá dài: nếu một thành viên bị xóa khỏi dự án do vi phạm, trong vòng 5 phút sau họ vẫn có thể thực hiện các thao tác như thể vẫn còn là thành viên. Đây là rủi ro bảo mật không thể chấp nhận.

Giải pháp là cơ chế vô hiệu hóa cache qua sự kiện. Mỗi khi Project Service thay đổi vai trò thành viên (thêm, xóa, đổi role), nó phát hành sự kiện tương ứng (project.member.added, project.member.removed, project.member.role_changed) qua RabbitMQ. Cả ba service tiêu thụ là Task, Collaboration và Notification đều có một consumer chuyên dụng cho các sự kiện này; consumer chỉ làm một việc duy nhất là xóa khóa cache liên quan trong Redis của service mình.

Nhờ đó, độ trễ giữa thay đổi vai trò ở Project Service và sự áp dụng vai trò mới ở các service khác chỉ rơi vào khoảng 100 đến 500 mili giây — về mặt thực tế là tức thì, đủ nhanh để loại bỏ rủi ro bảo mật. Hệ thống vẫn giữ được lợi ích về hiệu năng của cache trong khi không sacrifice tính kịp thời khi cần thiết.

3.8.7. Các tình huống biên trong phân quyền

Một số tình huống biên đặc biệt cần được thiết kế cẩn thận để hệ thống xử lý đúng và nhất quán. Bảng 3.8.6 tổng hợp các tình huống này cùng cách xử lý.

Bảng 3.8.7. Các tình huống biên trong phân quyền và cách xử lý

Tình huống	Cách xử lý
OWNER muốn rời dự án mà không xóa dự án	Bị chặn — yêu cầu chuyển quyền sở hữu cho thành viên khác trước, rồi mới rời

Tình huống	Cách xử lý
Xóa dự án khi còn nhiều công việc, bình luận, tệp	Soft delete cascade qua sự kiện project.deleted — Task xóa công việc, Collaboration xóa bình luận và tệp, Notification dọn thông báo liên quan
Hai người cùng kéo một công việc gần như đồng thời	Khóa lạc quan qua trường version — người gửi yêu cầu sau nhận HTTP 409 và phải tải lại
Vai trò bị hạ từ EDITOR xuống VIEWER khi đang mở form sửa	Lần submit tiếp theo nhận 403; giao diện chuyển sang chế độ chỉ đọc
Đang kết nối WebSocket khi bị xóa khỏi dự án	Sự kiện project.member.removed kích hoạt vô hiệu cache; lần gọi API tiếp theo nhận 403, đồng thời server đóng các phiên WebSocket liên quan
Reset password token bị tái sử dụng	Cờ used = true được đặt sau lần đầu thành công; mọi nỗ lực tiếp theo bị từ chối
JWT bị lộ	Refresh token được rotate (cấp mới mỗi lần dùng); chức năng đăng xuất xóa refresh token khỏi Redis nhằm thu hồi quyền truy cập

3.8.8. Tổng kết

Thiết kế phân quyền của TaskFlow kết hợp ba ý tưởng then chốt. Một là mô hình RBAC theo dự án phù hợp với đặc thù người dùng có vai trò khác nhau ở các dự án khác nhau. Hai là cơ chế hai cấp kiểm tra (gateway xác thực, service phân quyền) phù hợp với kiến trúc microservice — không lặp lại logic JWT ở mọi service nhưng vẫn bảo đảm an toàn nhờ kiểm soát chặt mạng nội bộ. Ba là chiến lược cache kết hợp invalidate qua sự kiện, vừa giải quyết được vấn đề hiệu năng khi tần suất kiểm tra quyền rất cao, vừa không hi sinh tính kịp thời khi vai trò thay đổi.

Cùng với năm tiêu mục đã trình bày, từ thiết kế cơ sở dữ liệu (3.4), thiết kế giao tiếp REST (3.5), thiết kế kênh truyền tin bất đồng bộ (3.6), thiết kế truyền thông

realtime (3.7) đến thiết kế phân quyền (3.8), Chương 3 đã trình bày trọn vẹn các quyết định thiết kế cốt lõi cho phần hạ tầng kết nối và an ninh của hệ thống TaskFlow.

CHƯƠNG 4. XÂY DỰNG HỆ THỐNG

4.1. Môi trường phát triển

Để đảm bảo tính đồng nhất hoàn toàn giữa các thành viên trong đội ngũ phát triển và giảm thiểu tối đa rủi ro xung đột môi trường khi đưa ứng dụng lên các máy chủ Production, toàn bộ các thành phần công nghệ và thư viện của TaskFlow đã được chuẩn hóa phiên bản chi tiết:

Bảng 4.1 Danh mục Đặc tả Công nghệ hệ thống TaskFlow

Thành phần	Công nghệ / Thư viện	Phiên bản	Mô tả chức năng
Ngôn ngữ & Framework	Java, Spring Boot	Java 17, Boot 3.3.5	Xây dựng logic xử lý cốt lõi tại Backend, tận dụng các tính năng tối ưu bộ nhớ của Java Lts và cấu hình tự động của Spring Boot 3.
Service Discovery	Spring Cloud Netflix Eureka	2023.0.3	Cung cấp máy chủ quản lý danh bạ dịch vụ, hỗ trợ các cơ chế đăng ký và định vị vị trí dịch vụ tự động.
API Gateway	Spring Cloud Gateway	2023.0.3	Lớp cổng vào tập trung, xử lý bài toán định tuyến, lọc bảo mật JWT và giới hạn lưu lượng truy cập.
Message Broker	RabbitMQ	3.12+	Trạm luân chuyển thông điệp bất đồng bộ, đóng vai trò nền tảng cho kiến trúc Event-Driven và điều phối luồng Saga.

In-memory Cache	Redis	7-alpine	Cơ sở dữ liệu bộ nhớ đệm, phục vụ lưu trữ khóa lũy đẳng, danh sách đen token, và tối ưu hóa tốc độ kiểm tra quyền truy cập.
Database	PostgreSQL	15-alpine	Hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, triển khai lưu trữ dữ liệu có cấu trúc riêng biệt cho từng microservice.
Object Storage	MinIO	Mới nhất	Kho lưu trữ đối tượng tương thích chuẩn S3, chuyên trách lưu trữ các tệp tin đính kèm, tài liệu dự án và ảnh đại diện người dùng.
Authentication	JWT (Java JWT)	0.11.5	Thư viện chuyên dụng hỗ trợ sinh mã, mã hóa dữ liệu claims và giải mã xác thực chữ ký mã thông báo bảo mật.
Containerization	Docker, Docker Compose	Engine 24+, v2	Đóng gói mã nguồn cùng toàn bộ môi trường phụ thuộc thành các Container độc lập, phục vụ triển khai tự động hóa nhanh chóng.

4.2. Xây dựng API Gateway

API Gateway được xây dựng dựa trên framework Spring Cloud Gateway, vận hành dựa trên kiến trúc không đồng bộ (Non-blocking) của dự án Project Reactor và Netty Server, giúp hệ thống có khả năng xử lý hàng vạn kết nối đồng thời với tài nguyên phần cứng tối giản. Gateway được cấu hình chạy tập trung tại cổng hệ thống 8080.

- Cấu hình Định tuyến (Routing Scheme): Hệ thống sử dụng file cấu hình `application.yml` kết hợp các Predicates để điều hướng dòng dữ liệu dựa trên tiền tố của đường dẫn URL. Ví dụ: Các request có mẫu `/api/v1/auth/` sẽ tự động

được tước bỏ tiền tố và chuyển tiếp đến tên dịch vụ logic lb://taskflow-user (User Service), trong khi các request dạng /api/v1/projects/ được định tuyến đến lb://taskflow-project. Cơ chế lb:// giúp tích hợp trực tiếp khả năng cân bằng tải phía Client (Client-side Load Balancing).

- Triển khai Lớp lọc Bảo mật (JwtAuthenticationFilter): Lớp lọc này kế thừa từ AbstractGatewayFilterFactory. Tại đây, phương thức filter() sẽ chặn request, trích xuất chuỗi mã trong HTTP Header Authorization. Sử dụng thư viện JWT, bộ lọc giải mã token, xác thực chữ ký đối xứng với Secret Key. Nếu hợp lệ, bộ lọc áp dụng phương thức ServerWebExchange.mutate() để thêm các thông tin định danh (User ID, Role, Email) vào cấu trúc Request Header mới rồi gọi chain.filter() để chuyển tiếp request xuống tầng dưới.
- Xây dựng Bộ giới hạn lưu lượng (Rate Limiter Filter): Gateway sử dụng cấu hình RedisRateLimiter được cung cấp sẵn bởi Spring Cloud Gateway. Hệ thống thiết lập hai thông số: replenishRate (tốc độ tái sinh token vào giỏ mỗi giây, cấu hình = 60) và burstCapacity (sức chứa tối đa của giỏ token, cấu hình = 100). Cơ chế này định danh đối tượng giới hạn thông qua một lớp hiện thực KeyResolver trả về địa chỉ IP hoặc định danh User ID của người gửi yêu cầu.

4.3. Xây dựng Eureka Service Discovery

Thành phần taskflow-eureka được xây dựng để giải quyết bài toán định vị vị trí dịch vụ trong môi trường phân tán, nơi các container có thể được khởi tạo, tắt đi hoặc thay đổi địa chỉ IP liên tục một cách tự động. Server hoạt động tại cổng mặc định 8761.

Tại các microservices thành viên, thư viện spring-cloud-starter-netflix-eureka-client được tích hợp vào file cấu hình. Khi một dịch vụ khởi động thành công, nó sẽ tự động gửi một request HTTP chứa thông tin về Tên dịch vụ (spring.application.name), Địa chỉ IP nội bộ, và Cổng hoạt động lên Eureka Server để ghi danh vào bảng quản lý vị trí (Service Registry).

Định kỳ 30 giây một lần, mỗi client sẽ gửi một tín hiệu kiểm tra trạng thái sống (Heartbeat) lên Server. Nếu quá thời gian cấu hình (mặc định 90 giây) mà Eureka Server không nhận được tín hiệu từ một client, hệ thống sẽ tự động coi như dịch vụ đó đã gặp sự cố và gạch tên ra khỏi danh bạ, giúp bảo vệ API Gateway khỏi việc định tuyến request vào các dịch vụ đã chết.

4.4. Xây dựng User Service

User Service chịu trách nhiệm quản lý vòng đời tài khoản người dùng, vận hành tại cổng 8081 và kết nối với database taskflow_user.

- Quản lý Mật khẩu: Khi người dùng đăng ký hoặc đổi mật khẩu, hệ thống sử dụng cấu hình bean BCryptPasswordEncoder để thực hiện băm mật khẩu với độ mã hóa mặc định là 10. Luồng khôi phục mật khẩu được hiện thực bằng việc sinh ra một chuỗi UUID ngẫu nhiên gắn với thực thể PasswordReset, thiết lập thời gian hết hạn cụ thể và lưu giữ trực tiếp trong bộ đệm Redis để thực hiện xác thực nhanh khi người dùng nhấn vào liên kết xác nhận từ Email.
- Tích hợp Lưu trữ Đối tượng MinIO: Để quản lý ảnh đại diện (Avatar), service cấu hình một Bean MinioClient kết nối tới cụm máy chủ MinIO. Khi nhận được file ảnh từ Client thông qua định dạng MultipartFile, dịch vụ thực hiện chuẩn hóa lại tên file theo định dạng avatars/user_\${userId}.png, kiểm tra tính hợp lệ của định dạng ảnh, rồi đẩy trực tiếp dữ liệu vào bucket taskflow-avatars bằng phương thức putObject(). URL truy cập trực tiếp của ảnh sau đó được cập nhật lại vào bảng người dùng trong PostgreSQL.

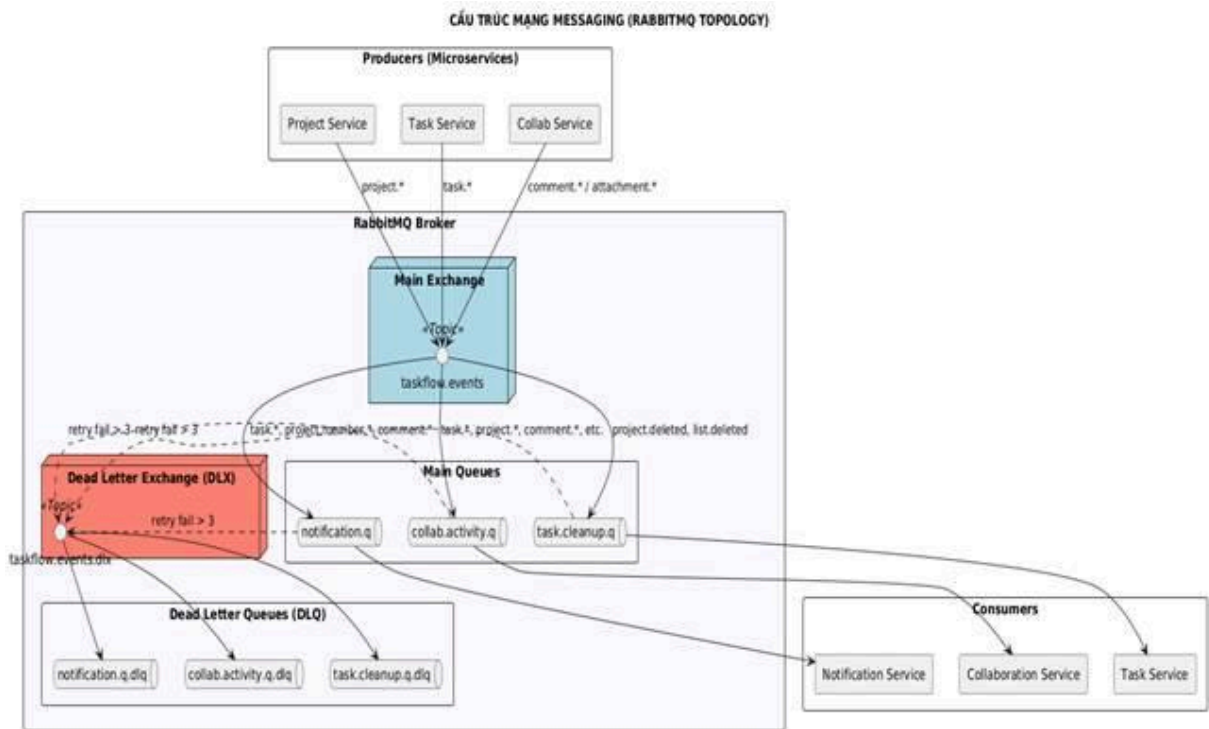
4.5. Xây dựng RabbitMQ Integration

Hệ thống kết nối và quản lý hạ tầng tin nhắn bất đồng bộ bằng giao thức AMQP thông qua Spring AMQP.

- Thiết lập Cấu trúc mạng (Topology): Tại lớp cấu hình RabbitMQConfig, hệ thống định nghĩa một Exchange trung tâm tên là taskflow.events thuộc kiểu TopicExchange. Kiểu Exchange này cho phép điều hướng tin nhắn cực kỳ linh hoạt dựa trên cơ chế so khớp chuỗi định tuyến (Routing Key). Các Queue của các service khác nhau được khai báo và liên kết (Binding) với Exchange này thông qua các từ khóa đại diện. Ví dụ: notification.q được binding với routing key là task.* và project.* để thu thập toàn bộ các sự kiện thay đổi nhằm tạo thông báo.
- Chuẩn hóa cấu trúc Tin nhắn: Để đảm bảo tính đồng nhất dữ liệu và giúp việc giải mã tin nhắn không bị lỗi ở các dịch vụ khác nhau, hệ thống xây dựng một thư viện Maven dùng chung tên là taskflow-events-contract. Thư viện này chứa lớp đối tượng Generic EventEnvelope<T> bao gồm các thông tin bắt buộc: eventId (mã sự kiện), timestamp (thời gian phát sinh), traceId (mã vết hệ thống

phục vụ Distributed Tracing), actorId (người thực hiện), và thuộc tính payload chứa đối tượng dữ liệu nghiệp vụ thực tế. Dữ liệu được cấu hình mã hóa sang dạng chuỗi JSON bằng Jackson2JsonMessageConverter trước khi đẩy vào đường truyền.

- Xử lý Lỗi và Bảo đảm tin nhắn (Dead-Letter Mechanism): Để hệ thống không bị tắc nghẽn khi gặp các tin nhắn lỗi không thể xử lý (Poison Pills), mỗi Queue chính được cấu hình kèm theo các thuộc tính mở rộng: x-dead-letter-exchange trở về một Exchange lỗi đặt tên là taskflow.dlx và x-dead-letter-routing-key. Khi một Consumer xử lý tin nhắn gặp lỗi và kích hoạt ngoại lệ, hệ thống sẽ thực hiện thử lại (Retry) tối đa 3 lần. Nếu vẫn thất bại, tin nhắn sẽ tự động được RabbitMQ chuyển hướng sang Dead-Letter Queue (DLQ). Tại đây, tin nhắn lỗi được lưu trữ cô lập để phục vụ công tác phân tích lỗi và xử lý thủ công từ quản trị viên mà không làm ảnh hưởng đến tiến trình hoạt động của các tin nhắn khác.



Hình 4.5: Sơ đồ cấu trúc mạng (Topology) của RabbitMQ

4.6. Xây dựng WebSocket Realtime

Dịch vụ thông báo Realtime chạy trên cổng 8085 sử dụng cấu hình WebSocketMessageBrokerConfigurer của Spring để thiết lập kênh giao tiếp trực tuyến với Client.

- Cấu hình Message Broker: Phương thức `configureMessageBroker()` được thiết lập để kích hoạt một bộ môi giới tin nhắn đơn giản trong bộ nhớ (Simple Broker) làm việc với các tiền tố đường dẫn `/topic` (dành cho mô hình truyền tin Broadcast) và `/queue` (dành cho mô hình truyền tin User-specific). Đồng thời, tiền tố `/app` được cấu hình để định tuyến các tin nhắn gửi ngược từ Client lên các hàm xử lý `@MessageMapping` tại Server.
- Xây dựng `WebSocketAuthInterceptor`: Do giao thức WebSocket tiêu chuẩn không hỗ trợ tùy biến các HTTP Header tùy thích trong suốt quá trình kết nối dài, việc xác thực phải được thực hiện ngay tại giai đoạn bắt tay kết nối ban đầu thông qua frame `CONNECT` của STOMP. Hệ thống hiện thực lớp `ChannelInterceptor` và can thiệp vào phương thức `preSend()`. Tại đây, interceptor trích xuất chuỗi token từ thuộc tính mã nguồn `stompHeaderAccessor.getNativeHeader("X-Authorization")`, thực hiện kiểm tra tính hợp lệ của JWT thông qua `TokenService`. Nếu token không hợp lệ hoặc đã hết hạn, phương thức sẽ trả về kết quả hủy bỏ, chủ động ngắt kết nối ngay từ tầng mạng để bảo vệ hệ thống khỏi các nguy cơ nghe lén hoặc tấn công khai thác dữ liệu.

4.7. Docker Compose Deployment

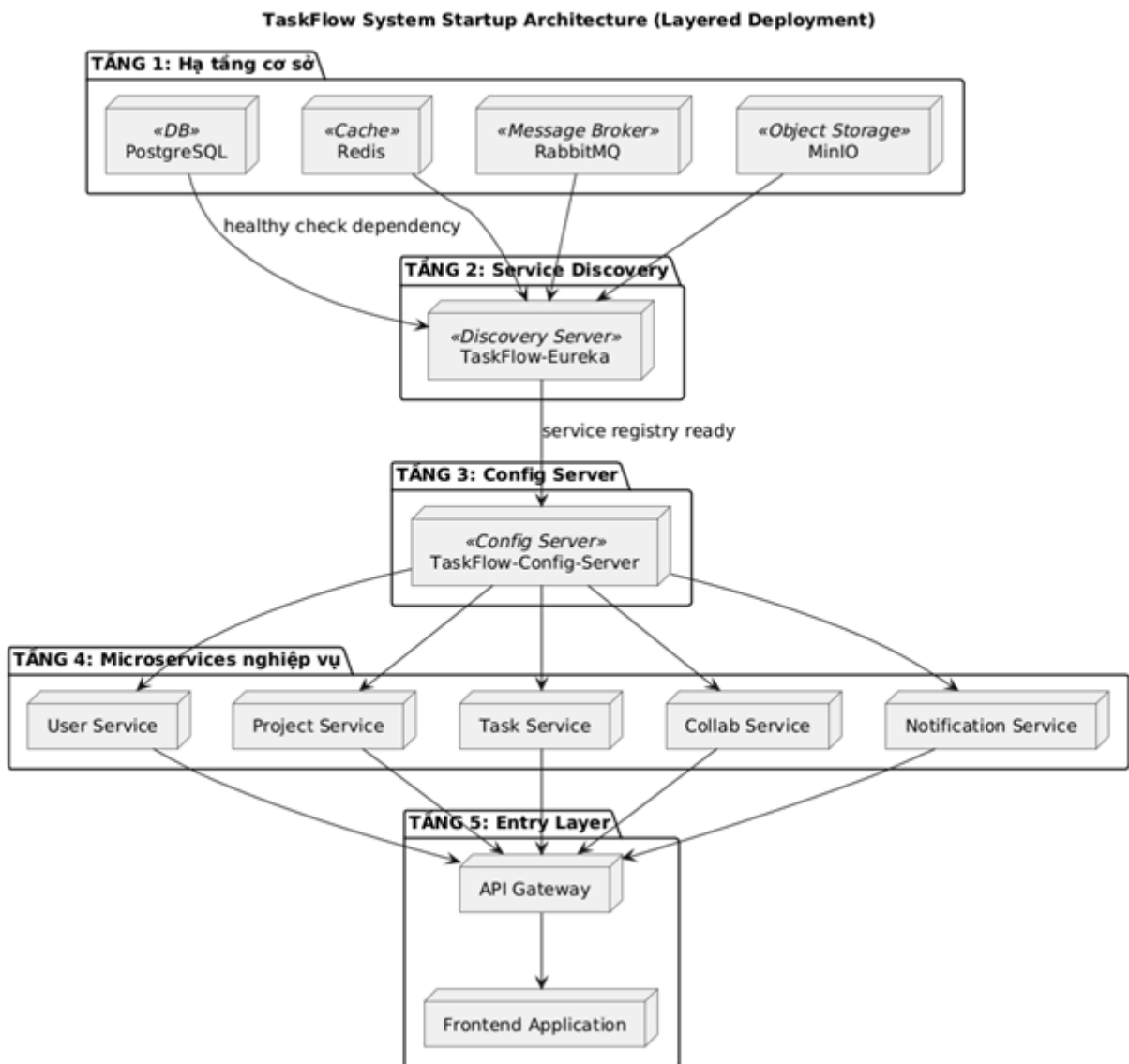
Toàn bộ giải pháp kiến trúc đa dịch vụ của TaskFlow được đóng gói và quản lý vận hành tập trung bằng Docker và công cụ điều phối Docker Compose. Mỗi microservice được viết một file mã nguồn Dockerfile áp dụng kỹ thuật đóng gói nhiều giai đoạn (Multi-stage build) kết hợp tính năng phân lớp Jar (Layered Jars) của Spring Boot 3, giúp tối ưu hóa dung lượng ảnh (Image Size) xuống mức thấp nhất và đẩy nhanh tốc độ build nhờ tận dụng tối đa cơ chế lưu bộ đệm tầng (Layer Caching) của Docker.

File cấu hình hệ thống `docker-compose.yml` khai báo tổng cộng 8 container đại diện cho các dịch vụ ứng dụng cùng 4 container hạ tầng gốc chạy trên một mạng ảo biệt lập nội bộ đặt tên là `taskflow-network`.

- Kiểm tra trạng thái sống (Healthcheck): Một trong những thách thức lớn nhất của việc triển khai microservices bằng Docker Compose là lỗi dịch vụ nghiệp vụ bị sập khi khởi động do các hệ thống hạ tầng (như PostgreSQL hoặc RabbitMQ) tuy container đã bật nhưng tiến trình bên trong chưa sẵn sàng tiếp

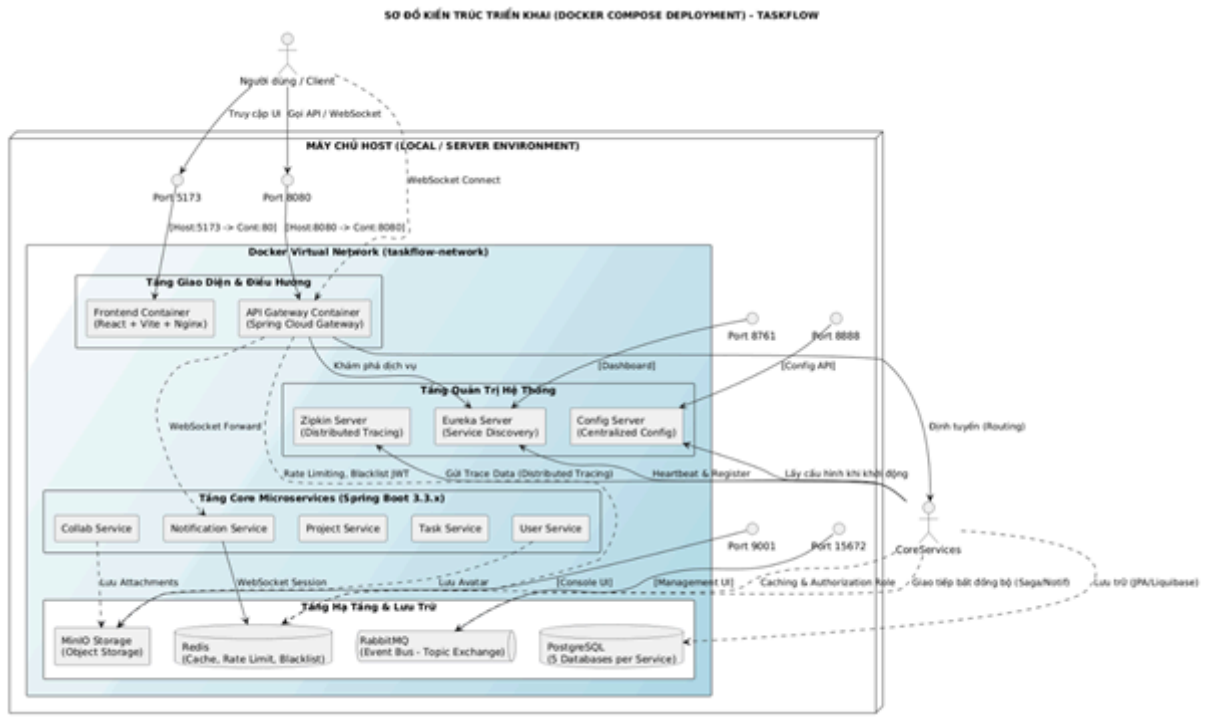
nhận kết nối. Để khắc phục triệt để tình trạng này, TaskFlow thiết lập các lệnh kiểm tra sức khỏe healthcheck nghiêm ngặt cho các container nền tảng:

- + PostgreSQL sử dụng lệnh: `pg_isready -U postgres -d taskflow_user`
- + Redis sử dụng lệnh: `redis-cli ping`
- + Các microservice sử dụng lệnh truy vấn trực tiếp vào công cụ giám sát: `curl -f http://localhost:<port>/actuator/health`
- Quản lý trình tự khởi động: Kết hợp với thuộc tính `depends_on` mở rộng sử dụng điều kiện trạng thái `condition: service_healthy`, Docker Compose điều phối trình tự khởi động của toàn hệ thống theo một kịch bản phân tầng chuẩn xác:



Hình 4.7.1: Trình tự khởi động của toàn hệ thống theo một kịch bản phân tầng chuẩn xác

Giải pháp cấu hình này đảm bảo toàn bộ hệ thống phân tán có khả năng tự động thiết lập liên kết, kết nối đúng đích và vận hành trơn tru chỉ bằng một câu lệnh khởi chạy duy nhất docker-compose up -d.

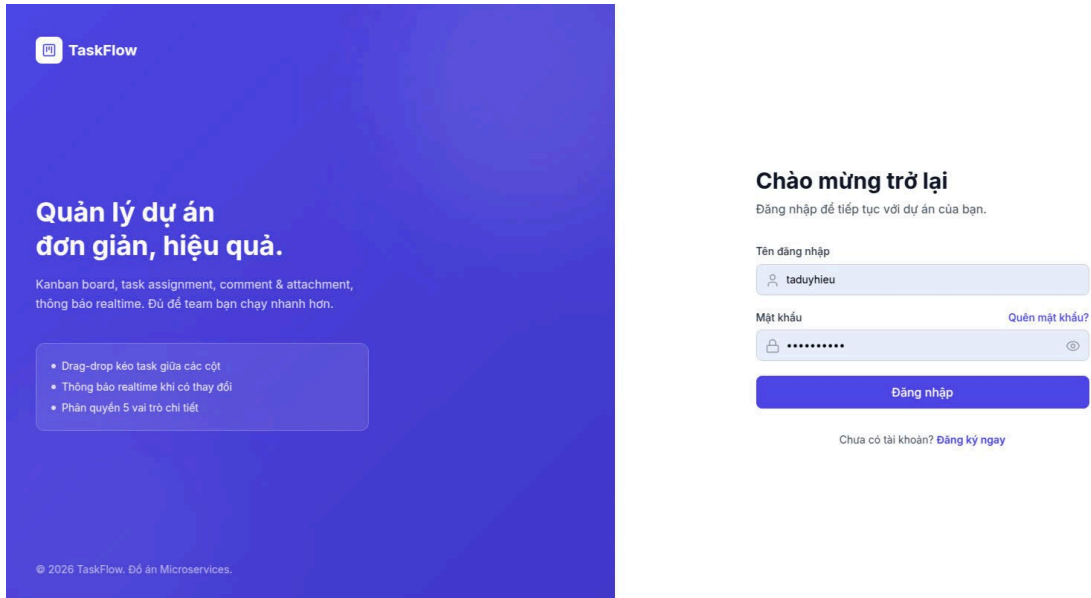


Hình 4.7.2 Sơ đồ triển khai trên Docker

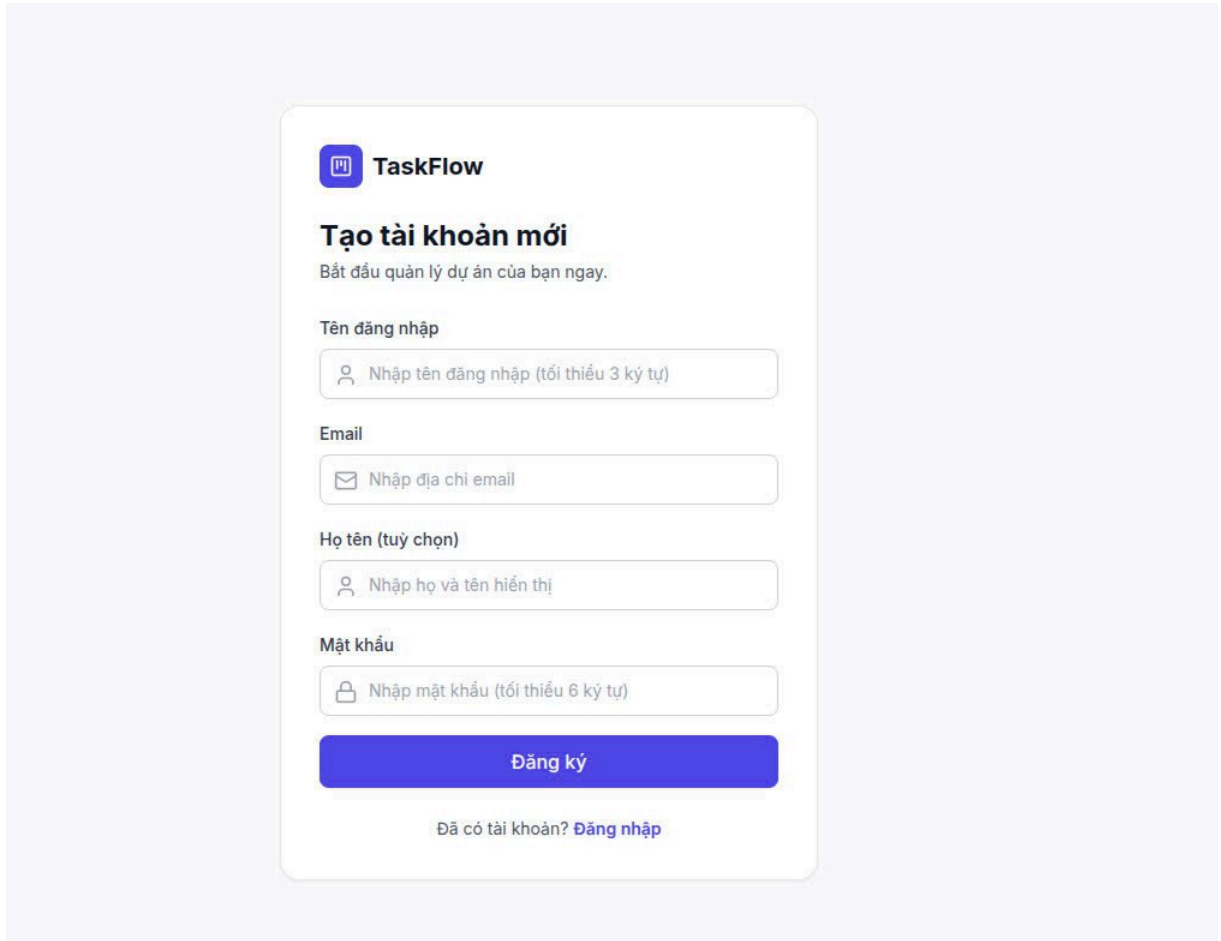
CHƯƠNG 5. DEMO VÀ ĐÁNH GIÁ

5.1 Giao diện hệ thống

5.1.1 Giao diện đăng ký đăng nhập

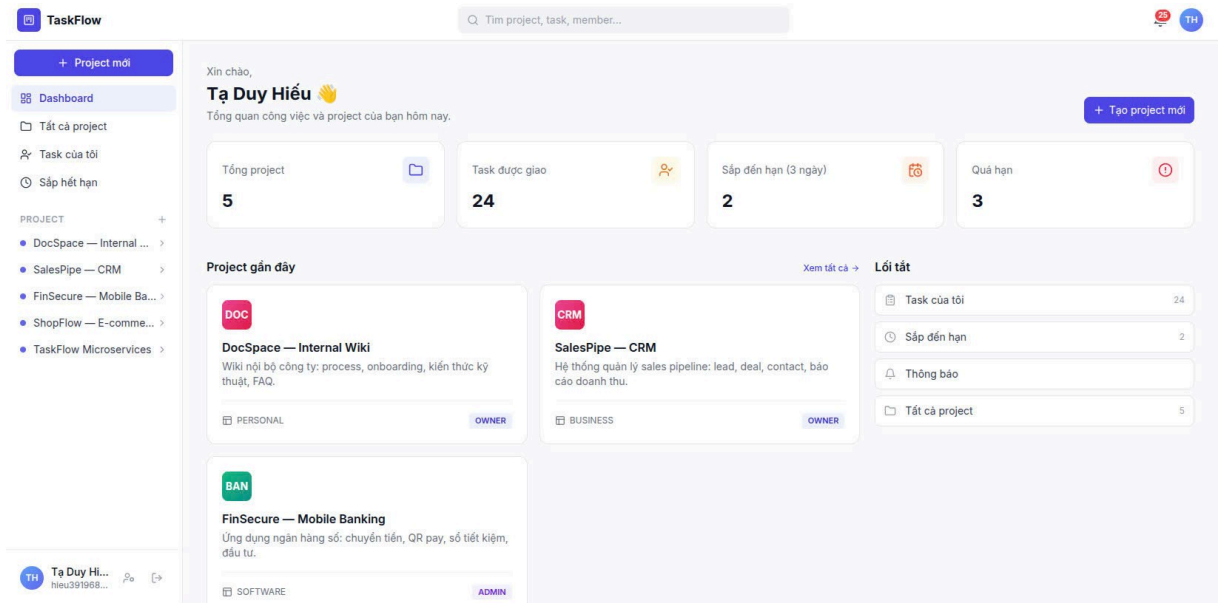


Hình 5.1.1.1 Giao diện đăng nhập



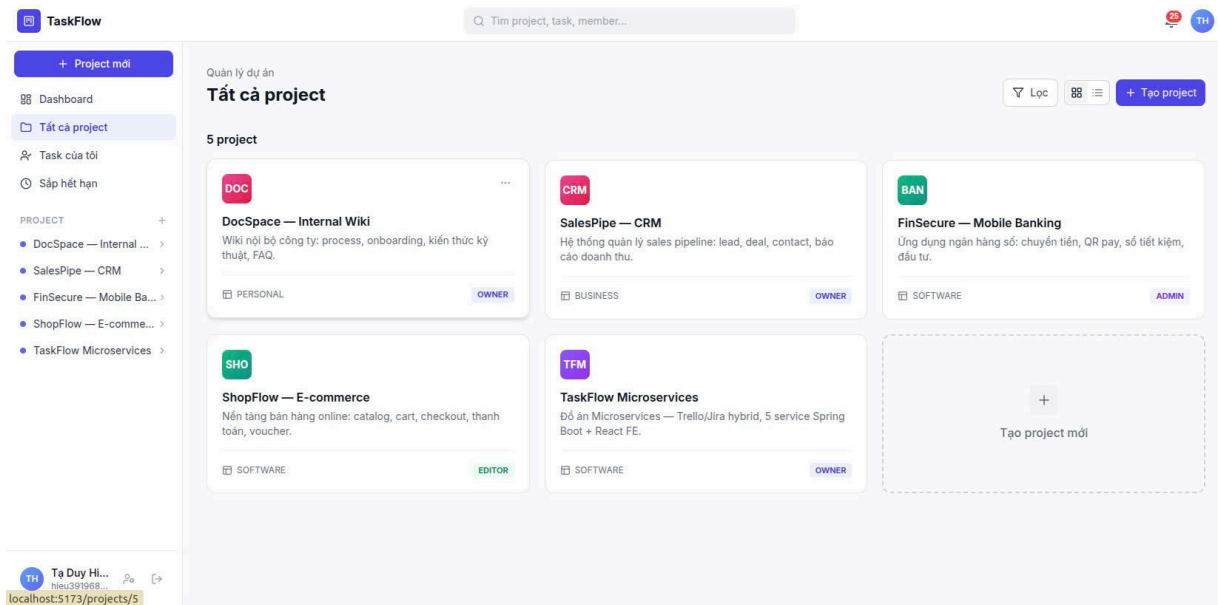
Hình 5.1.1.2 Giao diện đăng ký

5.1.2 Giao diện dashboard

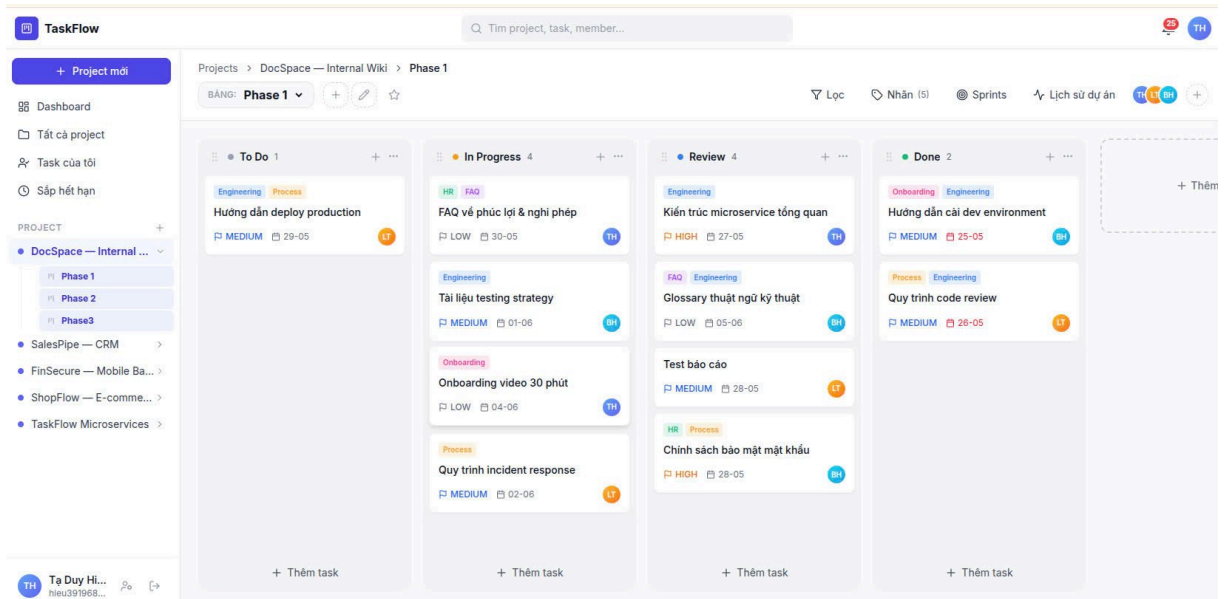


Hình 5.1.2 Giao diện dashboard

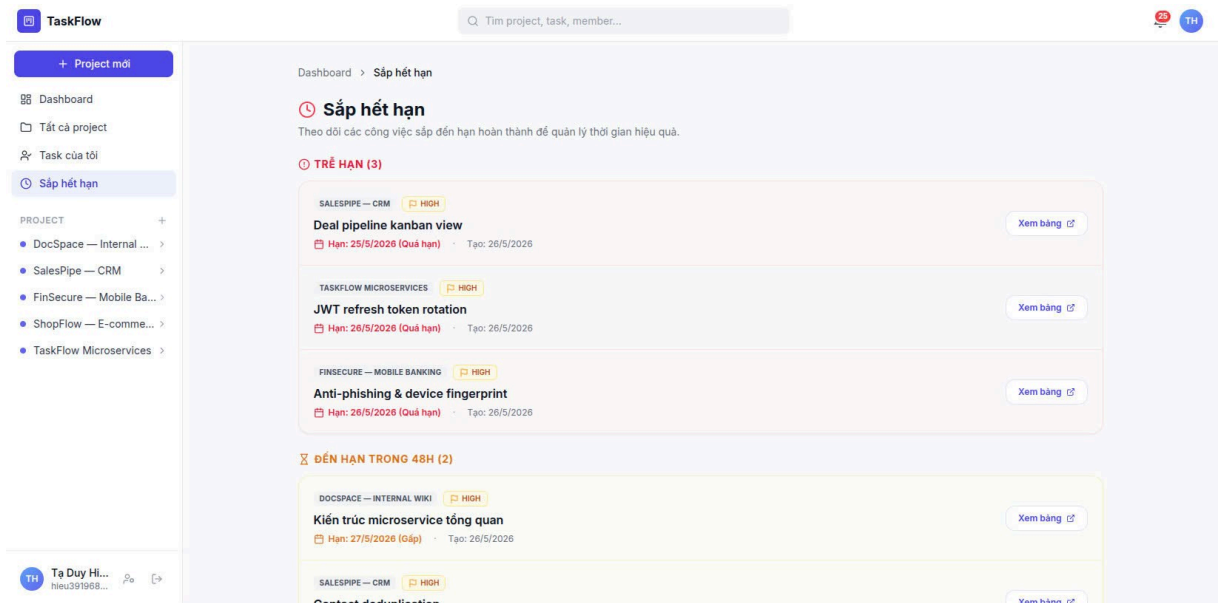
5.1.3 Giao diện quản lý dự án



Hình 5.1.3 Quản lý dự án



5.1.4 Giao diện quản lý task



Hình 5.1.4 Giao diện quản lý task

5.2 Đánh giá hệ thống

5.2.1 Ưu điểm

Hệ thống được xây dựng theo kiến trúc Microservice nên mỗi service có thể hoạt động độc lập và đảm nhận một chức năng riêng biệt. Điều này giúp quá trình phát triển, bảo trì và nâng cấp hệ thống trở nên linh hoạt hơn so với mô hình Monolithic

truyền thống. Khi cần thay đổi một chức năng, lập trình viên chỉ cần chỉnh sửa service liên quan mà không ảnh hưởng đến toàn bộ hệ thống.

Ngoài ra, hệ thống có khả năng mở rộng tốt do các service có thể được scale độc lập tùy theo lưu lượng sử dụng. Ví dụ, Notification Service hoặc Task Service có thể tăng số lượng instance khi lượng request tăng cao mà không cần scale toàn bộ hệ thống. Việc triển khai cũng thuận tiện hơn nhờ sử dụng Docker và Docker Compose giúp quản lý nhiều service dễ dàng.

Bên cạnh đó, hệ thống đã kết hợp nhiều công nghệ hiện đại như RESTful API, RabbitMQ, WebSocket và JWT nhằm hỗ trợ giao tiếp đồng bộ, bất đồng bộ và realtime hiệu quả. Điều này giúp tăng trải nghiệm người dùng và cải thiện khả năng xử lý trong môi trường phân tán.

5.2.2 Nhược điểm

Mặc dù mang lại nhiều lợi ích, kiến trúc Microservice cũng có độ phức tạp cao hơn đáng kể so với kiến trúc Monolithic. Hệ thống cần quản lý nhiều service độc lập cùng lúc nên việc cấu hình, triển khai và giám sát trở nên khó khăn hơn.

Quá trình debug và theo dõi lỗi cũng phức tạp hơn do một request có thể đi qua nhiều service khác nhau trước khi hoàn thành xử lý. Vì vậy, hệ thống cần bổ sung thêm các cơ chế như centralized logging hoặc distributed tracing để hỗ trợ giám sát và phân tích lỗi hiệu quả hơn.

Ngoài ra, việc quản lý dữ liệu phân tán giữa các service là một thách thức lớn trong kiến trúc Microservice. Do mỗi service sở hữu cơ sở dữ liệu riêng nên việc đảm bảo tính nhất quán dữ liệu đòi hỏi phải sử dụng các cơ chế như Eventual Consistency, Saga Pattern hoặc Message Queue. Điều này làm tăng độ phức tạp trong thiết kế và xử lý nghiệp vụ của hệ thống.

KẾT LUẬN

Sau quá trình nghiên cứu và thực hiện, đề tài “TaskFlow – Hệ thống Quản lý Nhiệm vụ và Dự án theo kiến trúc Microservice” đã hoàn thành các mục tiêu cơ bản đề ra. Hệ thống được xây dựng dựa trên kiến trúc Microservice với nhiều service hoạt động độc lập, hỗ trợ giao tiếp đồng bộ thông qua RESTful API và giao tiếp bất đồng bộ thông qua Message Queue. Đây là mô hình kiến trúc hiện đại đang được sử dụng rộng rãi trong các hệ thống phân tán và ứng dụng quy mô lớn hiện nay.

Trong quá trình phát triển hệ thống, nhóm đã nghiên cứu và áp dụng nhiều công nghệ phổ biến như Spring Boot, RabbitMQ, WebSocket, JWT, Docker, Netflix Eureka và Spring Cloud Gateway. Việc kết hợp các công nghệ này giúp hệ thống đảm bảo khả năng mở rộng, tính độc lập giữa các service, hỗ trợ realtime và tăng khả năng chịu lỗi trong môi trường phân tán.

Hệ thống đã triển khai được nhiều chức năng quan trọng phục vụ cho việc quản lý công việc và cộng tác nhóm. Người dùng có thể thực hiện xác thực và phân quyền thông qua JWT nhằm đảm bảo tính bảo mật trong quá trình sử dụng hệ thống. Ngoài ra, hệ thống hỗ trợ quản lý Project, Board và Task giúp người dùng dễ dàng theo dõi tiến độ công việc và tổ chức dự án theo mô hình Kanban. Bên cạnh đó, chức năng cộng tác nhóm và bình luận realtime giúp các thành viên trao đổi thông tin nhanh chóng và đồng bộ hơn trong quá trình làm việc. Hệ thống cũng hỗ trợ gửi Notification thông qua WebSocket kết hợp với RabbitMQ nhằm đảm bảo khả năng xử lý realtime và bất đồng bộ. Việc giao tiếp giữa các service được thực hiện thông qua RESTful API, giúp các service hoạt động độc lập nhưng vẫn có thể phối hợp hiệu quả với nhau. Ngoài ra, toàn bộ các service được triển khai độc lập bằng Docker Compose, giúp việc triển khai và quản lý hệ thống trở nên thuận tiện hơn.

Bên cạnh đó, đề tài còn giúp nhóm hiểu rõ hơn về quy trình thiết kế hệ thống Microservice, cách phân tách service theo domain nghiệp vụ, xử lý giao tiếp giữa các service và quản lý dữ liệu trong môi trường phân tán. Nhóm cũng có cơ hội tiếp cận với các khái niệm quan trọng như Event-Driven Architecture, Service Discovery, API Gateway, Circuit Breaker, Distributed Tracing và Saga Pattern.

Ngoài việc nâng cao kiến thức chuyên môn, quá trình thực hiện đề tài còn giúp các thành viên cải thiện kỹ năng làm việc nhóm, phân tích hệ thống, thiết kế kiến trúc

phần mềm và triển khai ứng dụng thực tế. Đây là những kỹ năng quan trọng đối với quá trình học tập và phát triển nghề nghiệp sau này.

Tuy nhiên, do giới hạn về thời gian và kinh nghiệm thực tế, hệ thống vẫn còn một số hạn chế nhất định như chưa tối ưu hoàn toàn về hiệu năng, chưa triển khai trên môi trường production thực tế và chưa tích hợp đầy đủ các cơ chế monitoring hoặc auto scaling. Mặc dù vậy, đề tài vẫn đáp ứng được các yêu cầu cơ bản của hệ thống Microservice theo đúng đặc tả đã đề ra.

HƯỚNG PHÁT TRIỂN

Trong tương lai, hệ thống TaskFlow có thể tiếp tục được mở rộng và hoàn thiện theo nhiều hướng nhằm nâng cao tính thực tiễn, khả năng mở rộng và hiệu năng xử lý trong môi trường Microservice. Một trong những hướng phát triển quan trọng là triển khai hệ thống trên các nền tảng cloud hoặc môi trường container orchestration như Kubernetes. Việc áp dụng Kubernetes sẽ giúp hỗ trợ auto scaling, rolling update, quản lý container hiệu quả hơn và đảm bảo hệ thống hoạt động ổn định khi lưu lượng truy cập tăng cao.

Bên cạnh đó, hệ thống có thể tích hợp thêm các công cụ centralized logging và monitoring như ELK Stack, Prometheus hoặc Grafana nhằm hỗ trợ giám sát tài nguyên, theo dõi log tập trung và phát hiện lỗi nhanh chóng trong môi trường phân tán. Việc bổ sung các công cụ này sẽ giúp quá trình vận hành, bảo trì và phân tích lỗi trở nên hiệu quả hơn.

Một hướng phát triển khác là tối ưu hiệu năng hệ thống bằng cách sử dụng Redis làm caching layer nhằm giảm tải cho database và tăng tốc độ phản hồi. Đồng thời, hệ thống cũng có thể triển khai thêm các cơ chế như Rate Limiting, API Security hoặc nâng cao khả năng xác thực để cải thiện tính bảo mật và hạn chế các nguy cơ tấn công từ bên ngoài.

Ngoài phiên bản web hiện tại, nhóm có thể phát triển thêm ứng dụng di động trên Android hoặc iOS nhằm hỗ trợ người dùng quản lý công việc thuận tiện hơn trên thiết bị di động. Các chức năng Notification cũng có thể được mở rộng thông qua Email Notification hoặc Push Notification realtime để nâng cao trải nghiệm người dùng.

Về mặt nghiệp vụ, hệ thống có thể bổ sung thêm nhiều chức năng nâng cao như Gantt Chart, Dashboard thống kê trực quan, quản lý tài liệu đính kèm, lịch làm việc hoặc cơ chế nhắc việc tự động. Ngoài ra, việc tích hợp AI Recommendation để gợi ý phân chia công việc hoặc dự đoán tiến độ dự án cũng là một hướng phát triển tiềm năng giúp nâng cao tính thông minh của hệ thống.

Đối với kiến trúc Microservice, hệ thống có thể triển khai hoàn chỉnh hơn các cơ chế nâng cao như Saga Pattern cho distributed transaction, Distributed Tracing với Zipkin, Circuit Breaker với Resilience4j hoặc CQRS và Event Sourcing cho các nghiệp vụ phức tạp. Việc áp dụng đầy đủ các cơ chế này sẽ giúp hệ thống tăng khả

năng chịu lỗi, cải thiện khả năng giám sát và đảm bảo tính ổn định trong môi trường phân tán quy mô lớn.

Việc tiếp tục nghiên cứu và phát triển các hướng trên sẽ giúp hệ thống TaskFlow hoàn thiện hơn, nâng cao khả năng mở rộng, tối ưu hiệu năng và đáp ứng tốt hơn các yêu cầu của các hệ thống quản lý công việc hiện đại trong thực tế.

TÀI LIỆU THAM KHẢO

- [1] Sam Newman, *Building Microservices*, O'Reilly Media, 2021.
[O'Reilly - Building Microservices](#)
- [2] Chris Richardson, *Microservices Patterns*, Manning Publications, 2018.
[Microservices Patterns](#)
- [3] Craig Walls, *Spring Boot in Action*, Manning Publications, 2016.
[Spring Boot in Action](#)
- [4] Martin Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
[Designing Data-Intensive Applications](#)
- [5] [Spring Boot Documentation](#)
- [6] [Spring Cloud Documentation](#)
- [7] RabbitMQ Official Documentation
- [8] [Redis Documentation](#)
- [9] [PostgreSQL Documentation](#)
- [10] [Docker Documentation](#)
- [11] [Resilience4j Documentation](#)
- [12] [Zipkin Documentation](#)
- [13] [Spring Security Documentation](#)
- [14] [JWT Introduction](#)
- [15] [WebSocket Protocol Documentation](#)
- [16] [Apache Kafka Documentation](#)
- [17] [Netflix Eureka Documentation](#)
- [18] [Micrometer Documentation](#)

LINK GITHUB CỦA NHÓM

<https://github.com/taduyhieu2004/taskflow-public>